Gift of the Givers · Relief Management System

APPR6312 · PORTFOLIO OF EVIDENCE

Part 3 — Complete Submission

Testing, Deployment and Handover · every project deliverable (01–12) in one document

DATE	DELIVERABLES
2 November 2026	13

Note to the marker: problem with our Azure account

We have a problem with our Azure account. Because of it, we could not complete the deployment to Azure before this submission: the live web app still shows the Azure "waiting for your content" page, and the deployment stages of the pipeline (C.1 web app, C.2 Azure Function, C.3 SQL database) have not yet run in Azure DevOps. The Azure DevOps access token we had for connecting GitHub to Azure DevOps was rejected (HTTP 401 Unauthorized) when we tested it on 2 November 2026.

Everything needed for the deployment is in place and documented: the pipeline (azure-pipelines.yml and pipelines/templates), the step-by-step Azure set-up and GitHub-to-Azure sync guide (Deliverable 11), the database migration and rollback scripts, and the rollback and restore procedures. The testing work (unit, integration, load, stress and Selenium UI tests) is complete, and its evidence is in Deliverable 09 and in docs/evidence/part3 in the repository.

As soon as the account problem is resolved, a team member will follow Deliverable 11, run the pipeline and add the deployment screenshots listed in Deliverable 09, section 10.

Contents

No.	Deliverable	Page
01	Product Requirements	3
02	Technical Requirements	12
03	Application Flow	19
04	Design Brief	30
05	Backend Schema	38
06	Implementation Plan	45
07	CI/CD and Release	49
08	Professional Engineering Audit	54
08-A	Audit Addendum — Remediation Review	83
09	Part 3 — Testing, Deployment and Handover	91
10	User Manual	108
11	Azure Deployment and GitHub Sync Guide	124
12	Team Contribution Report	134

Gift of the Givers · Relief Management System

DELIVERABLE 01

Product Requirements

What the Relief Management System must do, for whom, and how we know each requirement is met.

DOCUMENT	VERSION	DATE	STATUS
D-01	2.0	2026-11-02	Current — reflects the system after audit remediation

CONTENTS

1. Purpose and vision
2. Users and roles
3. Scope
4. Functional requirements
5. Business rules
6. Non-functional requirements (summary)
7. Assumptions, constraints and open questions
8. Glossary

1. Purpose and vision

The Relief Management System supports a humanitarian non-profit (modelled on the Gift of the Givers Foundation) in three areas:

1. **Raising money transparently.** The public can donate online. Every rand shown as "raised" can be traced to a confirmed payment or a staff-recorded offline contribution.
2. **Mobilising people.** Volunteers apply online. Staff review them and assign them to relief projects.
3. **Accountability to donors.** Donors see their history and download donation certificates for confirmed gifts.

> **Product principle — honesty over polish.** The system never shows a figure, status or document it cannot back with data. Unpaid donations are labelled as such, demonstration content is labelled as demonstration, and statistics are computed from the database.

Context

The project is an academic deliverable (Portfolio of Evidence, "Section B" ERD). It is hosted publicly as a demonstration and uses a real NGO's name for illustration. It is **not affiliated** with that organisation. Payments use the PayFast sandbox until real merchant details are configured.

2. Users and roles

Persona	Role	Goals	Key needs
Public visitor	Anonymous	Learn about projects, give, volunteer, ask a question	Fast, mobile-friendly pages; clear trust signals; no account required to donate
Donor	`Donor` (self-registered)	Track donations, download certificates	Accurate history; certificates only for real payments; privacy
Staff member	`Employee` (granted by staff)	Run projects, review volunteers, oversee donations	Accurate totals; filters and exports; audit trail; no accidental data exposure
Information Officer	Organisation (offline)	Answer privacy requests (POPIA)	Privacy notice, consent records, data export/deletion
PayFast	External system	Notify payment results	Reachable, idempotent webhook

Role rules:

- Self-registration always creates a **Donor**. Staff access is granted only by an existing employee (*Staff Access* page) or by the deployment configuration `StaffAccess:BootstrapEmployeeEmails`.
- One account may hold both roles.

3. Scope

In scope (current release)

- Public information pages, live impact figures, project listing with funding progress.
- Online donations in ZAR, USD and EUR, paid through PayFast in rand.
- Donation receipts, donor dashboard and history, donation certificates (HTML + PDF).
- Volunteer applications, review, approval/rejection and assignment to projects.
- Contact inquiries and their handling.
- Staff management of projects, project updates, offline funds and funding reconciliation.
- Staff access management and an audit log.
- Privacy notice and consent capture.

Out of scope (this release)

- Automatic recurring billing. Monthly *pledges* are recorded, but each payment is made separately.
- Live exchange rates (rates are configured).
- Native mobile apps (the site is responsive).
- Multi-organisation or multi-tenant use, and multiple languages.
- Accounting and general-ledger integration.

4. Functional requirements

Priority uses MoSCoW: **M**ust, **S**hould, **C**ould. Status reflects verified behaviour on 2026-11-02 (see the audit addendum).

4.1 Public site

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-01	Show active relief projects with funding progress	M	Projects list shows title, location, status, raised vs goal in rand; progress capped at 100% visually	Done	`HomeController.ReliefProjects`, smoke test
FR-02	Show impact figures computed from live data	M	No hard-coded statistics; figures update when data changes	Done	`ImpactStatsViewComponent` ; test `HomePage_ShowsLiveFiguresInsteadOfHardCodedStatistics`
FR-03	Publish a privacy notice	M	Notice explains data, purposes, sharing, retention, rights, cookies	Draft	`Views/Home/Privacy.cshtml` ; legal review pending (F-09)
FR-04	Contact form	M	Required name, email, subject, message, consent; stored; confirmation page	Done	test `Contact_RecordsConsentTimestamp`

4.2 Donations and payments

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-10	Donate without an account	M	Anonymous donors can donate; signed-in donors' gifts are linked to their account	Done	`DonationService.CaptureAsync`
FR-11	Choose currency ZAR / USD / EUR	M	Foreign amounts converted to rand at the configured rate; both values stored and shown	Done	test `Donate_ForeignCurrency_IsChargedInRandAtConfiguredRate`
FR-12	Pay securely through PayFast	M	Donor redirected to PayFast with signed fields; card data never touches our servers	Wired	smoke test redirect; end-to-end blocked by F-23
FR-13	Only confirmed payments count	M	Project totals, dashboards and certificates include only `Verified` donations	Done	tests `Donate_RecordsPendingDonation_…`, `Notify_Complete_…`
FR-14	Payment confirmation is safe to repeat	M	Duplicate or replayed notifications never add money twice; wrong amount or merchant is refused	Done	tests `Notify_*`
FR-15	Double-submission safe	M	Submitting the same form twice creates one donation	Done	test `Donate_SameIdempotencyTokenTwice_CreatesOneDonation`
FR-16	Receipt page with live status	M	Found by unguessable reference; shows amount, rand value, project, status	Done	test `Receipt_IsFoundByReferenceNotSequentialId`

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-17	Donations only to active projects	M	Suspended or completed projects are rejected with a message	Done	test `Donate_ToSuspendedProject_IsRejectedWithMessage`
FR-18	Monthly pledge	S	Pledge recorded and clearly described as not auto-billed	Pledge only	F-37; billing in the roadmap
FR-19	Anonymous donation option	S	Name hidden on certificates and staff reports	Done	`TaxCertificatePdf.DonorDisplayName`

4.3 Donor portal

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-20	Donor dashboard	M	Confirmed count and rand total, pending count, recent donations with status	Done	`DonorController.Dashboard`
FR-21	Donation history and details	M	Only the donor's own donations; status visible	Done	ownership filter in queries
FR-22	Donation certificate (HTML + PDF)	M	Only for `Verified` donations owned by the donor (or staff); marked *demonstration* until PBO details are configured	Done	test `Certificates_OnlyForOwnersOfVerifiedDonations`
FR-23	Account self-service	S	Change password, download or delete personal data	Done	ASP.NET Identity default UI

4.4 Staff portal

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-30	Manage relief projects	M	Create/edit title, description, location, status, dates, goal, offline funds, image URL; date and amount validation	Done	test `EditingOfflineFunds_AdjustsTotalByTheDifference`
FR-31	Post project updates	S	Create/edit updates per project	Done	`EmployeeController`
FR-32	Review volunteers	M	Filter by status, view details, approve or reject; applicant notified by email when configured	Done	`SetVolunteerStatusAsync`
FR-33	Assign volunteers to projects	S	Only approved volunteers to active projects; no duplicate assignments	Done	`AssignVolunteer`, unique index
FR-34	Donation oversight	M	Filter by status, paged list, rand totals of paid donations, CSV export	Done	`Donations`, `ExportDonations`
FR-35	Funding reconciliation	M	Per project: offline + verified vs recorded; recalculate or accept the difference as offline, both audited	Done	test `FundingReconciliation_RecalculateUsesOfflinePlusVerifiedOnly`
FR-36	Inquiries	S	Open first, mark handled	Done	`Inquiries`

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-37	Staff access management	M	Grant by email to an existing account; revoke others (not self); audited	Done	test `StaffAccess_GrantIsAuditedAndSelfRevokeIsRefused`
FR-38	Audit log	M	All staff changes, access changes, payment events, exports and certificate access, with actor and details	Done	`AuditService` ; test `Notify_Complete_…` asserts `PaymentVerified`

4.5 Volunteers

ID	Requirement	Priority	Acceptance criteria	Status	Evidence
FR-40	Apply to volunteer	M	Name, email, skills, availability required; phone, city, DOB and emergency contact optional; consent required; nothing invented	Done	tests `Volunteer_*`
FR-41	Age check	S	A date of birth, if given, must be at least 16 years ago	Done	`HomeController.Volunteer`

5. Business rules

ID	Rule
BR-01	A project's **raised** figure = offline funds (staff-recorded) + rand value of its **Verified** donations. Nothing else changes it.
BR-02	A donation is `Pending` from submission until a validated PayFast notification marks it `Verified` (COMPLETE) or `Failed` (any other status).
BR-03	PayFast is charged the rand amount: `AmountZar = round(Amount × rate, 2)`, where the rate is stored with the donation.
BR-04	A certificate exists only for a `Verified` donation. It is visible to its owner and to staff. Anonymous donations show "Anonymous Donor".
BR-05	Certificates are labelled *demonstration* until `Organisation:PboReference` is set and `Organisation:IsDemo` is false.
BR-06	Public registration never grants staff access.
BR-07	Every staff change to data or access is written to the audit log in the same transaction.
BR-08	Personal data is collected only with recorded consent (timestamp per record).
BR-09	Single-donation limit: 1 to 1,000,000 in the chosen currency.

6. Non-functional requirements (summary)

The details, targets and verification methods are in 02 — Technical Requirements.

Area	Requirement
Security	Role-based access enforced on the server; anti-forgery; rate limiting; security headers; no secrets in source
Privacy	POPIA-aligned notice, consent, minimisation, export and deletion
Reliability	Idempotent payments; concurrency-safe totals; health probes
Usability	Responsive (mobile first for public pages); accessible forms (WCAG 2.2 AA target)
Maintainability	Automated tests on every critical path; CI gates; documented setup

7. Assumptions, constraints and open questions

Type	Item
Assumption	The organisation will provide a PayFast merchant account, PBO number and Information Officer before going live.
Constraint	PayFast settles only in rand.
Constraint	Hosting on Azure App Service, with Azure SQL.
Open question	Retention periods for donation, volunteer and inquiry data (draft values in the privacy notice).
Open question	Should monthly pledges become PayFast subscriptions (needs product and legal sign-off)?
Open question	Which exchange-rate source is authoritative for foreign donations?

8. Glossary

Term	Meaning
ITN	PayFast *Instant Transaction Notification*: a server-to-server POST that reports a payment result
PBO	Public Benefit Organisation (South African tax status that enables section 18A certificates)
Offline funds	Money raised outside the website (cash, grants, pledges) and recorded by staff
Verified	A donation whose payment PayFast has confirmed
POPIA	Protection of Personal Information Act, 2013 (South Africa)

Gift of the Givers · Relief Management System

DELIVERABLE 02

Technical Requirements

Architecture, technology, integration contracts and measurable non-functional requirements for the Relief Management System.

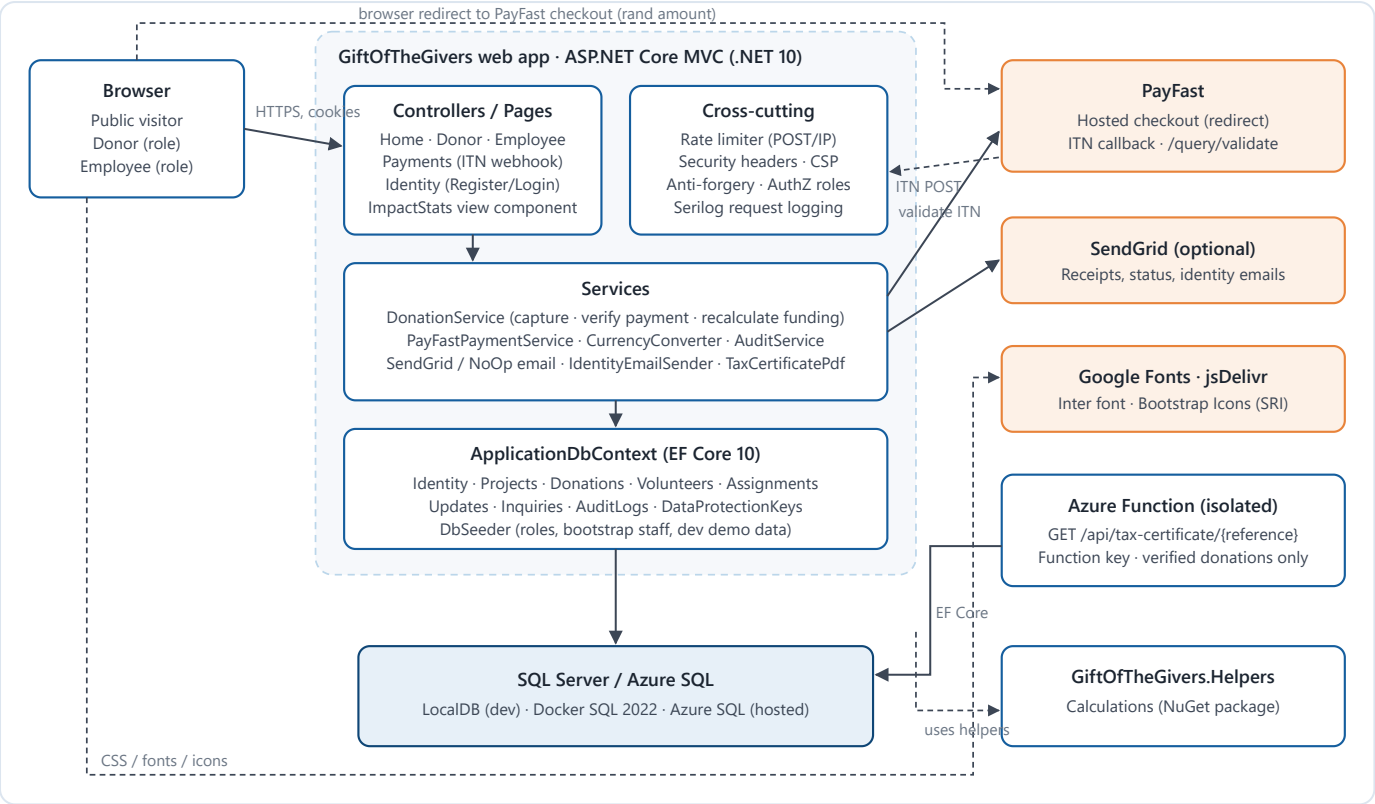
DOCUMENT	VERSION	DATE	STATUS
D-02	2.0	2026-11-02	Current — reflects the system after audit remediation

CONTENTS

1. Architecture overview
2. Technology stack
3. Environments and configuration
4. Integration contracts
5. Non-functional requirements
6. Data requirements
7. Engineering standards
8. Known technical constraints and debt

1. Architecture overview

The system is a **layered ASP.NET Core monolith**: MVC controllers and Razor Pages, an application service layer, and EF Core. It has one satellite Azure Function and one shared class library. This fits the scale: one team, one database, and moderate traffic with spikes during disaster appeals.



System architecture — components, external services and communication paths

Component	Responsibility	Key files
Web app (`GiftOfTheGivers`)	Public site, donor and staff portals, PayFast webhook, Identity	`Program.cs`, `Controllers/*`, `Views/*`
`DonationService`	Capture donations, apply payment notifications, recalculate funding	`Services/DonationService.cs`
`PayFastPaymentService`	Build signed checkout URLs; verify ITN signature and validate with PayFast	`Services/PayFastPaymentService.cs`
`CurrencyConverter`	Convert to rand at configured rates (fails at startup if a rate is missing)	`Services/CurrencyConverter.cs`
`AuditService`	Stage and save audit entries in the caller's transaction	`Services/AuditService.cs`
Email	SendGrid when configured, otherwise no-op; Identity emails via `IdentityEmailSender`	`Services/*Email*`
`TaxCertificatePdf`	PDF certificate (QuestPDF); issuer from configuration	`Services/TaxCertificatePdf.cs`
`DbSeeder`	Migrations (configurable), roles, bootstrap staff, development demo data	`Data/DbSeeder.cs`
Azure Function	Key-protected certificate generation by reference	`GiftOfTheGivers.Functions/*`
Helpers library	Funding and donation calculations, certificate numbering	`GiftOfTheGivers.Helpers/*`

2. Technology stack

Layer	Technology	Version	Notes
Runtime / SDK	.NET	SDK 10.0.401 (pinned, `rollForward: latestFeature`)	`global.json`
Web	ASP.NET Core MVC + Razor Pages	10.0	Server-rendered; no SPA
ORM	Entity Framework Core (SQL Server provider)	10.0.14	Migrations-first; drift check in CI
Identity	ASP.NET Core Identity (default UI, custom Register page)	10.0.14	Roles: `Donor`, `Employee`
Database	SQL Server 2022 / Azure SQL / LocalDB	—	`decimal(18,2)` money, `rowversion` concurrency
Data Protection	Keys stored in the database	10.0.14	Shared across instances
Payments	PayFast (redirect + ITN)	—	Sandbox by default
Email	SendGrid	9.29.3	Optional
PDF	QuestPDF	2026.9.0	Community licence (confirm eligibility)
Logging	Serilog.AspNetCore	10.0.0	Console sink + request logging
UI	Bootstrap 5, Bootstrap Icons 1.11.3 (SRI), Inter	—	`wwwroot/lib` vendored
Functions	Azure Functions isolated worker	2.52.0	HTTP trigger, function key
Tests	xUnit 2.9, Microsoft.AspNetCore.Mvc.Testing, EF InMemory, coverlet	—	98 tests
Packaging	NuGet lock files, source mapping, audit sources	—	`NuGet.config`

3. Environments and configuration

Environment	Purpose	Database	Payments	Demo data
Development	Local work	LocalDB or Docker SQL	PayFast sandbox	Yes
Testing (automated)	Integration tests	EF InMemory (isolated per test class)	Fake gateway	No
CI	Build, test, migration verification	SQL Server 2022 container	—	No
Production (Azure)	Public site	Azure SQL	PayFast live (own merchant, passphrase required)	No

All configuration keys are listed in the README. Requirements:

- **TR-CFG-01** No secrets in the repository. Use user-secrets locally and App Service or pipeline secrets in Azure.
- **TR-CFG-02** Unsafe live payment settings must stop startup: live mode without a passphrase, the sandbox merchant, or localhost/http URLs (`PayFastOptionsValidator`).
- **TR-CFG-03** Every supported currency needs a positive rate, or startup fails (`CurrencyConverter`).
- **TR-CFG-04** `Database:MigrateOnStartup=false` in production. Apply migrations from the idempotent script.

4. Integration contracts

4.1 PayFast checkout (outbound redirect)

Field	Source	Rule
`merchant_id` , `merchant_key`	Configuration	Required when enabled
`return_url`	`PayFast:ReturnUrl` or `/Donate/Confirmation` + `?reference=`	Absolute URL
`cancel_url`	`PayFast:CancelUrl` or `/Home/Donate` + `?cancelled=`	Absolute URL
`notify_url`	`PayFast:NotifyUrl` or `/Payments/Notify`	Must be publicly reachable in production
`email_address`	Donor email	Optional
`m_payment_id`	`Donation.TransactionReference` (32 hex)	Unique
`amount`	`Donation.AmountZar`	Invariant culture, two decimals
`item_name`	"Donation - {project or General Fund}"	≤ 100 characters
`signature`	MD5 of the URL-encoded fields in order (+ `passphrase`)	Lower-case hex

4.2 PayFast ITN (inbound webhook) — `POST /Payments/Notify`

Validation, in order. Any failure leaves the donation unchanged:

1. Signature recomputed over the posted fields in posted order, then a fixed-time comparison → `400` if wrong.
2. Server-to-server `POST /eng/query/validate` must return `VALID` (15-second timeout) → `400` otherwise.
3. Unknown `m_payment_id` → `404` .
4. `merchant_id` must equal the configured merchant → `400` .
5. Already `Verified` → `200` , no change (idempotent).
6. `payment_status = COMPLETE` and `amount_gross` equal to `AmountZar` → `Verified` , total incremented once, audit entry `PaymentVerified` . An amount mismatch → `400` .
7. Any other status while `Pending` → `Failed` , audit entry `PaymentFailed` .

The webhook is exempt from anti-forgery and rate limiting. Concurrency conflicts on the project row are retried up to three times.

4.3 Azure Function — `GET /api/tax-certificate/{reference}`

Requires a function key (`AuthorizationLevel.Function`). The reference must match `^[A-Za-z0-9]{16,100}$` . Only `Verified` donations are returned. Output is an `application/pdf` attachment. Intended for server-to-server use; donors use the web app, which also checks ownership.

4.4 SendGrid

Optional. Delivery failures are logged and never fail the user's request.

5. Non-functional requirements

ID	Requirement	Target	Current state	Verification
NFR-SEC-01	Server-side authorization on every staff and donor route	100% of routes	Met	Role-matrix integration tests (30 cases)
NFR-SEC-02	No self-service privilege escalation	Register cannot grant staff	Met	`Registration_AlwaysCreatesDonor_…`
NFR-SEC-03	CSRF protection on all state-changing forms	All POSTs except the webhook	Met	Global `AutoValidateAntiforgeryToken`
NFR-SEC-04	Abuse limits on public POSTs	≤ 20 POST/min/IP (configurable)	Met	`PublicPosts_AreLimitedPerClient_…`
NFR-SEC-05	Security headers	CSP, frame-ancestors none, nosniff, referrer and permissions policies	Met (CSP allows inline scripts)	`Responses_CarrySecurityHeaders`
NFR-SEC-06	No known-vulnerable packages	0 advisories	Met	CI gate `dotnet list package --vulnerable`
NFR-SEC-07	Staff MFA and login lockout	Required for Employee	**Not met**	Roadmap T-1.3 (F-36)
NFR-PRIV-01	Consent recorded for personal data	Timestamp per record	Met	`PrivacyConsentAt` columns, tests
NFR-PRIV-02	Data minimisation	Optional fields stay empty	Met	`Volunteer_WithoutDateOfBirth_…`
NFR-PRIV-03	Retention enforced	Per privacy notice	**Not met**	Roadmap T-2.4
NFR-INT-01	Money integrity	Totals = offline + verified	Met	Reconciliation page; tests
NFR-INT-02	Idempotent payments and submissions	No double counting	Met	`Notify_Complete_…` , `Donate_SameIdempotencyToken…`
NFR-INT-03	Schema matches model	No pending model changes	Met	CI drift check
NFR-REL-01	Health probes	`/health/live` (process), `/health` (database)	Met	Smoke test

ID	Requirement	Target	Current state	Verification
NFR-REL-02	Transient SQL faults retried	EF retry strategy	Met	`EnableRetryOnFailure`
NFR-REL-03	Sessions survive restarts and scale-out	Shared Data Protection keys	Met	`DataProtectionKeys` table
NFR-PERF-01	Staff lists bounded	≤ 50 rows per page	Met	Paged queries
NFR-PERF-02	Page response	p95 < 800 ms at 20 rps (proposed)	**Not measured**	Load test in roadmap
NFR-A11Y-01	Accessible forms	WCAG 2.2 AA	Partially verified (code review)	axe/Playwright in roadmap
NFR-OBS-01	Request logging and correlation	Every request logged	Met (console)	Serilog request logging
NFR-OBS-02	Alerting on failed payments and health	Alerts	**Not met**	Roadmap T-1.5
NFR-MNT-01	Automated tests on critical paths	≥ 60% line coverage of hand-written code	Met (67.9%)	coverlet in CI
NFR-BCK-01	Restorable backups	RPO ≤ 24 h, RTO ≤ 4 h (proposed)	**Not verified**	Restore drill in roadmap (F-17)

6. Data requirements

- Money: `decimal(18,2)`; exchange rates `decimal(18,6)`; all totals in rand.
- Time: UTC everywhere (`datetime2`). Views label UTC where it matters.
- Identifiers: database ids are internal. Public links use the 32-character transaction reference.
- Integrity is enforced in the database: check constraints for currencies, types, payment methods, statuses, amounts, dates and non-negative funds; unique idempotency key and transaction reference. See 05 — Backend Schema.

7. Engineering standards

Standard	Rule
Code	Nullable reference types on; input models for every form (no entity binding); business logic in services
Auditing	Every staff data change calls `IAuditService.Record` before `SaveChanges`
Money	Format with `Infrastructure/Money`; never `ToString("C")`
Migrations	One migration per model change, in the same PR; never edit an applied migration; backfill data in the migration
Tests	Integration test for anything touching money, roles or personal data
Dependencies	Lock files enforced (`--locked-mode` in CI); new private packages need a source-mapping entry

8. Known technical constraints and debt

Item	Impact	Plan
The function references the whole web project	Larger deployment, coupling	Extract a shared data project (T-2.3)
CSP allows `'unsafe-inline'`	Weaker XSS defence-in-depth	Move inline scripts to files, then use nonces (T-2.6)
Exchange rates are manual	Foreign amounts may drift from market	Rates provider or rate table (T-2.2)
No infrastructure-as-code	Environment drift	Bicep templates (T-3.1)

Gift of the Givers · Relief Management System

DELIVERABLE 03

Application Flow

Site map, user journeys, state machines and route inventory, traced from the screen to the database.

DOCUMENT	VERSION	DATE	STATUS
D-03	2.0	2026-11-02	Current — reflects the system after audit remediation

CONTENTS

1. Site map
2. Journey J1 — Donate (core journey)
3. Journey J2 — Register and staff provisioning
4. Journey J3 — Volunteer
5. Journey J4 — Donation certificate
6. Journey J5 — Funding reconciliation
7. Journey J6 — Contact
8. State machines
9. Route inventory

1. Site map

Public (anonymous)

Home	/ → intro once per session
About · live impact figures	/Home/About
Relief projects	/Home/ReliefProjects
Donate → PayFast	/Home/Donate
Receipt (by reference)	/Donate/Confirmation
Volunteer application	/Home/Volunteer
Contact	/Home/Contact
Privacy notice (POPIA)	/Home/Privacy
Register (always Donor) · Login	/Identity/Account/*
PayFast ITN webhook (server)	POST /Payments/Notify
Health probes	/health · /health/live

Donor role

Dashboard (confirmed totals)	/Donor/Dashboard
My donations + status	/Donor/MyDonations
Donation details	/Donor/DonationDetails/{id}
Certificate*	/Donor/TaxCertificate/{id}
Certificate PDF*	/Donor/DownloadTaxCertificate
Account, personal data	/Identity/.../Manage

* also available to Employees for any verified donation
Legacy /Donate/Tax-Certificate/{id} redirects here

Employee role (granted by staff)

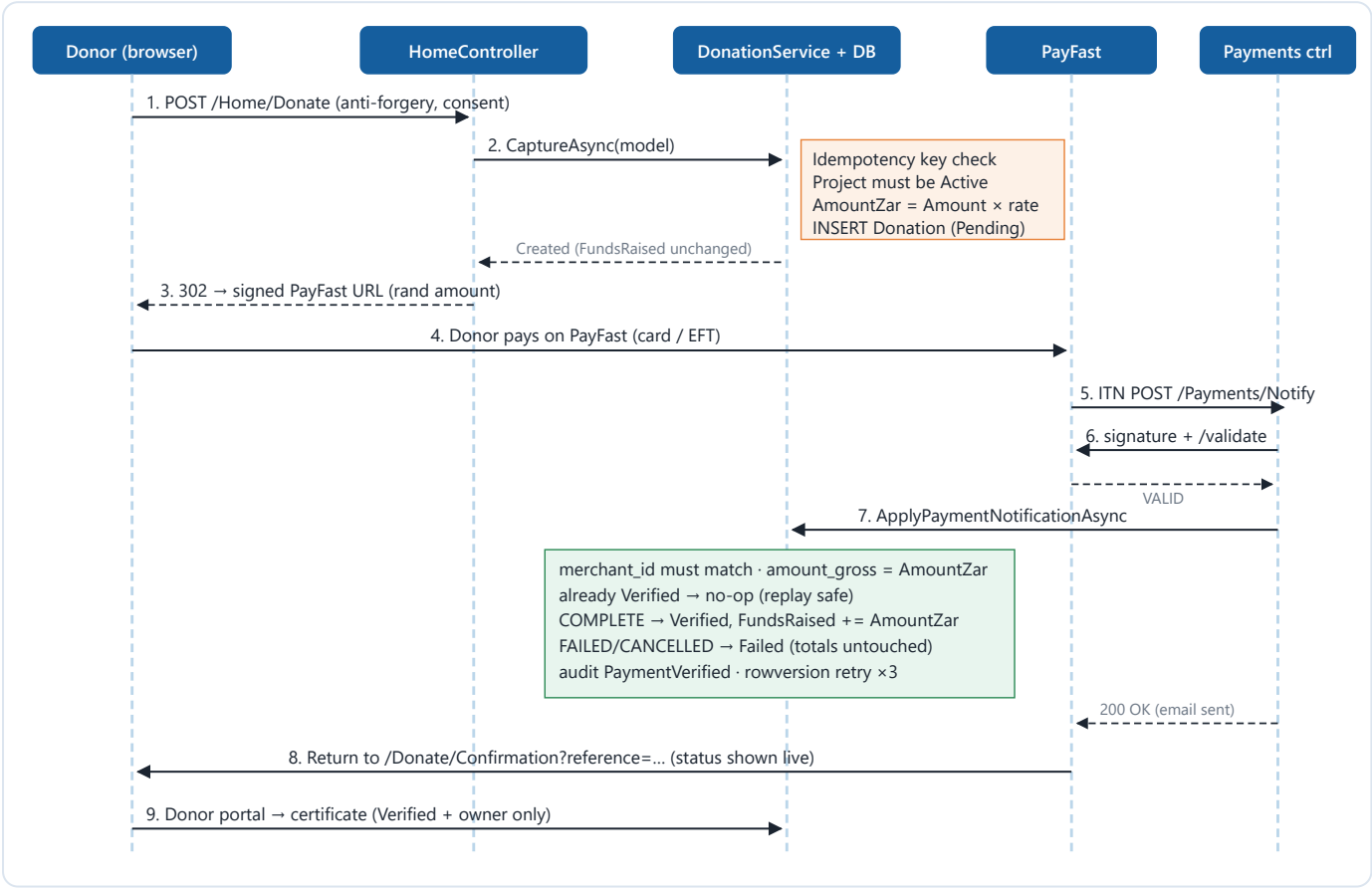
Dashboard + attention cards	/Employee/Dashboard
Projects · create/edit · updates	/Employee/ReliefProjects
Volunteers (filter, paged)	/Employee/Volunteers
Approve / reject / assign	/Employee/VolunteerDetails
Donations (status filter) · CSV	/Employee/Donations
Funding reconciliation	/Employee/FundingReconciliation
Inquiries	/Employee/Inquiries
Staff access (grant/revoke)	/Employee/StaffAccess
Audit log	/Employee/AuditLog

Site map by role — every page and the role that can reach it

Navigation rules:

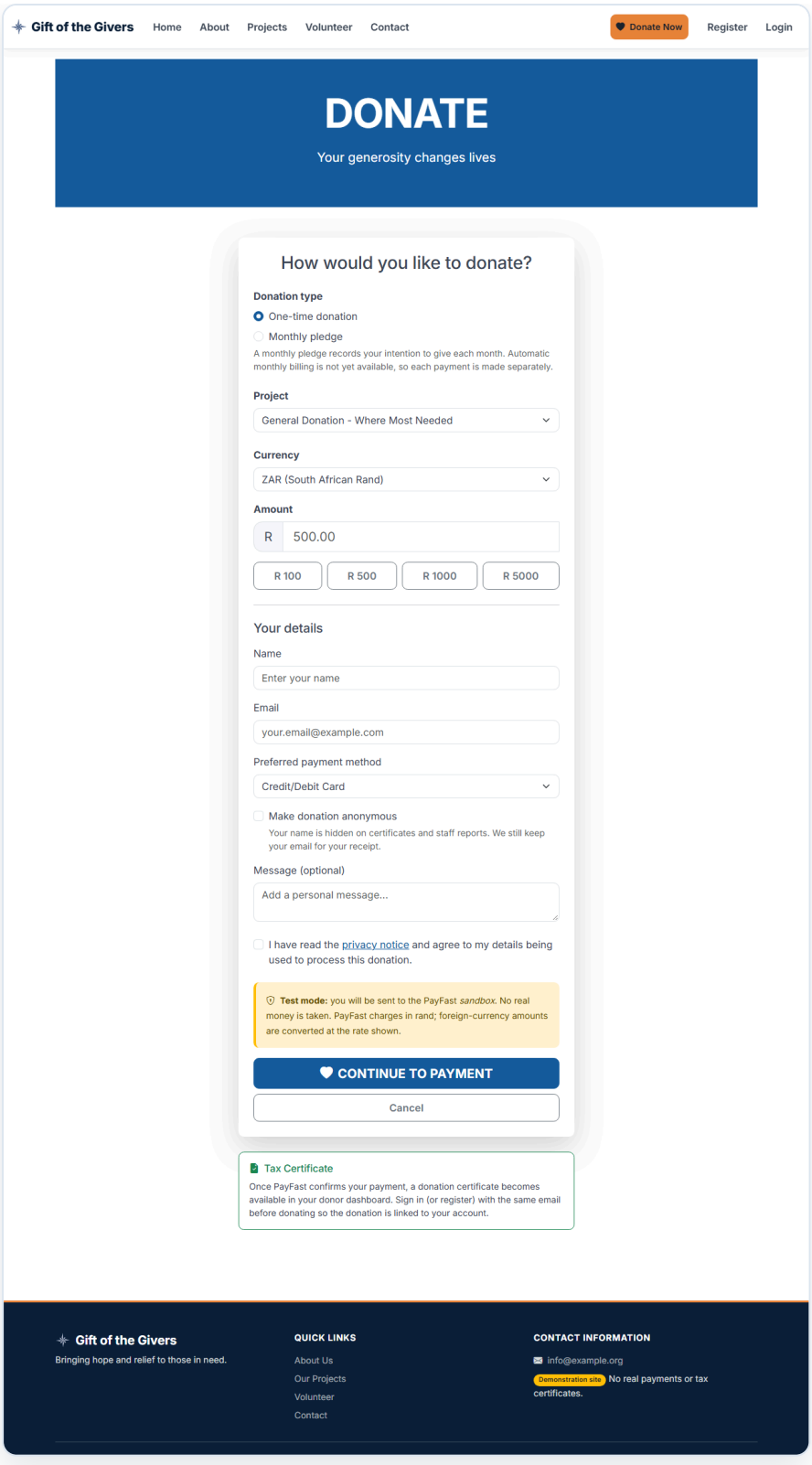
- The top navigation always shows Home, About, Projects, Volunteer, Contact and the **Donate Now** button.
- Signed-in donors also see *Donor Portal*. Employees see *Employee Portal*: Dashboard, Projects, Volunteers, Donations, Funding Reconciliation, Inquiries, Staff Access, Audit Log.
- On the first visit to `/` in each browser session, the visitor is sent to a short intro animation (skippable, and reduced-motion aware), which then returns them to the home page.

2. Journey J1 — Donate (core journey)

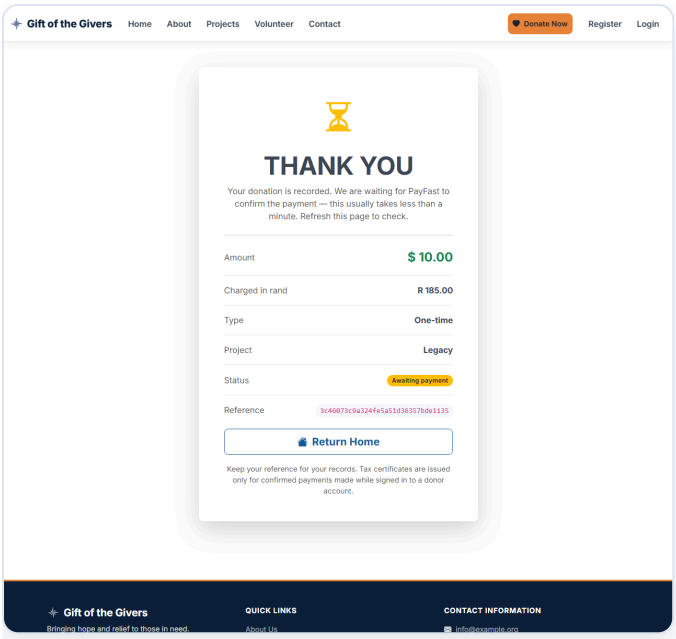


Donation and payment verification sequence

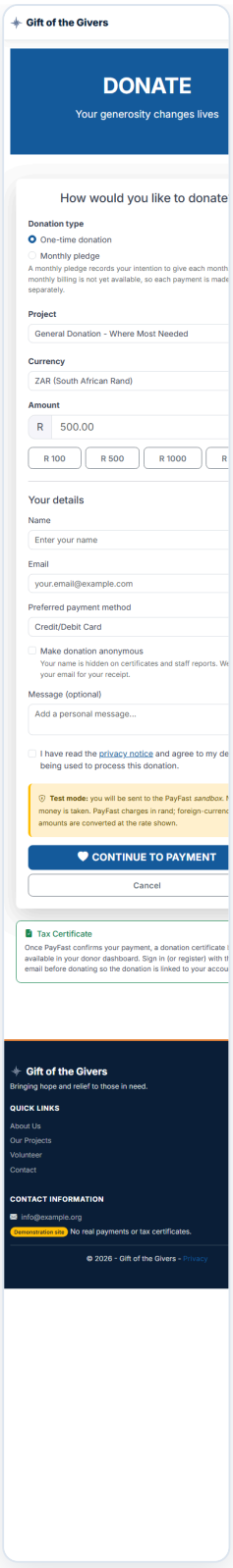
Step	Screen / route	System behaviour	Data
1	`GET /Home/Donate?projectId=`	Form shown. Signed-in donors get their name and email pre-filled. Indicative rand equivalent shown for USD/EUR	—
2	`POST /Home/Donate`	Anti-forgery, validation (type, currency, amount 1–1,000,000, payment method, consent). Errors re-display the form with a summary	—
3	`DonationService.CaptureAsync`	Duplicate token → same donation. Project must be Active. Rand conversion. Insert as **Pending**	`Donations` +1 row; **project total unchanged**
4	302 to PayFast	Signed URL for the rand amount; return, cancel and notify URLs carry the reference	—
5	PayFast	Donor pays by card or EFT, or cancels	—
6	`POST /Payments/Notify` (server to server)	Signature → validate → merchant → amount → status	`Donations.PaymentStatus`, `ReliefProjects.FundsRaised`, `AuditLogs`
7	`GET /Donate/Confirmation?reference=`	Receipt with live status (Awaiting payment / Paid / Failed)	—
8	Donor portal	Certificate available once Paid (owner only)	—



The donation form (desktop)



Receipt while the payment is pending (USD 10 charged as R 185.00)



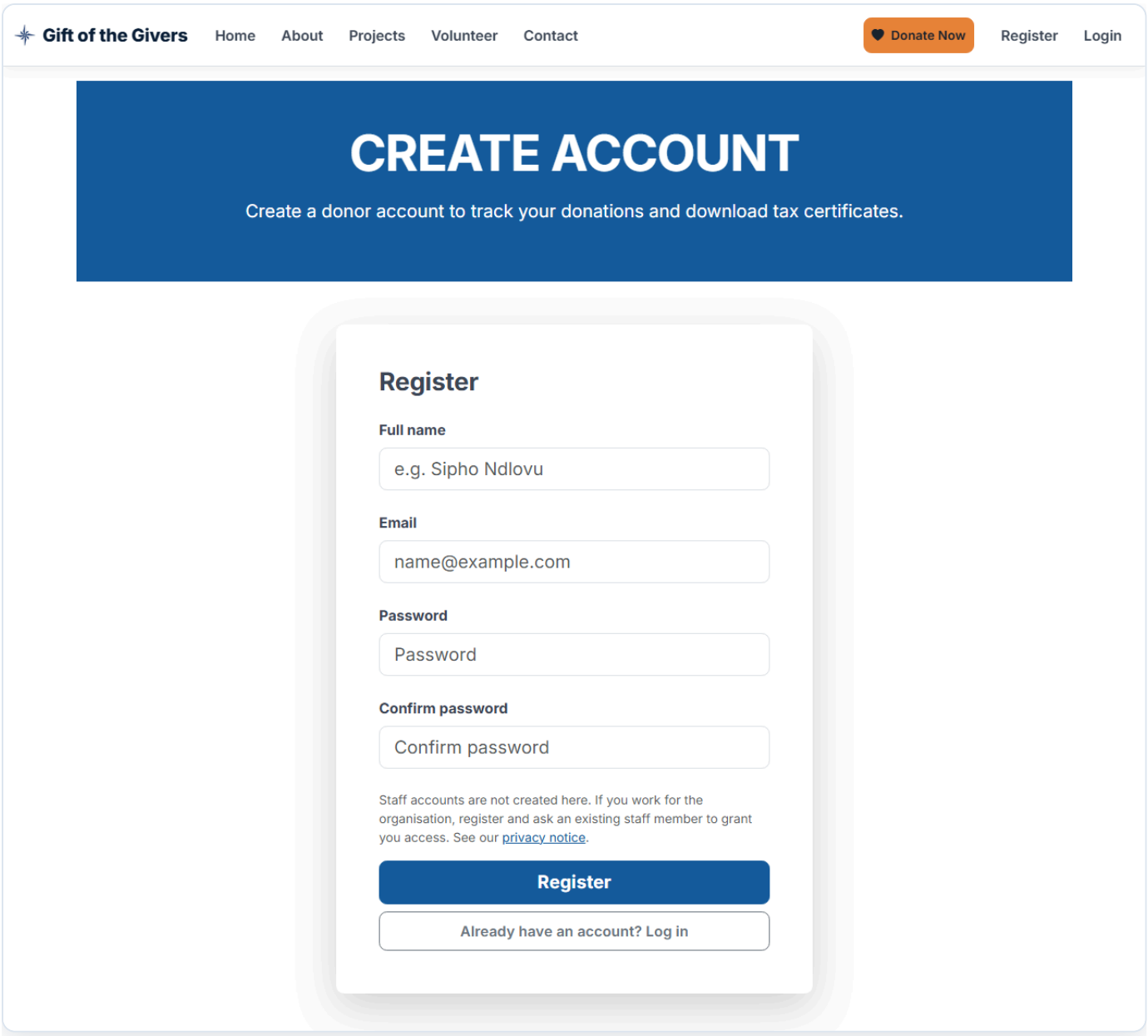
Donate on a 390 px wide phone

Failure and edge paths

Situation	Behaviour
Invalid input (e.g. unknown currency, consent not ticked)	Form re-displayed with errors (HTTP 200); nothing stored
Project suspended or completed	"Not accepting donations" message; nothing stored
Double click / resubmit	Same donation returned (idempotency key); the second redirect goes to the same checkout
Donor cancels on PayFast	Back to Donate with a "payment cancelled, not charged" banner; donation stays Pending
PayFast reports FAILED	Donation marked Failed; totals untouched; receipt offers *Try again*
ITN replayed	200, no change
ITN amount or merchant mismatch	400, donation stays Pending, error logged
Payments disabled (`PayFast:Enabled=false`)	Donation stored as Pending; receipt explains that nothing was charged
More than 20 POSTs per minute from one IP	429 Too Many Requests

3. Journey J2 — Register and staff provisioning

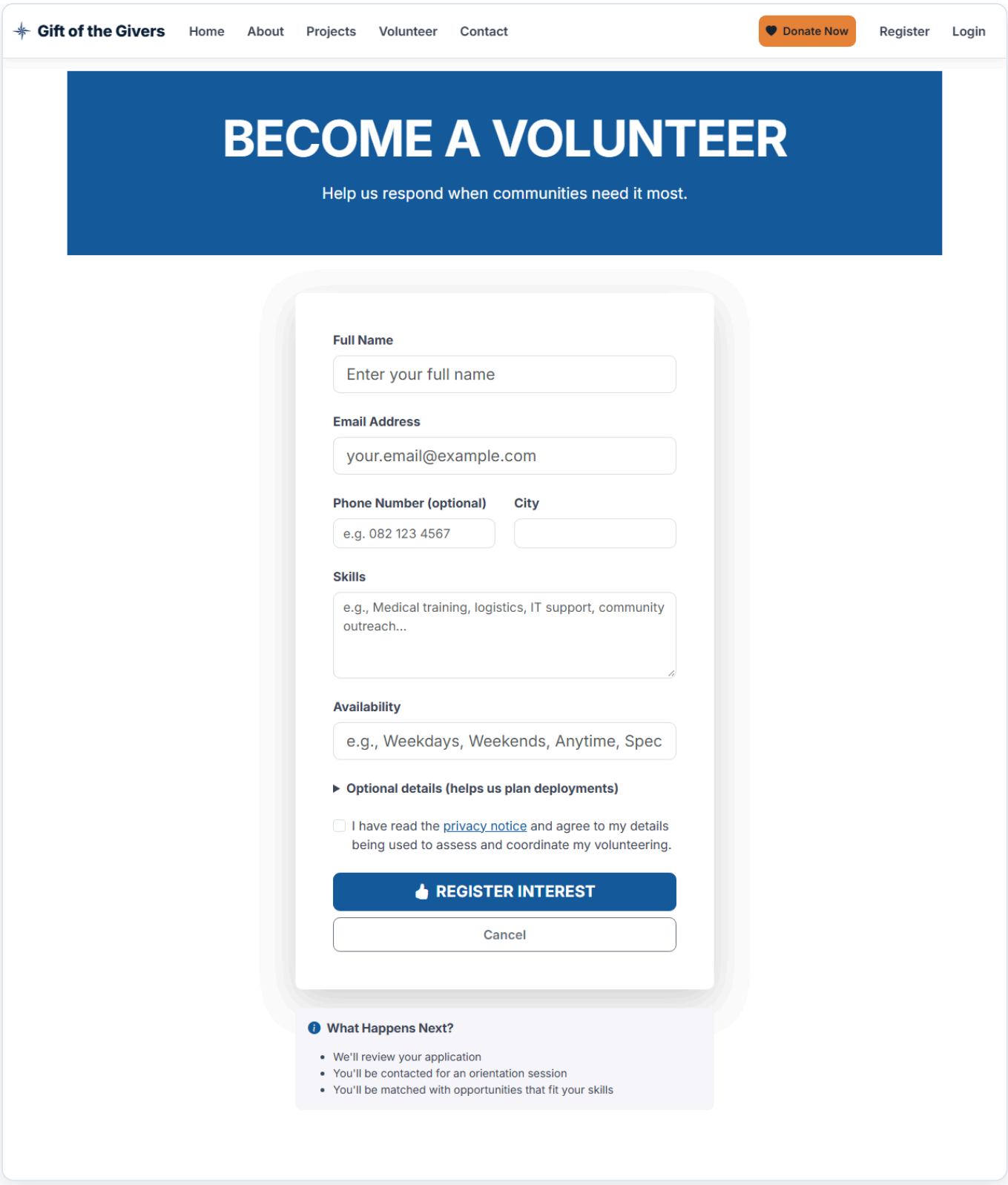
Step	Actor	Route	Result
1	Visitor	`GET/POST /Identity/Account/Register`	Account created with role **Donor** (no role choice). Signed in
2a	First staff member	Deployment sets `StaffAccess:BootstrapEmployeeEmails`	On the next start, the account gets **Employee** (logged)
2b	Existing employee	`GET /Employee/StaffAccess` → `POST GrantEmployee(email)`	Role granted; audit entry `EmployeeGranted`
3	Employee	`POST RevokeEmployee(userId)`	Role removed and security stamp rotated; cannot revoke self; audited



Registration page — no staff option

4. Journey J3 — Volunteer

Step	Actor	Route	Result
1	Visitor	`GET/POST /Home/Volunteer`	Validated (consent required; DOB optional, ≥ 16 years); stored as **Pending** with consent time
2	Employee	`/Employee/Volunteers?status=Pending`	Paged list filtered by status
3	Employee	`/Employee/VolunteerDetails/{id}` → Approve / Reject	Status set, audit entry, email if configured
4	Employee	Assign to an active project / Remove	`VolunteerAssignments` row (unique per pair); audited



Volunteer application form

5. Journey J4 — Donation certificate

Step	Actor	Route	Rule
1	Donor	`/Donor/MyDonations`	Certificate icon only on Paid rows
2	Donor / Employee	`/Donor/TaxCertificate/{id}`	404 unless Verified **and** (owner or Employee); view audited
3	Donor / Employee	`/Donor/DownloadTaxCertificate/{id}`	PDF; download audited; demonstration watermark until PBO configured
—	Legacy links	`/Donate/Tax-Certificate/{id}[/download]`	302 to the routes above
—	Integration	Function `GET /api/tax-certificate/{reference}` + function key	Verified only

6. Journey J5 — Funding reconciliation

1. The employee dashboard shows "*N projects whose funding total needs review*".
2. `/Employee/FundingReconciliation` lists, per project: offline funds, paid online, expected, recorded, difference, and awaiting payment.
3. **Recalculate** sets recorded = offline + paid (audit `FundingRecalculated`).
4. **Accept as offline** keeps the recorded total and moves the unexplained difference into offline funds (audit `FundingAcceptedAsOffline`). Only allowed when the difference is positive.

7. Journey J6 — Contact

`POST /Home/Contact` stores the inquiry with consent → confirmation page → an acknowledgement email if SendGrid is configured → staff see it at `/Employee/Inquiries` (open first) → *Mark handled* (audited).

8. State machines

Donation payment status

From	Event	To	Side effects
—	Form submitted	Pending	None on totals
Pending	ITN COMPLETE, amount and merchant match	Verified	`FundsRaised += AmountZar`; certificate available; email; audit
Pending	ITN with any other status	Failed	Audit only
Verified	Any ITN	Verified	None (idempotent)
Failed	Any ITN	Failed	None

Volunteer status

From	Event	To
—	Application submitted	Pending
Pending / Rejected	Approve	Approved (ApprovalDate set)
Pending / Approved	Reject	Rejected (ApprovalDate cleared)

Project status

`Active` ⇄ `Suspended` → `Completed` (set by staff). Only **Active** projects accept donations and new volunteer assignments.

9. Route inventory

Method	Route	Auth	Purpose
GET	`/`, `/Home/About`, `/Home/ReliefProjects`, `/Home/Privacy`	Public	Information pages
GET/POST	`/Home/Donate`	Public	Donation form / capture → PayFast
GET	`/Donate/Confirmation?reference=`	Public (unguessable reference)	Receipt
GET/POST	`/Home/Volunteer`, `/Home/Contact`	Public	Applications and inquiries
GET	`/Volunteer/Confirmation`, `/Contact/Confirmation`	Public	Confirmations
POST	`/Payments/Notify`	PayFast signature + validation	ITN webhook
GET	`/health`, `/health/live`	Public	Readiness / liveness
*	`/Identity/Account/*`	Varies	Register (Donor only), Login, Manage
GET	`/Donor/Dashboard`, `/Donor/MyDonations`, `/Donor/DonationDetails/{id}`	Donor	Donor portal
GET	`/Donor/TaxCertificate/{id}`, `/Donor/DownloadTaxCertificate/{id}`	Donor (owner) or Employee	Certificates
GET	`/Donate/Tax-Certificate/{id}[/download]`	Donor or Employee	Legacy redirects
GET	`/Employee/Dashboard`, `ReliefProjects`, `Volunteers`, `VolunteerDetails/{id}`, `Donations`, `ExportDonations`, `FundingReconciliation`, `Inquiries`, `AuditLog`, `StaffAccess`	Employee	Staff views
GET/POST	`/Employee/CreateReliefProject`, `EditReliefProject/{id}`, `CreateProjectUpdate`, `EditProjectUpdate/{id}`	Employee	Project management

Method	Route	Auth	Purpose
POST	`/Employee/ApproveVolunteer`, `RejectVolunteer`, `AssignVolunteer`, `UnassignVolunteer`, `MarkInquiryHandled`, `RecalculateFunding`, `AcceptFundingAsOffline`, `GrantEmployee`, `RevokeEmployee`	Employee	Staff actions (audited)
GET	Function `/api/tax-certificate/{reference}`	Function key	Certificate PDF

Gift of the Givers · Relief Management System

DELIVERABLE 04

Design Brief

Visual identity, components, content rules and accessibility requirements, documented from the implemented design system.

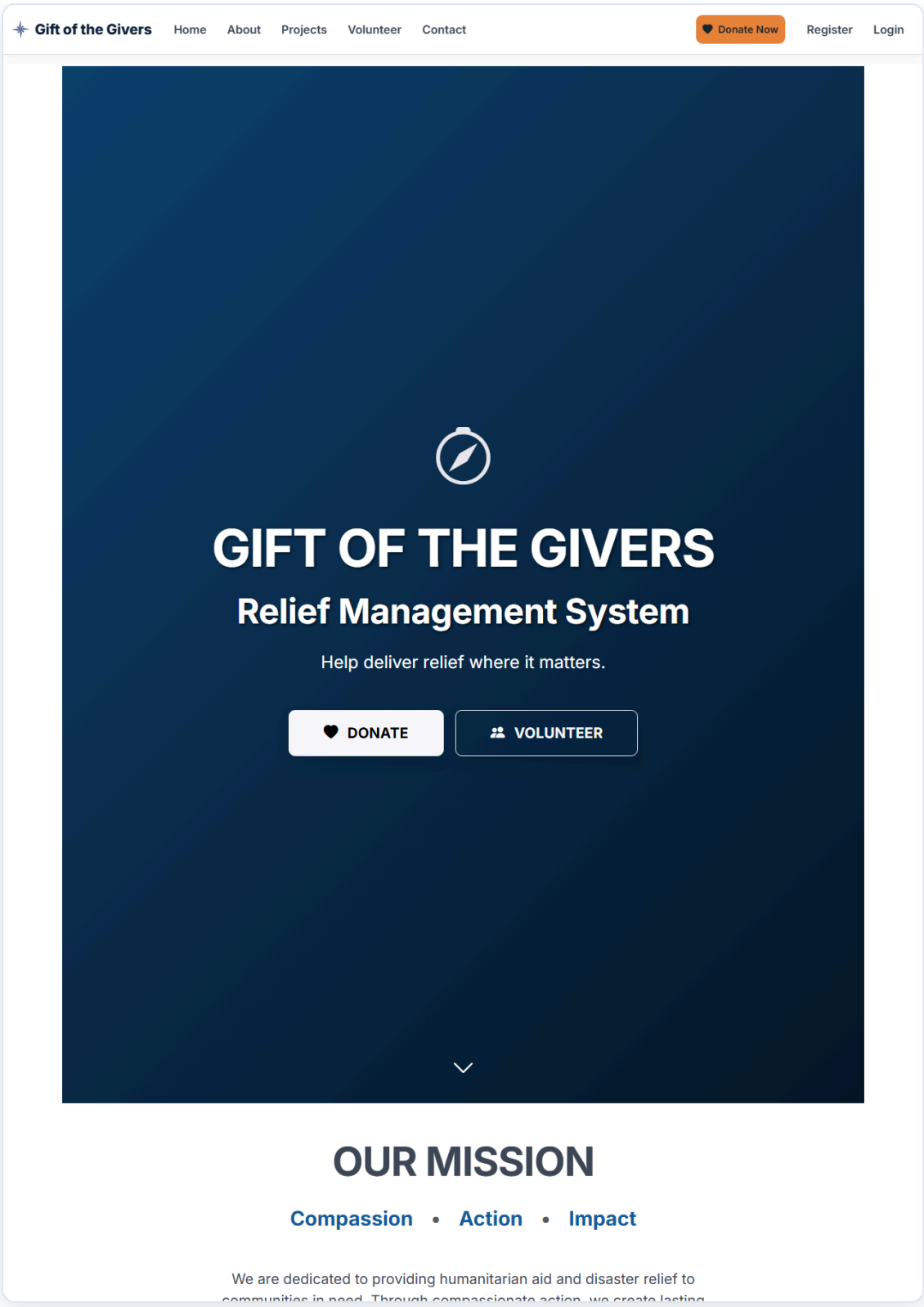
DOCUMENT	VERSION	DATE	STATUS
D-04	2.0	2026-11-02	Current — reflects the implemented UI

CONTENTS

1. Design goals
2. Brand tokens
3. Typography
4. Layout
5. Components
6. Content rules
7. States
8. Accessibility requirements (WCAG 2.2 AA target)
9. Screens
10. Design debt

1. Design goals

1. **Trust first.** Donors give money to a cause they cannot see. Every screen must look credible and say exactly what will happen. In particular, payment state is always explicit: *Awaiting payment*, *Paid*, *Failed*.
2. **Urgency without manipulation.** Strong calls to action ("Donate Now") but no fake counters, invented statistics or dark patterns.
3. **Mobile first for the public.** Most visitors arrive from social media on phones. The donate form must work one-handed on a 390 px screen.
4. **Calm, efficient staff tools.** Dense tables, clear filters, confirmations for destructive actions, nothing decorative.



Home page

2. Brand tokens

All tokens live in `GiftOfTheGivers/wwwroot/css/site.css` (`:root`) and override Bootstrap 5 variables. Do not hard-code colours in views; use the tokens or Bootstrap utilities mapped to them.

Primary
#155E9E · links, primary buttons, headers

Primary dark
#0F4676 · hover, emphasis

Navy
#0A2138 · footer, hero gradient

Accent
#E8833A · Donate Now, highlights

Primary subtle
#E7F0F8 · tints, table heads

Ink
#1B2430 · headings

Body
#3C4655 · body text

Muted
#6B7684 · secondary text

Success
Bootstrap green · Paid, approved

Warning
Bootstrap amber · Awaiting payment, test mode

Token	Value
Font	**Inter** (Google Fonts) with a system fallback stack; weights 400–800
Radius	0.375 / 0.55 / 0.85 / 1.1 rem (sm / default / lg / xl)
Focus ring	Primary at 25% opacity (Bootstrap focus ring)
Breakpoints	Bootstrap defaults; custom rules at ≥ 768 px

Contrast (measured, WCAG 2.x formula): primary #155E9E on white 6.72:1, body #3C4655 on white 9.55:1, muted #6B7684 on white 4.62:1 — all pass AA for normal text. **White text on the accent #E8833A is 2.71:1 and fails AA even for large text (3:1).** Ink text #1B2430 on the accent is 5.77:1 and passes, so the *Donate Now* / accent buttons now use ink text, with a lighter hover (#F09A5C, 7.07:1). This fixed design debt item D-03 during remediation.

3. Typography

Use	Style
Page hero titles	`display-3/4`, uppercase, bold, white on primary band
Section titles	`h2` – `h3`, ink colour
Body	1rem Inter, body colour, line-height ~1.6
Data and references	Monospace (`font-monospace`, `<code>`) for transaction references
Money	Always through `Money.Format` / `Money.Zar`: "R 1,234.50", "$ 75.00", "€ 10.00". Never culture-dependent `ToString("C")`

4. Layout

- **Public pages:** full-width coloured hero band, then content centred in `container` with max ~8 columns for forms (`col-lg-6` for the donate form), and a dark navy footer with three columns.
- **Staff pages:** `container my-5` with page title, lead sentence, filter pills or tabs, a summary card, then a responsive table (`table-responsive`) and the pager.
- **Grid:** Bootstrap 12 columns. Cards use `shadow-sm` (staff) and `shadow-lg` (primary public forms).

5. Components

Component	Use	Rules
Primary button	Main action per screen	One per form. Labels state the result: "CONTINUE TO PAYMENT", not "Submit"
Donate Now (accent)	Global navigation	Always visible; links to `/Home/Donate`; ink text on orange for contrast
Payment status badge	Anywhere a donation is shown	`_PaymentStatusBadge`: Paid (green), Awaiting payment (amber), Failed (red). Never infer status from other fields
Flash messages	After POST-redirect-GET	`_StatusMessages` in the layout, `role="status"`, dismissible
Validation summary	Every form	`asp-validation-summary="All"` with `role="alert"`; hidden when empty (`.validation-summary-valid`)
Consent checkbox	Every form collecting personal data	Links to the privacy notice; required on the server
Pager	Staff lists	`_Pager`: Previous/Next, "Page x of y · n records"
Confirmation	Destructive staff actions	`data-confirm="…"` on the button (handled in `site.js`). Never inline JavaScript with user data
Impact stats	Home and About	`ImpactStats` view component (band or cards); always labelled "Live figures from this system"
Demo badge	Footer, certificates	Shown while `Organisation:IsDemo` is true

6. Content rules

1. **No invented numbers.** Statistics come from the database or are removed. Narrative examples are labelled *illustrative*.
2. **No borrowed identity.** Contact details, registration numbers and PBO references come from configuration. Placeholders show "Not configured" instead of fake values.
3. **Say what happens next.** Before payment: "You will be sent to PayFast… No real money is taken (test mode)." After payment: the actual status.
4. **Plain language** at about a Grade 8 reading level; sentence case for body text; uppercase only for hero titles.
5. **Monthly pledge, not "recurring donation"**, until automatic billing exists.

7. States

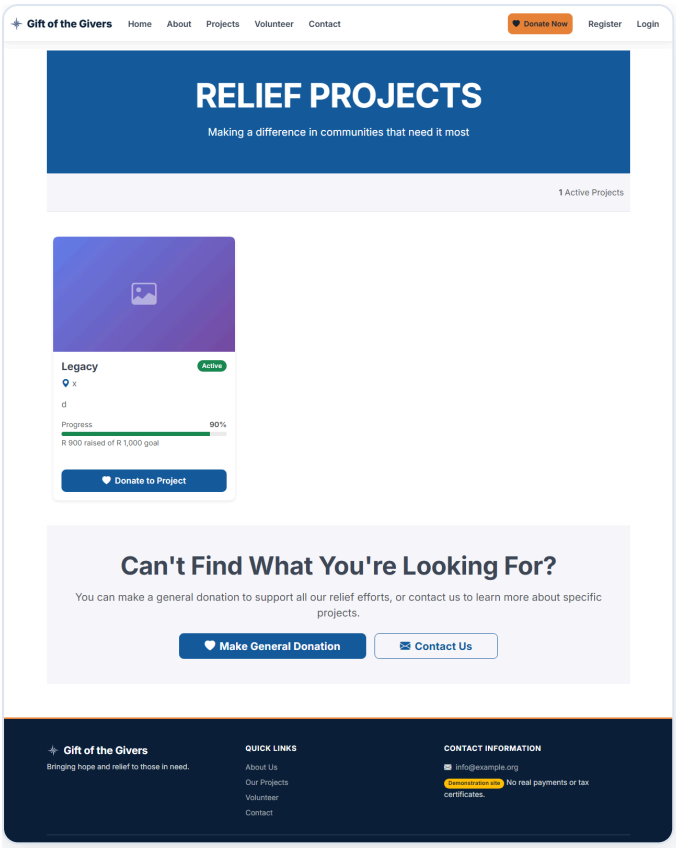
State	Pattern	Example
Empty	Neutral `alert-info` with what to do next	"No donations match this filter."
Loading	Server-rendered pages; no spinners needed. Long actions (PDF) open as downloads	—
Success	Green flash, or a receipt with a green check	Paid receipt
Pending	Amber hourglass and "Awaiting payment"	Receipt before the ITN arrives
Error (validation)	Summary at top + field messages	Donate form
Error (server)	`/Home/Error` with request id (production)	—
Forbidden	Identity Access Denied page	Donor opening a staff page

8. Accessibility requirements (WCAG 2.2 AA target)

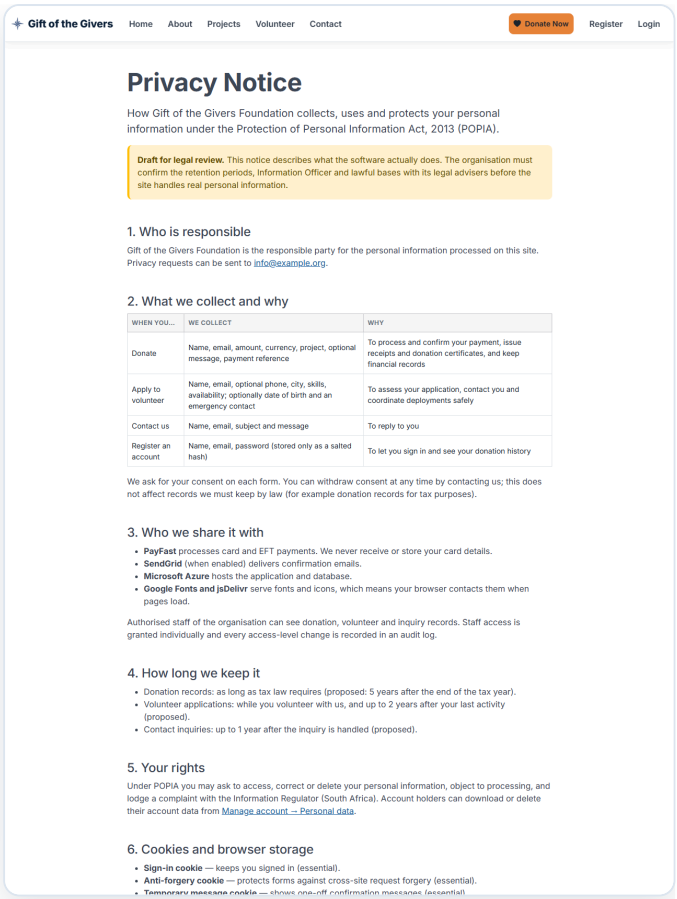
Requirement	Implementation
Labels for every input	`asp-for` labels; radio groups wrapped in `fieldset` + `legend`
Errors announced	Validation summary has `role="alert"` ; flash messages have `role="status"`
Keyboard	Native controls only; visible focus ring from Bootstrap
Icons	Decorative icons have `aria-hidden="true"` ; icon-only buttons have visually hidden text
Live region	Rand equivalent on the donate form uses `aria-live="polite"`
Motion	Intro animation honours `prefers-reduced-motion` and has a Skip button
Tables	`scope="col"` headers on staff tables
Language	`lang="en"`

Not yet verified with assistive technology or automated tools. An axe and Playwright suite is in the implementation plan (T-2.5).

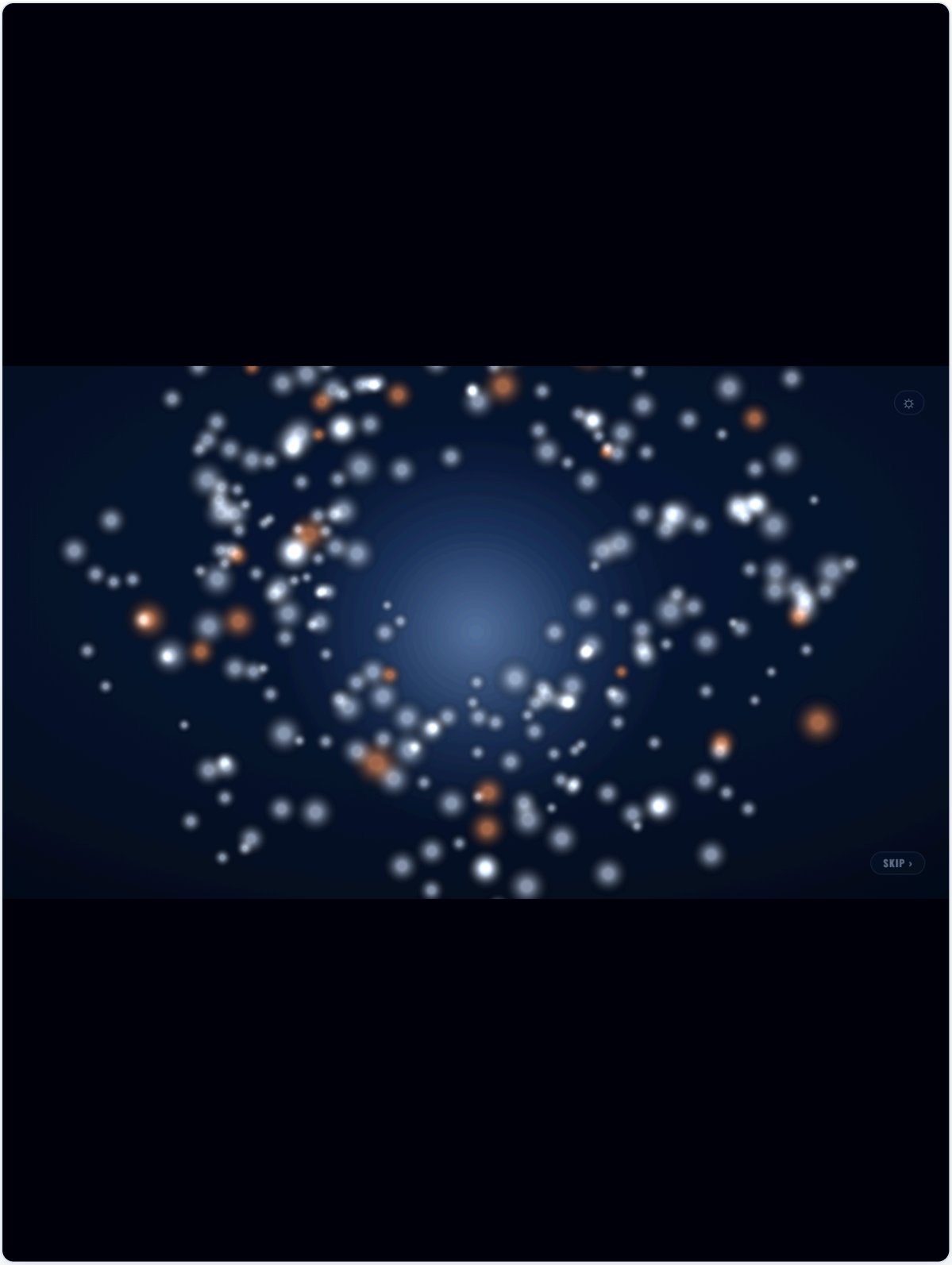
9. Screens



Relief projects with funding progress (rand)



Privacy notice with draft banner



Intro animation (first visit per session; skippable; reduced-motion aware)

10. Design debt

ID	Item	Proposal
D-01	Inline styles and scripts in several views	Move to `site.css` / `site.js` (also enables a strict CSP)
D-02	Hero bands use `bg-primary` with large uppercase headings on every page	Reduce on staff pages to save vertical space
D-03	~~White text on the accent button (2.71:1)~~	**Fixed:** ink text (5.77:1), lighter hover (7.07:1)
D-04	Staff tables on phones scroll horizontally	Card layout for narrow screens
D-05	Fonts and icons come from third-party CDNs	Self-host (privacy, reliability)

Gift of the Givers · Relief Management System

DELIVERABLE 05

Backend Schema

Database design, constraints, relationships, derived values and migration history for SQL Server / Azure SQL.

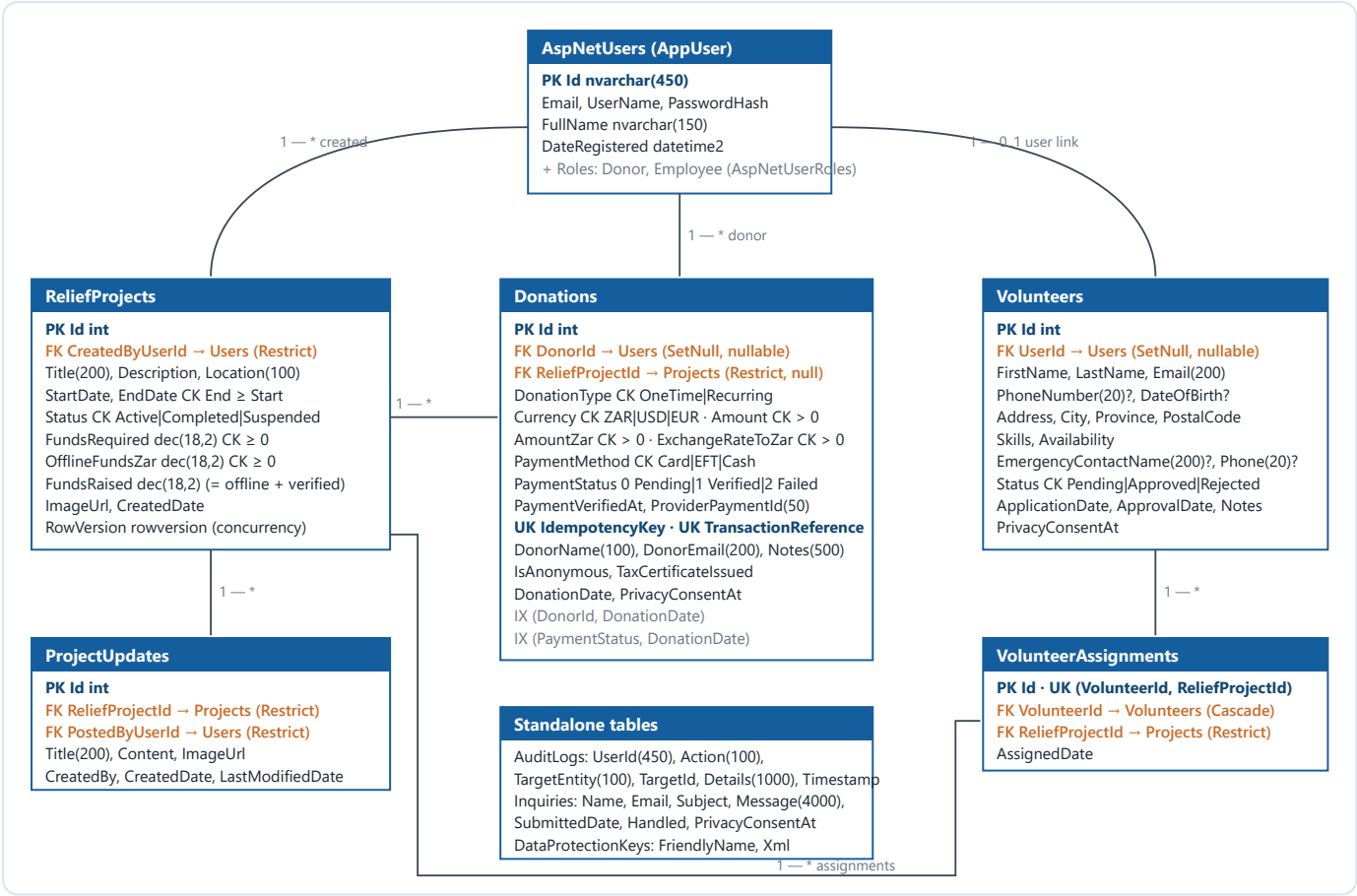
DOCUMENT	VERSION	DATE	STATUS
D-05	2.0	2026-11-02	Current — at migration AuditRemediationNov2026

CONTENTS

1. Overview
2. Tables
3. Relationships and delete behaviour
4. Derived values and invariants
5. Concurrency
6. Migration history
7. Data lifecycle and privacy
8. Seed data (Development only)

1. Overview

- **Engine:** SQL Server 2022, Azure SQL, or LocalDB, through EF Core 10. **Migrations are the source of truth**; CI fails if the model and migrations differ.
- **Context:** `GiftOfTheGivers/Data/ApplicationDbContext.cs`, which extends `IdentityDbContext<AppUser>` and implements `IDataProtectionKeyContext`.
- **Integrity strategy:** constraints in the database (check, unique, foreign key) plus validation in the application. Money is `decimal`; time is UTC `datetime2`; concurrency uses `rowversion`.



Entity relationship diagram

2. Tables

2.1 ReliefProjects

Column	Type	Null	Rules
Id	int identity	No	PK
CreatedByUserId	nvarchar(450)	Yes	FK → AspNetUsers (Restrict)
Title	nvarchar(200)	No	
Description	nvarchar(max)	No	Input limited to 2,000 characters
Location	nvarchar(100)	No	
StartDate / EndDate	datetime2	No / Yes	`CK_ReliefProject_Dates` : EndDate is null or ≥ StartDate
Status	nvarchar(max) (enum as string)	No	`CK_ReliefProject_Status` : Active, Completed, Suspended
FundsRequired	decimal(18,2)	No	`CK_ReliefProject_FundsRequired_NonNegative`
OfflineFundsZar	decimal(18,2)	No	`CK_ReliefProject_OfflineFunds_NonNegative` ; staff-maintained
FundsRaised	decimal(18,2)	No	**Derived**: OfflineFundsZar + Σ AmountZar of Verified donations
ImageUrl	nvarchar(max)	Yes	http(s) only, ≤ 500 characters (validated)
RowVersion	rowversion	—	Optimistic concurrency token
CreatedDate	datetime2	No	UTC

2.2 Donations

Column	Type	Null	Rules
Id	int identity	No	PK (internal only, never in public URLs)
DonorId	nvarchar(450)	Yes	FK → AspNetUsers (**SetNull** when the user is deleted)
ReliefProjectId	int	Yes	FK → ReliefProjects (Restrict); null = General Fund
DonationType	nvarchar(20)	No	`CK_Donation_Type` : OneTime, Recurring (pledge)
Currency	nvarchar(3)	No	`CK_Donation_Currency` : ZAR, USD, EUR
Amount	decimal(18,2)	No	`CK_Donation_Amount` > 0; in the chosen currency
AmountZar	decimal(18,2)	No	`CK_Donation_AmountZar` > 0; charged to PayFast; used for all totals
ExchangeRateToZar	decimal(18,6)	No	`CK_Donation_ExchangeRate` > 0; 1 for ZAR
PaymentMethod	nvarchar(50)	No	`CK_Donation_PaymentMethod` : Card, EFT, Cash
PaymentStatus	int	No	0 Pending, 1 Verified, 2 Failed
PaymentVerifiedAt	datetime2	Yes	Set by a verified ITN

Column	Type	Null	Rules
ProviderPaymentId	nvarchar(50)	Yes	PayFast `pf_payment_id`
IdempotencyKey	uniqueidentifier	No	**Unique**; one donation per form submission
TransactionReference	nvarchar(100)	Yes	**Unique**; 32 hex characters; PayFast `m_payment_id` ; public receipt key
DonorName / DonorEmail	nvarchar(100) / nvarchar(200)	Yes	
IsAnonymous	bit	No	Hides the name on certificates and reports
TaxCertificateIssued	bit	No	True once verified
Notes	nvarchar(500)	Yes	
DonationDate	datetime2	No	UTC
PrivacyConsentAt	datetime2	Yes	Null only for rows created before remediation

Indexes: `DonorId` , `ReliefProjectId` , `(DonorId, DonationDate)` , `(PaymentStatus, DonationDate)` , and unique `IdempotencyKey` and `TransactionReference` .

2.3 Volunteers

Column	Type	Null	Rules
Id	int identity	No	PK
UserId	nvarchar(450)	Yes	FK → AspNetUsers (SetNull)
FirstName / LastName	nvarchar(100)	No	
Email	nvarchar(200)	No	Indexed
PhoneNumber	nvarchar(20)	**Yes**	Optional, never invented
Address / City / Province / PostalCode	200 / 100 / 50 / 10	Yes	
DateOfBirth	datetime2	**Yes**	Optional; ≥ 16 years if given
Skills / Availability	nvarchar(max)	No	Input limited to 2,000 / 200 characters
EmergencyContactName / Phone	nvarchar(200) / nvarchar(20)	Yes	
Status	nvarchar(450) (enum as string)	No	`CK_Volunteer_Status` : Pending, Approved, Rejected; index `(Status, ApplicationDate)`
ApplicationDate / ApprovalDate	datetime2	No / Yes	UTC
Notes	nvarchar(max)	Yes	
PrivacyConsentAt	datetime2	Yes	

2.4 Other tables

Table	Key columns	Notes
VolunteerAssignments	VolunteerId (Cascade), ReliefProjectId (Restrict), AssignedDate	Unique `(VolunteerId, ReliefProjectId)`
ProjectUpdates	ReliefProjectId (Restrict), PostedByUserId (Restrict), Title(200), Content, ImageUrl, CreatedBy, CreatedDate, LastModifiedDate	
Inquiries	Name(200), Email(200), Subject(200), Message(4000), SubmittedDate, Handled, PrivacyConsentAt	
AuditLogs	UserId(450), Action(100), TargetEntity(100), TargetId, Details(1000), Timestamp	Index `(TargetEntity, TargetId)` and `Timestamp`. Payment events use actor `system:payfast`
DataProtectionKeys	FriendlyName, Xml	ASP.NET Core key ring shared by all instances
AspNet*	Identity schema (key length 450)	`Stores.MaxLengthForKeys = 0` keeps the runtime model matching this schema

3. Relationships and delete behaviour

Relationship	Cardinality	On delete	Reason
User → Donations	1 : * (optional)	SetNull	Deleting an account must not delete financial records
User → Volunteers	1 : 0..1 (optional)	SetNull	Same
User → Projects (creator), Updates (poster)	1 : *	Restrict	Keep authorship; deactivate staff instead of deleting
Project → Donations	1 : *	Restrict	Financial history must survive
Project → Updates, Assignments	1 : *	Restrict	Avoid multiple cascade paths in SQL Server
Volunteer → Assignments	1 : *	Cascade	Assignments have no meaning without the volunteer

4. Derived values and invariants

Invariant	Enforced by
`FundsRaised = OfflineFundsZar + Σ(AmountZar where PaymentStatus = Verified)`	`DonationService` (verification), offline-funds delta on edit, `RecalculateFundsRaisedAsync`, Funding Reconciliation page
A donation is counted once	Status check (Verified short-circuits) + rowversion retry
One donation per form submission	Unique `IdempotencyKey`, plus catch-and-return of the winning row
Certificate only for paid gifts	Query filter `PaymentStatus = Verified` in every certificate path
Staff changes are traceable	`AuditService.Record` staged in the same `SaveChanges`

5. Concurrency

`ReliefProjects.RowVersion` protects totals. Payment verification and recalculation retry up to three times on `DbUpdateConcurrencyException`. A project edit that conflicts shows "changed by someone else, reload". The EF SQL Server retry strategy (`EnableRetryOnFailure`) handles transient faults. No user-initiated transactions are used, so it is compatible.

6. Migration history

Migration	Summary	Notes
20260828181123_InitialCreate	Identity, projects, donations, volunteers, updates, assignments	
20260924181920_SyncCurrentModel	RowVersion, PaymentStatus, FK changes	
20260924191049_P1PaymentsEmailAndStatuses	DonorEmail, IdempotencyKey, AuditLogs, Inquiries, check constraints	**Edited during remediation (F-21):** the `AlterColumn` on `RowVersion` was removed because SQL Server cannot alter a rowversion column, so the migration failed on every empty database
20260924195240_P0Phase0CriticalFixes	Donor FK; raw-SQL check constraints	The raw constraints duplicated EF-managed ones (dropped in the remediation migration)
20260924200101_FixPendingModelChanges	Empty	Symptom of drift-chasing under the suppressed warning
20260925180656_SyncDonorCascade	Donor FK SetNull	
(removed) 20260924190000_P0SecurityAndDonationIntegrity	—	Orphan file without a `[Migration]` attribute; never discovered by EF; deleted
20261102112731_AuditRemediationNov2026	AmountZar, ExchangeRateToZar, DonorName, PaymentVerifiedAt, ProviderPaymentId, PrivacyConsentAt (×3), OfflineFundsZar, nullable DOB/phone, sized audit columns + Details, DataProtectionKeys, new check constraints and index	Backfills rand values (ZAR exact; USD 18.50 and EUR 20.00 indicative), normalises PaymentMethod (`PayPal` → `Card`), sets PaymentVerifiedAt for verified rows, drops the duplicate constraints. Down re-creates them

Verification (2026-11-02, LocalDB): old chain applied to an empty database, legacy rows inserted (ZAR, USD with PayPal, EUR), upgrade applied, values checked, rolled back and re-applied, drift check clean. CI repeats the empty-database apply plus rollback/re-apply on SQL Server 2022 for every push.

7. Data lifecycle and privacy

Data	Retention (proposed, pending legal review)	Deletion path
Donations	5 years after the end of the tax year	Not deleted; anonymise donor fields on request after retention
Volunteers	2 years after last activity	Staff deletion (to be built, T-2.4)
Inquiries	1 year after handled	Scheduled purge (to be built, T-2.4)
Accounts	Until the user deletes them	Identity *Delete personal data*; donations keep SetNull links
Audit logs	5 years	Append-only

8. Seed data (Development only)

Roles `Donor` and `Employee` are created in every environment. In Development (or with `Seed:DemoData=true`): two demo accounts, six projects with offline funds, three volunteers (two approved, assigned to two projects), and three donations (two verified, one pending) so dashboards and reconciliation show both states.

Gift of the Givers · Relief Management System

DELIVERABLE 06

Implementation Plan

What has been delivered, what remains, and the order to build it in — with acceptance criteria and preliminary estimates.

DOCUMENT	VERSION	DATE	STATUS
D-06	2.0	2026-11-02	Current — Phase 0 complete, Phase 1 next

CONTENTS

1. Planning principles
2. Milestones
3. Phase 0 — Audit remediation (complete)
4. Phase 1 — Production candidate (next)
5. Phase 2 — Engineering quality and features
6. Phase 3 — Strategic
7. Risks to the plan

1. Planning principles

1. **Order by dependency, not just severity.** Tests and CI gates come before refactors; legal and merchant prerequisites come before real money.
2. **Every task closes a finding or a requirement.** Task IDs reference audit finding IDs (`F-nn`) or requirement IDs (`FR-nn` , `NFR-…`).
3. **Definition of done** (applies to every task): code with an integration test, CI green (tests, vulnerability gate, drift check, migrations on SQL Server), audit entries for staff actions, docs updated, finding status updated in `docs/audits/findings-register.json` .
4. **Estimates are preliminary** (one developer familiar with ASP.NET Core). The main uncertainties are external: PayFast onboarding, legal review and Azure access.

2. Milestones

Milestone	Exit criteria	Target maturity
M0 — Stabilised (done, 2026-11-02)	No open Critical findings; money integrity; staff access controlled; 98 tests; CI gates	Beta
M1 — Production candidate	F-23 closed (sandbox payment end to end), live environment verified, backups tested, legal review of privacy, staff MFA, monitoring and alerts	Production candidate
M2 — Production	Live merchant, PBO details, first real donation reconciled, runbooks rehearsed	Production ready
M3 — Growth	Recurring billing, live FX, retention automation, reporting	Mature

3. Phase 0 — Audit remediation (complete)

Task	Outcome	Findings closed	Status
T-0.1	Registration always creates Donor; Staff Access page; bootstrap staff via configuration	F-01	Done
T-0.2	Function key, reference lookup, verified-only certificates	F-02	Done
T-0.3	`DonationService` : pending capture, single verified increment, amount/merchant checks, idempotency	F-03, F-04, F-20	Done
T-0.4	Rand normalisation (`AmountZar` , rates) with historical backfill	F-05	Done
T-0.5	PayFast checkout wired; return, cancel and receipt by reference; options validator	F-06 (partial)	Done, E2E pending

Task	Outcome	Findings closed	Status
T-0.6	Certificates verified-only; issuer from configuration; demo watermark	F-07, F-40	Done
T-0.7	Patched crypto packages; source mapping; audit sources	F-08, F-24, F-25	Done
T-0.8	Privacy notice, consent capture, no invented data	F-09 (partial), F-32	Done, legal review pending
T-0.9	Validation and error display on all forms	F-16	Done
T-0.10	Rate limiting, security headers, SRI, Data Protection keys, forwarded headers	F-12, F-15, F-19, F-44	Done
T-0.11	Offline funds + Funding Reconciliation	F-03	Done
T-0.12	Audit log for all staff actions; paging; navigation	F-28, F-30, F-41	Done
T-0.13	Volunteer assignment UI; donation CSV export	FR-33, FR-34	Done
T-0.14	Migration chain fixed for empty databases; drift unsuppressed; remediation migration	F-11, F-21, F-22	Done
T-0.15	CI: locked restore, coverage, vuln gate, drift, SQL Server migrations; CD gated; publish fixed	F-14, F-34	Done
T-0.16	Live impact stats; organisation config; honest copy; accent contrast	F-18, D-03	Done
T-0.17	Docs: README, setup, contributing, security, PR template, deliverables 01–08	F-35	Done

4. Phase 1 — Production candidate (next)

Task	Problem / evidence	Desired outcome	Areas	Effort	Depends on	Acceptance criteria	Tests
T-1.1 Verify payments end to end	F-23: shared sandbox rejects signatures	Own PayFast sandbox merchant with passphrase; sandbox payment completes and the ITN verifies the donation	Config, ops	0.5–1 d	PayFast sandbox account; public tunnel or staging URL	A real sandbox payment shows *Paid*, the project total increases once, audit entry `PaymentVerified`	Manual script recorded in the addendum; keep the integration tests
T-1.2 Verify the live environment	F-10, F-02	App Service runs Production; demo accounts removed; old anonymous function removed	Azure	2–4 h	Azure access	`/ApplyDatabaseMigrations` absent; no demo logins; function returns 401 without a key	Smoke checklist
T-1.3 Staff MFA and lockout	F-36	Scaffolded Login with `lockoutOnFailure: true`; TOTP MFA required for Employees	Identity pages, policy	1–2 d	—	5 failed logins lock for 15 min; Employee pages redirect to MFA setup until enabled	Integration tests for lockout and the MFA policy
T-1.4 Backups and restore drill	F-17	Azure SQL PITR retention confirmed; documented restore to a copy	Ops, docs	0.5 d	Azure access	Restore performed and timed; RPO/RTO recorded in 07	Drill record
T-1.5 Monitoring and alerts	F-38	Application Insights (or OpenTelemetry); alerts on 5xx rate, health failures, ITN rejections	Program.cs, Azure	1 d	Azure access	Alert fires in a test; dashboards exist	Synthetic check
T-1.6 Production migration step	F-13	`Database:MigrateOnStartup=false`; pipeline applies `migrate.sql` before deploy	Pipelines	0.5 d	T-1.2	Deploy log shows the script step; app starts with no pending migrations	Pipeline run
T-1.7 Privacy sign-off	F-09	Legal review of the notice; Information Officer named; retention periods confirmed	Content, config	External	Organisation	Notice banner removed; `Organisation:InformationOfficerEmail` set	Content review
T-1.8 Reconcile live data	F-03	Funding Reconciliation run on the live database after deploy	Ops	1 h	T-1.6	All projects balanced or accepted with audit entries	Audit log
T-1.9 Review existing staff accounts	F-01	Self-registered Employee accounts revoked	Ops	1 h	T-1.2	Staff list matches the real team	Audit log

5. Phase 2 — Engineering quality and features

Task	Outcome	Findings / requirements	Effort
T-2.1	Recurring billing through PayFast subscriptions: subscription token, child donation per ITN, cancel from the donor portal	F-37, FR-18	4–6 d
T-2.2	Exchange-rate table with effective dates (staff-maintained or provider-fed); rate shown on the receipt	F-46	2–3 d
T-2.3	Extract `GiftOfTheGivers.Data` (DbContext, models, PDF) used by web and function; function deploy job and tests	F-29, F-42	2 d
T-2.4	Retention jobs (purge or anonymise per policy); staff deletion of volunteers and inquiries; data export for privacy requests	NFR-PRIV-03	2–3 d
T-2.5	Playwright smoke tests + axe accessibility checks in CI (donate, volunteer, register, staff pages; desktop and mobile)	NFR-A11Y-01, F-39	2 d
T-2.6	Move inline scripts and styles to files; strict CSP with nonces	F-19, D-01	1–2 d
T-2.7	Reporting: monthly totals per project and currency, volunteer pipeline	FR-34	2 d
T-2.8	Self-host fonts and icons	D-05	0.5 d
T-2.9	Load test (k6) of public pages and the ITN; record p95	NFR-PERF-02	1 d

6. Phase 3 — Strategic

Task	Outcome	Effort
T-3.1	Bicep IaC for App Service (Standard tier with slots), Azure SQL, Function App, Key Vault	3–4 d
T-3.2	Blue/green deploys with slot swap and automatic rollback on failed health	1–2 d
T-3.3	Consolidate on one CI system (GitHub Actions or Azure Pipelines)	1 d
T-3.4	Notification centre (email templates, retries via a queue)	3 d

7. Risks to the plan

Risk	Impact	Mitigation
PayFast onboarding delays	Blocks M1/M2	Start the merchant application now; keep the sandbox path working
Legal review unavailable	Blocks collecting real volunteer data	Keep `IsDemo` and the draft banner until signed off
Single maintainer	Bus factor	PR template, CI gates, these documents, audit register
Free hosting tier	No slots, cold starts	T-3.1 before real traffic

Gift of the Givers · Relief Management System

DELIVERABLE 07

CI/CD and Release

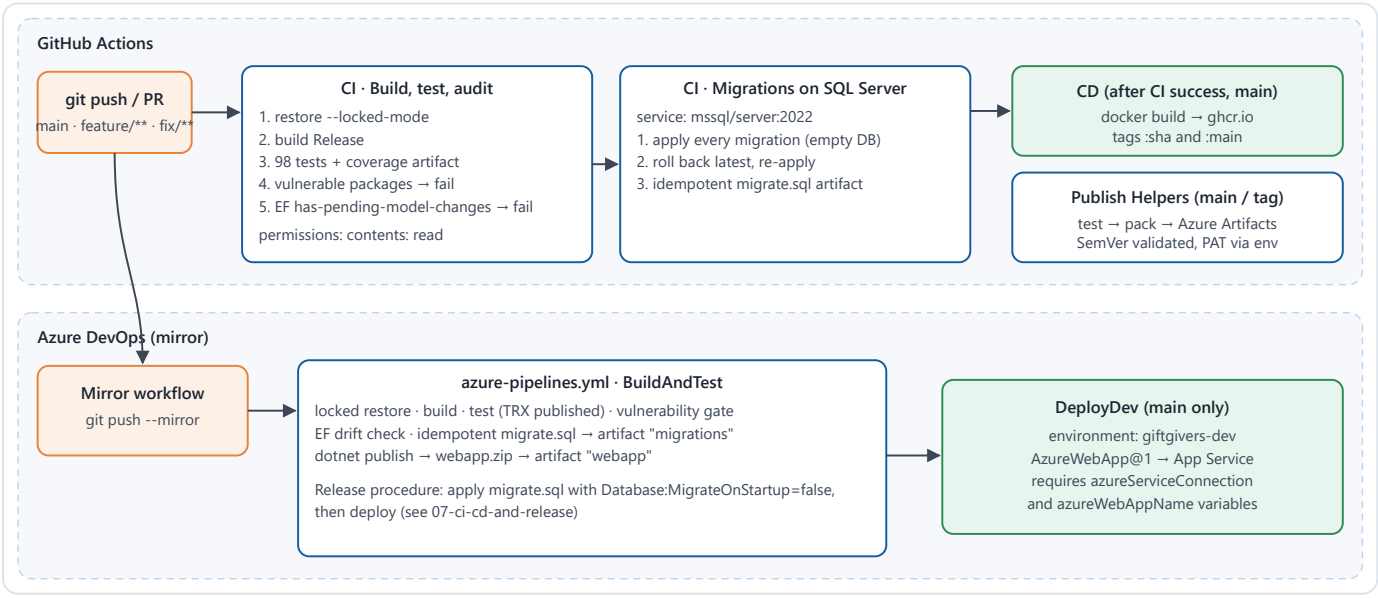
Pipelines, quality gates, secrets, environments, the release procedure, migrations, rollback, backup and recovery.

DOCUMENT	VERSION	DATE	STATUS
D-07	2.0	2026-11-02	Current — pipelines updated during remediation; release steps partly manual

CONTENTS

1. Pipeline overview
2. Secrets and variables
3. Branching and protection
4. Environments
5. Release procedure
6. Rollback
7. Backup and disaster recovery
8. Monitoring and incident response

1. Pipeline overview



CI/CD pipelines as configured in the repository

Pipeline	File	Trigger	Gates / outputs
CI — Build, test, audit	`.github/workflows/ci.yml`	Push to `main`, `feature/**`, `fix/**`; PR to `main`	Locked restore → Release build → 98 tests + coverage (artifact) → **fail on vulnerable packages** → **fail on EF model drift**
CI — Migrations on SQL Server	same file, job `migrations`	Same	SQL Server 2022 service container → apply all migrations to an empty DB → roll back the latest and re-apply → `migrate.sql` (idempotent) artifact
CD	`.github/workflows/cd.yml`	`workflow_run`: CI completed **successfully** on `main` (push)	Container image to GHCR `:sha` and `:main`, OCI labels
Mirror	`.github/workflows/mirror-to-azure-devops.yml`	Any push	`git push --mirror` to Azure DevOps
Publish Helpers	`.github/workflows/publish-helpers-package.yml`	`main`, tags `helpers-v*`, manual	SemVer check → explicit `AZURE_ARTIFACTS_PAT` preflight and feed authentication → helper tests → pack → push to Azure Artifacts
Azure Pipelines	`azure-pipelines.yml`	Mirrored `main`, `feature/*`; PR to `main`	Locked restore, build, test (TRX), vuln gate, drift check, `migrations` artifact, `webapp` zip → **DeployDev** on `main`

All GitHub workflows declare least-privilege `permissions`. CD is the only one with `packages: write`.

> **Status.** These pipeline definitions were changed during remediation and have not yet run on GitHub or Azure DevOps. Their steps were executed locally where possible (restore, build, tests, vulnerability list, drift check, migrations on LocalDB, idempotent script). Record the first green runs in the next audit.

2. Secrets and variables

Name	Where	Used by	Notes
`GITHUB_TOKEN`	GitHub (automatic)	CD	`packages: write` only in CD
`AZURE_DEVOPS_REPO_URL`, `AZURE_DEVOPS_PAT`	GitHub secrets	Mirror	PAT scope: Code read and write
`AZURE_ARTIFACTS_PAT`	GitHub secret	Publish Helpers	Required; the workflow fails early with a setup hint if missing. PAT scope: Packaging read and write; passed via `env`
`azureServiceConnection`, `azureWebAppName`	Azure DevOps variables	DeployDev	Deploy is skipped if empty
App settings (connection string, `PayFast__*`, `SendGrid__ApiKey`, `Organisation__*`, `StaffAccess__BootstrapEmployeeEmails__0`)	App Service configuration / Key Vault	Runtime	Never in the repository

The CI migrations job uses a throwaway SQL Server password that exists only inside the job's container. It is not a secret.

3. Branching and protection

- Branches: `feature/<name>`, `fix/<name>`; PRs into `main` using the PR template.
- **Required on `main`:** one approval; status checks `Build, test, audit` and `Apply migrations to SQL Server`; no direct pushes; linear history recommended.
- Tag releases as `vYYYY.MM.N` (web app) and `helpers-vX.Y.Z` (package).

4. Environments

Environment	Hosting	Database	Deploy source	Config highlights
Local	`dotnet run` / Docker Compose	LocalDB / container	Developer	Development, demo data, sandbox
CI	GitHub runner	SQL Server 2022 service	—	Throwaway
Dev (`giftgivers-dev`)	Azure App Service (Free F1 today)	Azure SQL	Azure Pipelines `DeployDev`	**Production** environment name, sandbox payments, `IsDemo=true`
Production (planned)	App Service Standard with slots	Azure SQL (PITR)	Same artifact, after approval	Live PayFast (passphrase required), `IsDemo=false`, `MigrateOnStartup=false`, `ForwardedHeaders:Enabled=true` on Linux

5. Release procedure

1. **Prepare.** Merge to `main` with CI green. Note the commit SHA and the finding IDs closed.
2. **Database.** Download the `migrations/migrate.sql` artifact from the run. Review it for destructive statements. Take an on-demand backup (or note the PITR timestamp). Apply the script to the target database: it is idempotent, so applying it twice is safe.
3. **Deploy** the `webapp` artifact (Azure Pipelines `DeployDev`, or the GHCR image) with `Database:MigrateOnStartup=false`.
4. **Verify (smoke checklist):**
   - `GET /health` returns `Healthy` (database reachable) and `/health/live` returns 200.
   - Home, Projects, Donate and Privacy return 200; response headers include CSP and `X-Frame-Options: DENY`.
   - Make a sandbox donation (or a live R 5 test in production), confirm PayFast returns to the receipt, and that the donation shows *Paid* after the ITN.
   - The Employee dashboard shows no unexpected *projects needing review*; the audit log shows the `PaymentVerified` entry.
5. **Record** the release (tag, date, SHA, migration ids, verification result) in the release notes.

Release notes template

```
Release vYYYY.MM.N – <date> – <sha>
Changes: <summary>; closes F-nn, F-nn
Database: migrations <ids> (backfill: yes/no; destructive: yes/no)
Config changes: <keys>
Verification: health ✓, smoke ✓, sandbox/live payment ✓ (<reference>)
Rollback point: <PITR timestamp / previous artifact>
```

6. Rollback

Situation	Action
App defect, schema unchanged	Redeploy the previous artifact (or swap slots once T-3.2 exists)
App defect after an additive migration	Redeploy the previous artifact. Additive columns are ignored by older code
Migration defect	`dotnet ef database update <previous migration>` generates the `Down()` script. The remediation migration's `Down` has been tested (LocalDB). For data loss, restore to a new database from PITR and switch the connection string
Payment issue	Set `PayFast:Enabled=false` (donations are recorded as Pending and nothing is charged), investigate, re-enable

Backward compatibility: migrations must be additive in the release that introduces them. Drop columns one release later, after no running code uses them.

7. Backup and disaster recovery

Item	Target (proposed)	Current state
Database backups	Azure SQL automated backups with point-in-time restore, retention ≥ 14 days	Depends on the Azure SQL tier — **not verified**
RPO	≤ 24 h (PITR gives minutes)	Not measured
RTO	≤ 4 h	Not measured
Restore drill	Quarterly, restore to a copy and run the smoke checklist	**Never performed** (finding F-17, task T-1.4)
Configuration backup	App settings exported to Key Vault / IaC	Manual (T-3.1)
Data Protection keys	In the database (backed up with it)	Done

Restore steps (Azure SQL): Portal → SQL database → *Restore* → point in time → new database name → update the App Service connection string → restart → smoke checklist → record the timings.

8. Monitoring and incident response

Signal	Source	Current	Planned
Liveness / readiness	`/health/live`, `/health` (DB check)	Available	App Service health check + alert (T-1.5)
Request logs	Serilog request logging (console)	Available	Application Insights
Payment problems	Log warnings/errors for rejected ITNs, amount or merchant mismatch	Logged	Alert on any rejection
Data integrity	Dashboard card "projects whose funding total needs review"	Available	Weekly review
Security events	Audit log (staff grants and revokes, exports)	Available	Alert on `EmployeeGranted`

Incident checklist:

1. Assess impact (payments? personal data?).
2. Contain (disable payments, revoke staff access, rotate secrets).
3. Preserve logs and audit entries.
4. Fix.
5. Reconcile funding.
6. Notify (POPIA requires notifying the Information Regulator and affected people after a personal-data compromise).
7. Add a finding to the register.

Gift of the Givers · Relief Management System

DELIVERABLE 08

Professional Engineering Audit

Baseline audit **AUD-2026-11-01** — evidence-based review of the system as found at commit `efa2b02` on `feature/erd-schema-and-pdf`.

DOCUMENT	VERSION	DATE	STATUS
D-08	1.0 (frozen baseline)	2026-11-02	Baseline — superseded by addendum

CONTENTS

Audit metadata
4. Executive Summary
5. Product Requirements and Scope Audit
6. Project Structure and File Inventory
7. System Architecture Review
8. System Integration and Data Flow
9. Architecture Decisions and Technical Debt
10. Web Frontend Audit
11. UI/UX and Accessibility
12. Browser Compatibility
17. Backend Application Audit
18. API Inventory
19. API Contract Consistency
20. API Security
21. External Integrations
22. Database Architecture
23. Database ↔ Application Consistency
24. Transactions, Concurrency and Integrity
25. File and Object Storage
26. Data Protection and Privacy (South Africa, POPIA)
27. Identity and Access Control
28. Application Security Audit
29. Threat Model (STRIDE, prioritised)
30. Cross-Platform Feature Completeness
31. End-to-End Workflows
32. Automated Testing Audit
33. Recommended Test Strategy (risk-ordered)
34. Code Quality and Maintainability
35. Dependency and Supply-Chain Audit
36. Performance
37. Scalability and Concurrency
38. Reliability and Resilience
39. Logging, Monitoring and Observability
40. Build and Development Workflow
41. CI/CD Pipeline Audit
42. Git and Collaboration
43. Infrastructure and Deployment
44. Cloud Architecture and Cost
45. Environments, Secrets and Configuration
46. Release, Rollback and Migration Safety
47. Backup and Disaster Recovery
48. README Audit — Rating: **Adequate**
49. Documentation Audit
50. Developer Experience
51. Ongoing Development Readiness
52. Technical Debt Register
53. Demo, Placeholder and Dead-Code Audit
54. Governance and Delivery Risks
55. Feature and Engineering Completeness Matrix
56. Risk Register
57. Prioritised Engineering Roadmap
58. What Should Be Preserved
59. Professional Scorecard
60. Final Professional Verdict

Read this with the addendum. This document records the system *as it was found* and is kept unchanged as the baseline for future audits. Remediation work, verification evidence, new findings and the re-scored scorecard are in 08 — Audit Addendum (Remediation Review). Risk IDs R1–R18 below map to stable finding IDs in `docs/audits/findings-register.json`.

Audit metadata

Item	Value
Repository	`jBlack-MC/GiftOfTheGivers-HumanitarianAid` (GitHub; mirrored to Azure DevOps)
Branch audited	`feature/erd-schema-and-pdf`
Commit	`efa2b02d252399706574f7fca6eda43a298fd991` (45 commits in history)
Audit date	2026-11-02
Method	Static inspection of every first-party source, view, config, migration, test, workflow and doc file, plus local execution of the existing restore, build and test commands
Working tree	Clean before and after the audit. No project files were modified; this report is the only file added.

Applications and services discovered

Project	Type	Purpose
`GiftOfTheGivers/`	ASP.NET Core MVC + Razor Pages (Identity UI), .NET 10	The whole product: public site, donor portal, employee portal
`GiftOfTheGivers.Helpers/`	.NET class library (packaged to Azure Artifacts)	Three small calculation helpers
`GiftOfTheGivers.Functions/`	Azure Functions (isolated worker, v4)	One HTTP function that generates a tax-certificate PDF
`GiftOfTheGivers.Tests/`	xUnit	Unit tests

Technologies detected (with evidence): C# / .NET SDK 10.0.401 (`global.json`), EF Core 10.0.12 on SQL Server / LocalDB / Azure SQL, ASP.NET Core Identity (cookie auth, roles), Bootstrap 5 + jQuery validation (vendored in `wwwroot/lib`), QuestPDF, SendGrid, PayFast (South African payment gateway), Serilog (console sink), Docker / Docker Compose, GitHub Actions (4 workflows), Azure Pipelines, Azure App Service (README), GHCR, Azure Artifacts.

Not present (verified by search): mobile applications, desktop applications, SPA framework, GraphQL/gRPC/WebSockets, queues or background workers, infrastructure-as-code, Kubernetes, file upload, monitoring/APM integration, backup configuration.

Checks actually executed

Command	Result
`dotnet restore GiftOfTheGivers.slnx --locked-mode`	**Succeeded.** NuGet audit raised **NU1903 (high)** and **NU1902 (moderate)** for transitive `System.Security.Cryptography.Pkcs` 9.0.11, plus NU1900 (it could not reach the Azure Artifacts feed for vulnerability data).
`dotnet build GiftOfTheGivers.slnx -c Release --no-restore`	**Succeeded:** 0 errors, 11 warnings (all NuGet). All four projects compiled, including Functions.
`dotnet test GiftOfTheGivers.slnx -c Release --no-build`	**Passed:** 19 / 19 tests, about 2 s.
Secret-pattern scan of the full git history (values never printed)	No real credentials found. Historical Azure SQL strings are `<placeholder>` values. The PayFast ID/key in `appsettings.json` match PayFast's public sandbox merchant.

Not executed: the app was not run in a browser, and no live site, database, PayFast, SendGrid, Azure or CI system was contacted. Migrations were not applied, and CI pipelines were not observed running. Any statement about runtime UI behaviour, the live deployment or pipeline success comes from code inspection only and says so.

Exclusions and limitations

- Vendored `wwwroot/lib/**` (Bootstrap, jQuery, jquery-validation) was not reviewed line by line.
- Migration `*.Designer.cs` files and `ApplicationDbContextModelSnapshot.cs` are generated. They were inspected only to check that they match the model and that EF can discover each migration.
- `wwwroot/intro/intro.js` (~400 lines of animation) was skimmed for redirect, skip and reduced-motion behaviour only.
- No requirements document, rubric, user stories or issue tracker is in the repo. Code comments refer to "Section B ERD" and "Portfolio of Evidence Part 1", which suggests an academic brief that was **not supplied**. Requirements below come from the README and code comments only.
- **Live deployment:** Unable to verify from the available evidence which environment (`ASPNETCORE_ENVIRONMENT`), database or configuration the README's live Azure site uses.

PART I — PROJECT AND PRODUCT ASSESSMENT

4. Executive Summary

1. **Intended purpose.** A management web app for a humanitarian NGO. It covers public information, donations (ZAR/USD/EUR, one-time or recurring), volunteer applications, contact inquiries, a donor portal (history and Section 18A tax certificates) and an employee portal (projects, project updates, volunteer approval, donation oversight).
2. **Intended users.** The anonymous public, registered donors, and NGO staff ("Employee" role).
3. **Composition.** One ASP.NET Core MVC monolith, one shared SQL Server database, one Azure Function, a helpers NuGet library, and xUnit tests.
4. **What works (by code inspection; partly by tests):** public browsing; volunteer and contact submissions persisted to the DB; employee CRUD for projects and project updates; volunteer approve/reject; donor history; PDF certificate generation; role-based controller authorization; global anti-forgery tokens; a well-built EF model with check constraints, indexes and a rowversion.
5. **Partially implemented:**
   - Payments: a PayFast service and an ITN webhook exist, but the donation flow **never redirects to PayFast**.
   - Email: a SendGrid service exists but needs configuration.
   - Audit logging: only certificate views are recorded.
   - Recurring donations: a label only.
6. **Unfinished, disconnected or misleading:**

- Donation totals are inflated by unpaid donations.
- Tax certificates are issued for unpaid donations and carry a fake tax registration number.
- The Privacy page is template placeholder text.
- Home and About statistics are hard-coded.
- The Inquiries and Audit Log pages are not linked anywhere.
- One migration file is orphaned.

7. **Engineered well:** the data model and constraints, the use of input models (no over-posting), server-side ownership checks in the web app, lock files, a pinned SDK and tooling, Docker Compose with a health check, the Serilog setup, and the PayFast signature service.
8. **What blocks safe production use** (detail in §56):
   - **Anyone can self-register as Employee** (staff) and read all volunteer and donor PII.
   - **An anonymous Azure Function returns any donor's tax certificate by sequential ID.**
   - **Project funding totals can be inflated by anyone** without paying, and are double-counted when a payment does complete.
   - **The payment flow is not connected.**
   - There is no privacy notice or consent, even though the app collects dates of birth, addresses and emergency contacts (POPIA applies in South Africa).
   - A dependency has a known high-severity advisory.
9. **Priority:** fix authorization (registration and the Function), then donation and funding integrity, then the payment wiring, then tests for those flows.
10. **Health for ongoing development:** *Moderately healthy.* The codebase is small, consistent and compiles cleanly, so incremental work is realistic. The blockers are logic and authorization defects, not structural ones.

Maturity classification: Functional prototype. All core screens and CRUD work. Money, privilege and privacy flows are not production-grade, and the code says so itself (`TaxCertificatePdf.cs:9–11`, `Donate.cshtml` "This is a prototype. No actual payment will be processed.").

Measure	Estimate	Basis and uncertainty
Feature implementation	~65% of README-claimed features	Payments, recurring donations, emailed certificates and reporting are missing or partial. Medium confidence.
Test coverage confidence	Low (~10% of critical paths)	19 unit tests. None cover Donate POST, payment notify, authorization, registration or any employee action. No coverage report exists.
Documentation completeness	~55%	The README is detailed but contradicts `DATABASE_SETUP.md`, overclaims features, and lacks architecture, API, operations and security docs.
Production readiness	~20%	Two critical authorization defects, a data-integrity defect, an unwired payment flow, and no privacy controls.

5. Product Requirements and Scope Audit

There is no formal specification. The sources are `README.md` §Features and code comments ("Section B" ERD, "Part 1 prototype"). All requirements below are **inferred from the README** unless noted.

Requirement	Source	Implementation evidence	Test evidence	Status	Gap	Priority
Browse relief projects	README	`HomeController.ReliefProjects` (`HomeController.cs:51`)	None	Implemented, unverified	No pagination	P3
Multi-currency donation (ZAR/USD/EUR)	README	`HomeController.Donate` POST (`:84–170`); DB check `CK_Donation_Currency`	None	Partial	Currencies are summed together into one `FundsRaised` and shown with `ToString("C")`	P1
One-time and recurring donations	README	`DonationType` string only	None	**Label only**	No schedule, mandate or subscription; README's "Recurring donation management" is not implemented	P2 (roadmap)
Payment processing	README note; `PayFastPaymentService`	`BuildCheckoutRedirectUrl` is never called (grep). The ITN webhook exists (`PaymentsController.cs`).	1 signature test	**Disconnected**	Donations stay `Pending` forever; nothing collects money	P0/P1
Volunteer registration	README	`HomeController.Volunteer` POST (`:272`)	None	Implemented, unverified	No consent; a missing DOB is replaced with a fabricated date (`:294`)	P1
Contact form	README	`HomeController.Contact` (`:327`), `Inquiry` entity	None	Implemented, unverified	Inquiries page is not linked in the UI	P2
Donor dashboard and history	README	`DonorController.Dashboard/MyDonations`	None	Implemented, unverified	Payment status not shown; totals mix currencies	P2

Requirement	Source	Implementation evidence	Test evidence	Status	Gap	Priority
Tax certificate generation	README	`TaxCertificatePdf.cs`, Donor/Home controllers, Function	2 PDF magic-byte tests	Partial / **unsafe**	Issued for unpaid donations; Function is anonymous; fake registration number	P0
Certificate "sent to your email"	`DonationSuccess.cshtml:64`, `Donate.cshtml`	Not implemented	None	**Missing**	Misleading UI claim	P2
Employee: project CRUD and updates	README	`EmployeeController.cs:77–288`	None	Implemented, unverified	No delete; no audit trail	P2
Employee: volunteer review	README	`ApproveVolunteer/RejectVolunteer` (`:317–350`)	None	Implemented, unverified	No assignment UI for `VolunteerAssignment` (seed only)	P2
Employee: donation oversight and reporting	README	`EmployeeController.Donations` (list only)	None	Partial	No filter, export or report; mixed-currency totals	P2
Role-based access (Donor/Employee)	README	`[Authorize(Roles= … )]` on controllers	None	**Broken by design**	Employee role is self-selectable at registration	P0
Audit log	Code only (`AuditLog`, `IAuditService`)	Logs certificate views only	None	Partial	Undocumented; mutations not audited	P2
Volunteer↔project assignment (ERD)	Code comments ("Section B 3.6")	Entity + seed only	None	Schema only	No UI or endpoint	P3

Inconsistent terminology.

- Volunteer rejection sets `Rejected`, but the flash message says "marked as inactive" (`EmployeeController.cs:348`).
- Donation success uses the `Donate/Confirmation` route, but `PayFast:ReturnUrl` points there too, so it serves both a receipt and a payment return.
- Two parallel certificate routes exist: `/Donor/TaxCertificate` and `/Donate/Tax-Certificate/{id}`.

6. Project Structure and File Inventory

File or directory	Purpose	Main dependencies	Status	Risk	Notes
`GiftOfTheGivers/Program.cs`	Composition root and pipeline	EF, Identity, Serilog, SendGrid	Implemented	Medium	Suppresses `PendingModelChangesWarning` (`:27`); migrates on startup; no rate limiting or security headers
`Controllers/HomeController.cs` (367 lines)	Public pages, donate, volunteer, contact, certificate routes	DbContext, audit, email	Implemented, **defects**	High	Fat controller; funding-integrity bug
`Controllers/DonorController.cs`	Donor portal	DbContext, audit	Implemented	Medium	Action-level role widening is ineffective (§27)
`Controllers/EmployeeController.cs` (406 lines)	Staff portal	DbContext, email	Implemented, unverified	Medium	No pagination; no audit
`Controllers/PaymentsController.cs`	PayFast ITN webhook	`IPaymentGateway`, email	Implemented, **unreachable in practice**	High	Double-counts funds (§24)
`Services/PayFastPaymentService.cs`	Signature build/verify, validate call	HttpClient, MD5	Partially implemented	Medium	Checkout builder unused
`Services/SendGridEmailService.cs`, `NoOpEmailService.cs`	Email	SendGrid	Implemented, unverified	Low	NoOp is used when no key is set
`Services/AuditService.cs`	Audit persistence	DbContext	Implemented	Low	Calls `SaveChanges` itself, which can flush unrelated tracked changes
`Services/TaxCertificatePdf.cs`	PDF	QuestPDF	Complete, tested (smoke)	Medium	"Not a valid tax document" footer
`Data/ApplicationDbContext.cs`	EF model	EF Core	**Well designed**	Low	Constraints, indexes, delete behaviours

File or directory	Purpose	Main dependencies	Status	Risk	Notes
`Data/DbSeeder.cs`	Migrate + dev seed	Identity	Implemented	Medium	Migrates in every environment; roles only seeded in Development
`Data/ApplicationDbContextFactory.cs`	Design-time factory	—	Implemented	Low	Hard-coded LocalDB string; ignores user-secrets
`Data/Migrations/` (7 migrations)	Schema history	—	Mostly fine	Medium	`20260924190000_P0SecurityAndDonationIntegrity.cs` is **orphaned**; `FixPendingModelChanges` is empty
`Models/` , `Models/ViewModels/`	Entities and input models	DataAnnotations	Implemented	Low	Good separation of input models
`Areas/Identity/Pages/Account/Register.*`	Scaffolded registration with role choice	Identity	Implemented, **insecure**	Critical	§27
`Views/**` (32 views)	Razor UI	Bootstrap	Implemented	Medium	Placeholder Privacy page, fake stats
`wwwroot/intro/`	Animated intro, redirected to on first home visit	—	Implemented	Low	Has reduced-motion support and a Skip button
`GiftOfTheGivers.Functions/`	Anonymous PDF function	Web project reference	Implemented, **insecure**, deployment unknown	Critical	§20; no deploy pipeline
`GiftOfTheGivers.Functions/local.settings.json`	Local function settings	—	Committed (no secrets)	Low	Should be git-ignored; `local.settings.example.json` already exists
`GiftOfTheGivers.Helpers/`	Calculation helpers + NuGet package	—	Complete, tested	Low	Very small to justify a separate package
`GiftOfTheGivers.Tests/`	19 tests	xUnit, EF InMemory	Implemented	High (gap)	No critical-path tests
`.github/workflows/*.yml` (4)	CI, CD to GHCR, ADO mirror, package publish	—	Partially verified	Medium	§41
`azure-pipelines.yml`	ADO build/test/deploy	—	Unable to verify execution	Medium	Likely failing on the vulnerability gate
`docker-compose.yml` , `GiftOfTheGivers/Dockerfile`	Local container stack	SQL Server 2022	Implemented	Low/Medium	Dev-only settings
`setup.ps1` , `setup.sh`	Onboarding scripts	dotnet-ef	Implemented	Low	Print demo passwords
`README.md` , `CONTRIBUTING.md` , `DATABASE_SETUP.md`	Docs	—	Partially accurate	Medium	§48
`GIT_SETUP_GUIDE.md`	One-off "initial commit" guide	—	**Stale**	Low	References commit `91bed65` and a different repo name
`allfiles.txt` , `helpers.txt` , `testlog.txt`	Scratch output	—	**Temporary artifacts**	Low	Contain absolute local paths (`D:\Projects\...`); `testlog.txt` is UTF-16 and stale (23 tests vs 19 now)

Dead or unreachable code:

- `IPaymentGateway.BuildCheckoutRedirectUrl` has no caller.
- The orphan migration is never discovered by EF.
- The `NuGet.Protocol` package reference is never used.
- `VolunteerApplicationViewModel.FirstName/LastName` are never bound by the view.
- `DonateViewModel.DonorName` is validated but never stored.
- `ViewBag.DonorName/DonorEmail` in `Confirmation` are never set.
- The `Inquiries` and `AuditLog` employee pages are not linked from any view.

PART II — ARCHITECTURE AND SYSTEM DESIGN

7. System Architecture Review

Actual architecture: a layered **monolith** (MVC controllers → EF DbContext, with a thin service layer for cross-cutting integrations), plus one satellite Azure Function that **references the web project directly** to reuse `ApplicationDbContext` and `TaxCertificatePdf`.



Diagram source (Mermaid) as recorded at the audit date

> Dashed `U → PayFast` is intentionally absent: nothing in the code sends the user to PayFast.

Assessment.

- **Business logic placement:** mostly in controllers (`HomeController.Donate` holds transaction, idempotency and funding logic). That is acceptable at this size, but it is why the funding rule is duplicated in two controllers and diverges (§24).
- **Dependency direction:** `Functions → Web` is inverted. A Functions app depending on an `Sdk.Web` project pulls the whole MVC app, Identity and its vulnerable transitive deps into the function. The usual fix is a `GiftOfTheGivers.Core` / `Data` library.
- **Helpers:** `GiftOfTheGivers.Helpers` is a clean leaf. Publishing it as a NuGet package adds pipeline complexity for three functions with no external consumer in evidence.
- **Design fit:** the design is right for the present scale. **Do not** move to microservices. Extracting a donation service class and a shared data project is enough.

Preserve: input-model separation, the EF configuration, the `IPaymentGateway` and `IEmailService` abstractions with a no-op fallback, and the global anti-forgery filter.
Constrains development: the controller-held domain logic, the Functions→Web reference, and the suppressed pending-model-changes warning.

8. System Integration and Data Flow

Donation (actual): user submits `Donate` form → `HomeController.Donate` POST → idempotency lookup → insert `Donation(PaymentStatus=Pending)` → `project.FundsRaised += Amount` **immediately** → commit → redirect to `Donate/Confirmation` (receipt from TempData) → **no payment step**.

Where it breaks:

- The payment step is missing.
- The funding total changes before money exists.
- If PayFast were wired up, the ITN handler (`PaymentsController.cs:52`) would add the amount **a second time**.
- A `FAILED` ITN does not reverse the earlier increment (`:61`).

Tax certificate: a link from the receipt or donor pages → ownership check → PDF. The Function path bypasses identity entirely.

Volunteer approval: employee clicks Approve → status saved → email sent after commit. If SendGrid throws, the status change has already persisted but the user sees a 500 error. Medium risk; the SendGrid client does not throw on non-2xx by default, only on network errors.

Identifiers: donations use an int `Id` (sequential, guessable) for certificate URLs, and a GUID `TransactionReference` for PayFast. That GUID would be the safer external identifier.

9. Architecture Decisions and Technical Debt

Decision	Current	Recommendation (ADR to write)
ADR-1 Payment confirmation is the only source of `FundsRaised`	Incremented in two places	Increment only on verified ITN; or compute from `SUM(Verified donations)` by currency
ADR-2 Role provisioning	Self-service	Employee by admin invite or approval only; seed an admin from config
ADR-3 Currency model	Mixed amounts summed	Store `AmountZar` (converted at capture with rate and source), or keep per-currency totals
ADR-4 Shared data library	Functions references Web	Extract `GiftOfTheGivers.Data` (+ PDF service)
ADR-5 Migrations at startup	`MigrateAsync` on every boot, all environments	Run migrations as a pipeline step or idempotent script in prod; keep startup migration for dev only
ADR-6 Certificate eligibility	Any donation	Only `Verified` donations; sequential certificate numbers per tax year

---

PART III — WEB APPLICATION

10. Web Frontend Audit

Server-rendered Razor with Bootstrap 5. Client-side scripting is minimal (`site.js` , inline scripts on Donate).

Finding	Evidence	Impact	Severity	Confidence
No `asp-validation-summary` in any MVC view. Model-level errors added by the controller ("funding total just changed", `HomeController.cs:160`) are **never displayed**	grep: `asp-validation-summary` only in `Areas/Identity/ … /Register.cshtml`	User sees the form again with no explanation	Medium	High
Donor name and email inputs are raw HTML, not `asp-for` ; there are no validation spans for them, `Currency` or `ReliefProjectId`	`Donate.cshtml:100,105` ; no `asp-validation-for` for those fields	Values lost and errors invisible on redisplay	Medium	High
Donation-type radios and the currency `<select>` are hard-coded to the defaults (`checked` / `selected`)	`Donate.cshtml:35,41,68–71`	A re-displayed form silently resets to OneTime/ZAR. A user could resubmit as ZAR an amount they meant as USD	Medium	High
"PayPal" payment option offered; not supported by PayFast integration	`Donate.cshtml:114`	Misleading	Low	High
Contradictory trust messaging: "This is a prototype. No actual payment will be processed" next to "Secure donation processing" and "you'll receive a tax-deductible certificate via email"	`Donate.cshtml` (Note, Security Badge, Tax Certificate card)	Misleading if publicly hosted	Medium	High
Hard-coded impact statistics ("1000+", "50,000+", "25+", "5000+")	`Index.cshtml:227–239` , `About.cshtml:303–324,360`	Presented as facts; not data-driven	Low	High
Fake organisation details: "Tax Registration: 123-456-789", phone "+27 (0) 12 345 6789", "123 Main Street, Cape Town"	`Donor/TaxCertificate.cshtml:84` , `_Layout.cshtml` footer	On a public site, impersonation-adjacent content using a real NGO's name	Medium	High
Privacy page is template text ("Use this page to detail your site's privacy policy.")	`Views/Home/Privacy.cshtml`	Legal/compliance gap (§26)	High	High
Inquiries and Audit Log pages have no navigation link	grep of all `asp-action` values	Features undiscoverable	Low	High
`Donor/TaxCertificate.cshtml` back-links use `asp-action` without a controller. Rendered from `/Donate/Tax-Certificate/{id}` (Home controller), they target non-existent `Home/DonationDetails` and `Home/MyDonations`	`TaxCertificate.cshtml:93,96` ; view shared by `HomeController.cs:233`	Broken links (empty href)	Low	Medium (needs a browser check)
Money shown with `ToString("C")` , which depends on the server culture. On Linux containers or invariant culture it renders `¤` / `$` for ZAR values, and mixed currencies are summed	`Employee/Donations.cshtml:24,28,75,96`	Wrong financial display	Medium	High (code); runtime symbol unverified
No pagination on any list (projects, donations, volunteers, inquiries)	`EmployeeController.cs:291,353,363`	Degrades with data growth	Low now	High
Intro animation: JS redirect from `/` to `/intro/index.html` on the first visit of each session	`_Layout.cshtml:8–21`	Extra round-trip; crawlers and no-JS users unaffected; skip and reduced-motion supported	Low	High
Empty `<script type="importmap">`	`_Layout.cshtml:22`	Harmless leftover	Info	High

Positive: global anti-forgery (`Program.cs:50`), unobtrusive validation partials, consistent layout, role-aware navigation, and Razor auto-encoding (no `Html.Raw` anywhere; verified by grep).

11. UI/UX and Accessibility

Static review only; no browser or automated accessibility tool was run.

- **Positive:** `lang="en"` , viewport meta, `aria-label` on the navbar toggler, Identity forms with `autocomplete` / `aria-required` , confirm dialogs on approve/reject, reduced-motion handling in the intro, decorative logos with `alt=""` , and no `<img>` without `alt` (grep).
- **Gaps:**
  - Donate radio groups lack a `<fieldset>/<legend>` .
  - Validation errors are not announced (no summary or `role="alert"` in MVC views).
  - Quick-amount buttons change the amount without announcing it.
  - Colour contrast of `.text-muted` on the dark footer and on brand colours is **unable to verify** without rendering.
  - The project card image has an inline fixed `height: 220px` .
- WCAG 2.2 AA conformance: **unable to verify**. Recommend axe-core or Playwright checks in CI (§33).

12. Browser Compatibility

- Standard Bootstrap 5 and ES2015+ (`const` , arrow functions, `forEach` on NodeList) in inline scripts; fine for evergreen browsers.
- `sessionStorage` access is wrapped in try/catch (good).
- External CSS from Google Fonts and jsDelivr has **no Subresource Integrity attribute** and no fallback. If the CDN is unavailable, icons disappear (`_Layout.cshtml:25,27`).
- **No cross-browser testing evidence exists.**

PART IV — MOBILE APPLICATIONS (§13–16)

Not applicable. No Android, iOS, Flutter, React Native, MAUI, PWA manifest or service worker exists (verified by file listing). "Mobile" in the README means a responsive Bootstrap layout. Responsive behaviour on device-sized viewports is **unable to verify** without a browser.

PART V — BACKEND AND API

17. Backend Application Audit

Finding	Evidence	Expected	Severity	Confidence
Funding totals updated on unpaid donations	`HomeController.cs:132–145`	Update only on verified payment	**Critical**	High
Double increment on ITN COMPLETE	`PaymentsController.cs:47–53`	A single source of truth	**High**	High
FAILED ITN does not reverse the earlier increment	`PaymentsController.cs:59–64`	Compensate or never pre-increment	High	High
Donation saved **before** the project existence check. A non-existent `ReliefProjectId` fails on the FK at the first `SaveChangesAsync` (`:130`) and returns a 500, so the friendly "no longer exists" branch (`:135–141`) is effectively unreachable	`HomeController.cs:129–141` ; `DeleteBehavior.Restrict` FK	Validate the project (and that it is **Active**) before insert	Medium	High
Donations accepted for Completed/Suspended projects (project ID only filtered in the dropdown)	`HomeController.cs:105`	Server-side status check	Medium	High
`Currency` , `DonationType` and `PaymentMethod` are not validated against allowed values in the view model. Invalid values hit DB check constraints or the `nvarchar(50)` limit and return a **500** instead of a validation error	`DonateViewModel.cs:12–38` ; `ApplicationDbContext.cs:32–34`	`[AllowedValues]` / enum binding	Medium	High
`Amount` range is `1..double.MaxValue` , but the column is `decimal(18,2)` . Very large values overflow and return a 500; there is no business maximum	`DonateViewModel.cs:22`	Realistic max (e.g. 10,000,000)	Medium	High
Idempotency is check-then-insert; concurrent duplicates hit the unique index and return a 500	`HomeController.cs:92–103` ; unique index `ApplicationDbContext.cs:53`	Catch the unique violation and return the existing donation	Low	High
`DonorName` required but discarded; anonymous donors have no name on record	`DonateViewModel.cs:29` vs `Donation`	Persist or remove the field	Low	High

Finding	Evidence	Expected	Severity	Confidence
A missing volunteer DOB is replaced with "today minus 18 years" (fabricated PII)	`HomeController.cs:294`	Make DOB nullable or required	Medium	High
GET request mutates state (`TaxCertificateIssued = true`)	`HomeController.cs:225-230`	Issue the certificate via an explicit POST or on payment verification	Low	High
Mixed time bases: `DateTime.Now` (seeder, `Volunteer.ApplicationDate` , `ProjectUpdate` / `ReliefProject` defaults, dashboard month start) vs `DateTime.UtcNow` (donations, edits)	`EmployeeController.cs:33` , `HomeController.cs:113,300` , `Models/*`	Use UTC consistently (Azure App Service runs in UTC)	Low	High
Wrong audit action: viewing donation details is logged as `CertificateViewed`	`DonorController.cs:91`	Use a distinct action name	Low	High
`AuditService.LogAsync` calls `SaveChanges` on the shared scoped context, which flushes any other pending changes as a side effect	`AuditService.cs:18`	Audit inside the caller's unit of work	Low	Medium
No global rate limiting or request-size policy beyond defaults	`Program.cs`	`AddRateLimiter` on Donate/Contact/Volunteer/Register/Login	Medium	High

18. API Inventory

The system is server-rendered. Its "API" is the MVC/Razor endpoints plus one webhook and one Function. All POSTs carry anti-forgery tokens except the webhook.

Method	Route	Purpose	Authn	Authz	Validation	Test evidence
GET	`/` , `/Home/About` , `/Home/ReliefProjects` , `/Home/Privacy`	Public pages	None	—	—	None
GET/POST	`/Home/Donate`	Donation form / capture	Optional	—	DataAnnotations (gaps above)	None
GET	`/Donate/Confirmation`	Receipt from TempData	None	—	—	1 (TempData only)
GET	`/Donate/Tax-Certificate/{id}`	Certificate HTML (marks issued)	Cookie	Donor (owner) or Employee	int id	None
GET	`/Donate/Tax-Certificate/{id}/download`	Certificate PDF	Cookie	Donor (owner) or Employee	int id	None
GET/POST	`/Home/Volunteer` , GET `/Volunteer/Confirmation`	Volunteer application	Optional	—	DataAnnotations	None
GET/POST	`/Home/Contact` , GET `/Contact/Confirmation`	Inquiry	None	—	DataAnnotations	None
GET	`/Donor/Dashboard` , `/Donor/MyDonations` , `/Donor/DonationDetails/{id}`	Donor portal	Cookie	Donor; owner filter	—	None
GET	`/Donor/TaxCertificate/{id}` , `/Donor/DownloadTaxCertificate/{id}`	Certificate	Cookie	**Donor only in practice** (§27)	—	None
GET	`/Employee/Dashboard` , `ReliefProjects` , `Volunteers` , `VolunteerDetails/{id}` , `Donations` , `Inquiries` , `AuditLog`	Staff views	Cookie	Employee	—	None
GET/POST	`/Employee/CreateReliefProject` , `EditReliefProject/{id}` , `CreateProjectUpdate` , `EditProjectUpdate/{id}`	Staff CRUD	Cookie	Employee	Input models	None
POST	`/Employee/ApproveVolunteer` , `RejectVolunteer` , `MarkInquiryHandled`	Staff actions	Cookie	Employee	id	None
POST	`/Payments/Notify`	PayFast ITN webhook	Signature + PayFast validate call	—	No amount/merchant check	1 (signature)
GET	`/health`	Liveness	None	—	—	None
GET/POST	`/Identity/Account/*`	Login, Register (custom), Manage, etc. (default UI)	—	—	Identity	None
GET	(dev only) `/ApplyDatabaseMigrations` via `UseMigrationsEndPoint`	EF migrations endpoint	None	Development only	—	—
GET	Function `/api/tax-certificate/{donationId}`	PDF	**Anonymous**	**None**	positive int	None

There is no API versioning, OpenAPI spec or JSON contract, which is appropriate for a server-rendered app.

19. API Contract Consistency

- There is no OpenAPI or typed client, so §19 is largely **not applicable**.
- Internal contract mismatch: `PayFast:ReturnUrl` → `/Donate/Confirmation`, which depends on TempData that will not exist after a cross-site return from PayFast. The page would render with empty values.
- `HomeControllerTests` injects `Amount` as `decimal`, while the controller writes a **string** (`HomeController.cs:149`). The test passes but does not reflect the real contract.

20. API Security

Finding	Severity	Evidence	Impact	Confidence	Action
Anonymous Function exposes any donor's certificate (name, amount, date, project) by **sequential int ID**	**Critical** (if deployed)	`GenerateTaxCertificateFunction.cs:18` `AuthorizationLevel.Anonymous`	Mass enumeration of donor PII	High (code); deployment status unknown	Require auth (Function key at minimum, preferably Entra ID / App Service auth), enforce ownership, use the GUID reference
ITN handler does not compare `amount_gross` to `donation.Amount`, does not check `merchant_id`, and does not restrict source IPs	Medium	`PaymentsController.cs:18-66`	Lower assurance (the PayFast validate call is the main guard)	High	Add amount and merchant checks per PayFast ITN guidance
With an empty `Passphrase` (the default in `appsettings.json:22`), the MD5 signature has no secret and anyone can compute it. The test proves this: it signs with no secret	Medium	`PayFastPaymentService.cs:63-75`; `PayFastPaymentServiceTests.cs:16-19`	Signature check adds no authenticity; reliance shifts fully to the validate call	High	Make the passphrase mandatory outside sandbox and fail startup if it is missing
`HttpClient` for PayFast validate has the default 100 s timeout and no retry policy	Low	`Program.cs:35`	Slow webhook handling	High	Set a timeout; use standard resilience handlers
Mass assignment	—	Input models used everywhere	Not found	High	Preserve
SQL injection	—	EF LINQ only; raw SQL is constant DDL in migrations	Not found	High	—
CORS	—	Not configured (same-origin default)	Fine	High	—

21. External Integrations

Integration	Config	Failure behaviour	Sandbox/prod	Risk
PayFast	`PayFast` section; sandbox flag	Validate call failure → `false` → 400. PayFast retries. No timeout tuning	Sandbox default; `ReturnUrl/NotifyUrl` hard-coded to `https://localhost:7052` (`appsettings.json:24-26`). Production values must be overridden or PayFast cannot reach the webhook	High (checkout not wired)
SendGrid	Registered only if `SendGrid:ApiKey` is set (`Program.cs:37-46`)	Exceptions propagate after the DB commit, so the user sees a 500 although data was saved	`FromAddress` is `noreply@example.org`	Medium
QuestPDF	Community licence set in code	In-process	—	Low. Confirm Community licence eligibility for the organisation's revenue
Google Fonts, jsDelivr	`_Layout.cshtml`	Visual degradation only	—	Low (privacy: third-party requests leak visitor IPs)
Azure Artifacts feed	`NuGet.config`	Unauthenticated restore warns NU1900. No package currently comes from it	—	Low

PART VI — DATABASE AND DATA

22. Database Architecture

Engine: SQL Server (LocalDB for dev, a SQL Server 2022 container, Azure SQL per README). **Schema** (`ApplicationDbContext.cs`, snapshot):

```
erDiagram
    AspNetUsers ||--o{ Donations : "DonorId (SetNull)"
    AspNetUsers ||--o{ Volunteers : "UserId (SetNull)"
    AspNetUsers ||--o{ ReliefProjects : "CreatedByUserId (Restrict)"
    AspNetUsers ||--o{ ProjectUpdates : "PostedByUserId (Restrict)"
    ReliefProjects ||--o{ Donations : "ReliefProjectId (Restrict, nullable)"
    ReliefProjects ||--o{ ProjectUpdates : "ReliefProjectId (Restrict)"
    ReliefProjects ||--o{ VolunteerAssignments : "Restrict"
    Volunteers ||--o{ VolunteerAssignments : "Cascade"
    AspNetUsers { string Id PK; string FullName; datetime DateRegistered }
    Donations { int Id PK; decimal Amount "18,2 CK>0"; string Currency "CK ZAR/USD/EUR"; string DonationType "CK"; int PaymentStatus; guid IdempotencyKey UK; string TransactionRefer
    ReliefProjects { int Id PK; string Status "CK"; decimal FundsRequired "CK≥0"; decimal FundsRaised; rowversion RowVersion; date StartDate; date EndDate "CK≥Start" }
    Volunteers { int Id PK; string Email; string Status "CK"; date DateOfBirth; string Address; string EmergencyContactPhone }
    VolunteerAssignments { int Id PK; int VolunteerId; int ReliefProjectId; UK "VolunteerId+ReliefProjectId" }
    ProjectUpdates { int Id PK; string Title; string Content; string ImageUrl }
    AuditLogs { int Id PK; string UserId; string Action; string TargetEntity; int TargetId; datetime Timestamp }
    Inquiries { int Id PK; string Email; string Message; bool Handled }
```

Diagram source (Mermaid) as recorded at the audit date

Strengths (preserve):

- Check constraints for enums, amounts and dates.
- Unique idempotency and transaction-reference indexes.
- Composite indexes for donor history and the volunteer queue.
- Deliberate delete behaviours that avoid multiple cascade paths.
- `rowversion` on `ReliefProject`.
- Decimal precision set.
- Enums stored as strings.

Weaknesses:

- `FundsRaised` is a denormalised aggregate with no reconciliation against donations.
- `AuditLogs.UserId` has no FK and is `nvarchar(max)` (no length limits on several string columns: `Description`, `Content`, `Skills`, `ImageUrl`).
- `PaymentStatus` is stored as int while other enums are strings (inconsistent).
- There is no soft delete and no retention metadata for PII.

23. Database ↔ Application Consistency

Issue	Evidence	Risk	Confidence
Orphan migration: `20260924190000_P0SecurityAndDonationIntegrity.cs` has no `[Migration]` / `[DbContext]` attribute and no Designer file, so EF never discovers it. Its contents duplicate `P1PaymentsEmailAndStatuses`	grep for `[Migration(` lists 6 Designer files only	Confusion; anyone who "fixes" it by adding attributes would create duplicate tables and indexes	High
`P0Phase0CriticalFixes` adds check constraints via raw SQL under **different names** (`CK_ReliefProjects_*`, `CK_Donations_Amount_Positive`) from the model's (`CK_ReliefProject_*`, `CK_Donation_Amount`). The DB carries **duplicate constraints EF does not know about**	`20260924195240_P0Phase0CriticalFixes.cs:25-40`; snapshot `:210,363,365`	Schema drift; future constraint changes will leave the hidden duplicates in force	High
`RelationalEventId.PendingModelChangesWarning` suppressed	`Program.cs:27`	Model/migration drift can ship silently	High
Empty migration `FixPendingModelChanges` (0 operations)	file	Noise, symptomatic of drift-chasing	High
Seed relies on roles being created; outside Development, roles are created lazily by Register only	`DbSeeder.cs:25-39`; `Register.cshtml.cs:150-153`	Works, but provisioning is undocumented	Medium

Recommended: run `dotnet ef migrations has-pending-model-changes` in CI (EF 8+) and remove the warning suppression.

24. Transactions, Concurrency and Integrity

- **Donate:** an explicit transaction plus `rowversion` gives correct lost-update protection on `FundsRaised`. Two concurrent donors to one project: one gets `DbUpdateConcurrencyException` and the "try again" message, which is **not shown** (§10). There is no automatic retry, so popular projects will reject legitimate donors under load.
- **Notify:** same rowversion; a concurrency failure gives a 500 and PayFast retries, which is acceptable. `PaymentStatus == Verified` short-circuit makes it idempotent for duplicates.

- **Integrity defect (Critical, High confidence):** `FundsRaised` reflects *submitted* donations and would reflect paid ones *twice*. An anonymous attacker can submit any number of donations up to ~10^{16} with no payment and push every project to "100% funded", corrupting the main public KPI.
- **Currency defect (High):** USD and EUR amounts are added to a ZAR-denominated `FundsRaised` 1:1 (`HomeController.cs:143`).
- **Recommended pattern:**
  - Don't pre-increment.
  - On verified ITN, increment by the converted ZAR amount inside the transaction, or replace the column with a computed or periodically reconciled `SUM` .
  - Add a reconciliation job or report.

25. File and Object Storage

Not applicable. No uploads. `ImageUrl` fields hold external URLs validated to http(s) and ≤500 chars (`AllowedImageUrlAttribute.cs`). These are rendered as `<img src>` (`Home/ReliefProjects.cshtml:37`), which allows mixed content over `http` and third-party tracking pixels. Low risk, because only employees set them, but employee is self-assignable (§27).

26. Data Protection and Privacy (South Africa, POPIA)

PII collected:

- Volunteers: full name, email, phone, address, city, province, postal code, **date of birth**, emergency contact name and phone.
- Donors: name (discarded), email, donation history.
- Inquiries: name, email, free text.
- Accounts: email, full name.

Gap	Evidence
No privacy notice (placeholder page)	`Privacy.cshtml`
No consent capture or purpose statement on the Volunteer, Contact or Donate forms	grep for consent/POPIA: no hits
PII exposed to any self-registered "Employee"	§27
No retention, deletion or export workflow. Identity's default "Delete personal data" removes the user, but volunteer, donation and inquiry rows keep the email and other fields (`SetNull` FK only)	`ApplicationDbContext.cs:70–79`
Seeded fake PII uses realistic South African names and phone numbers (dev only)	`DbSeeder.cs:163–208`
Third-party font/CDN requests on every page	`_Layout.cshtml:23–27`

Compliance with POPIA cannot be established from code. **Legal or policy review is required** before collecting real volunteer data.

PART VII — AUTHENTICATION AND SECURITY

27. Identity and Access Control

Area	Finding	Evidence	Severity	Confidence
Self-service privilege escalation	Any visitor can register as **Employee** and immediately read all volunteers' PII, all donations and inquiries, edit projects and approve volunteers. The README acknowledges this as a prototype simplification, but the site is publicly hosted	`Register.cshtml:31` ; `Register.cshtml.cs:114-116,131,150-154` ; `README.md:288`	**Critical**	High
Email confirmation off; `IEmailSender` is Identity's no-op default	`Program.cs:30`	Accounts with arbitrary or unowned emails	Medium	High
Password policy: Identity defaults (6+ chars with complexity); lockout uses defaults (enabled for new users in Identity defaults); no MFA enforcement for staff	`Program.cs:30` ; `README.md:291`	Weak for staff accounts holding PII	Medium	High
Demo credentials published in README and setup scripts, and README says they work **on the live site**	`README.md:15-21,296-301` ; `setup.*` ; `DbSeeder.cs:41-44`	Anyone gets Employee access to the live demo, and the seeder only runs in Development, so either the live site runs in Development or the README is wrong. **Unable to verify** which	High (if live site runs Development: developer exception page and migrations endpoint exposed)	Medium
Action-level role widening is ineffective. `DonorController` has class-level `[Authorize(Roles="Donor")]` ; ASP.NET Core requires **all** authorize attributes, so `[Authorize(Roles="Donor,Employee")]` on `TaxCertificate` / `DownloadTaxCertificate` does not admit Employee-only users	`DonorController.cs:12,96,121`	Employee-only users get 403 on these routes (they can still use the `/Donate/Tax-Certificate` routes)	Low	High
Ownership enforced server-side in donor queries and certificate routes	`DonorController.cs:84,111,132` ; `HomeController.cs:218,252`	Good, preserve	—	High
Cookies: Identity defaults (HttpOnly, SameSite=Lax, Secure under HTTPS); HSTS outside dev; HTTPS redirection on	`Program.cs:68-73`	Acceptable	—	High
Behind Azure App Service / containers: no `UseForwardedHeaders` , so `Request.Scheme` may be `http` behind TLS termination, which affects generated callback URLs	`Program.cs`	Low	Medium	

28. Application Security Audit

Finding	Severity	Evidence	Component	Impact	Confidence	Action
Self-registration as Employee	Critical	§27	Identity	Full staff access, PII breach	High (confirmed in code)	Remove role choice; admin-assigned roles
Anonymous certificate Function (IDOR)	Critical if deployed	§20	Functions	Donor PII enumeration	High (code) / Unknown (deployment)	AuthN + ownership; non-sequential IDs
Unauthenticated funding-total manipulation	Critical (integrity)	§24	Donate	Public figures falsified; donor trust	High	Count only verified payments; rate limit
Known-vulnerable transitive dependency `System.Security.Cryptography.Pkcs` 9.0.11 (via `Microsoft.Data.SqlClient` 6.1.6): NuGet audit NU1903 high (GHSA-846f-f764-76mx) and NU1902 moderate (GHSA-2fmv-rv2c-59x7)	High (per advisory rating)	Restore/build output; `GiftOfTheGivers/packages.lock.json`	Web, Tests, Functions	Exploitability in this app **not assessed**	High (advisory present)	Pin a patched `System.Security.Cryptography.Pkcs` or update SqlClient; regenerate lock files
Demo credentials public	High (on live)	README	Live site	Staff access	Medium	Remove from README for any public deployment; disable in prod
No rate limiting or CAPTCHA on public forms and login/register	Medium	`Program.cs`	Web	Spam, DB growth, inflation attack	High	`AddRateLimiter` ; CAPTCHA on public forms
No security headers (CSP, X-Content-Type-Options, frame-ancestors, Referrer-Policy)	Medium	`Program.cs`	Web	Clickjacking and XSS defence-in-depth	High	Add header middleware
`AllowedHosts: "*"`	Low	`appsettings.json:35`	Web	Host-header issues	High	Set real hostnames in prod
CDN assets without SRI	Low	`_Layout.cshtml:25,27`	Web	Supply chain	High	Self-host or add `integrity`
`local.settings.json` committed and `<Content Include>` d (currently no secrets)	Low	`Functions.csproj:13`	Functions	Future secret leakage	High	Ignore it; use the example file
XSS	Not found	Razor encoding; no `Html.Raw` ; image URLs scheme-validated	—	—	High	Preserve
CSRF	Mitigated	Global `AutoValidateAntiforgeryToken` ; webhook exempt by design	—	—	High	Preserve

29. Threat Model (STRIDE, prioritised)

- **Assets:** volunteer and donor PII, donation records, public funding figures, tax certificates, staff accounts, PayFast merchant credentials.
- **Trust boundaries:** browser ↔ web app; PayFast ↔ `/Payments/Notify` ; internet ↔ Function; app ↔ SQL; GitHub ↔ Azure DevOps (PAT).

Threat	STRIDE	Realistic scenario	Current mitigation	Gap
Become staff	Elevation	Register with "Employee"	None	Critical
Harvest certificates	Info disclosure	Loop `/api/tax-certificate/1..N`	None	Critical
Inflate funding	Tampering	Script anonymous POSTs to `/Home/Donate` with large amounts	Anti-forgery (a token is easy to obtain by GET)	Critical
Forge payment notification	Spoofing	POST forged ITN	PayFast validate round-trip	Medium (no amount/merchant check; passphrase optional)
Form spam / DB bloat	DoS	Bulk contact/volunteer submissions	None	Medium
Repudiation of staff actions	Repudiation	Employee edits project or rejects volunteer	No audit of mutations	Medium
CI secret abuse	Elevation	`AZURE_DEVOPS_PAT`, `AZURE_ARTIFACTS_PAT` in Actions	Secrets; `permissions: contents: read` on two workflows	Low/Medium (CI and CD lack explicit `permissions`)

PART VIII — BUSINESS FLOWS

30. Cross-Platform Feature Completeness

Feature	Web UI	Mobile	API/Backend	Database	Authorization	Tests	Docs	Status
Browse projects	Yes	N/A	Yes	Yes	Public	None	README	Implemented, verification incomplete
Donate	Yes (form gaps)	N/A	Partial	Yes	Public	None	README	**Partial**
Payment	No redirect	N/A	Webhook only	Status column	Signature	1	README note	**Broken (disconnected)**
Recurring donations	Radio button	N/A	No	Label	—	None	README overclaims	**Missing**
Volunteer apply	Yes	N/A	Yes	Yes	Public	None	README	Implemented, verification incomplete
Volunteer review	Yes	N/A	Yes	Yes	Employee (escalatable)	None	README	Implemented, verification incomplete
Volunteer assignment	No	N/A	No	Yes	—	None	README	Partial (schema only)
Contact / inquiries	Form yes; list unlinked	N/A	Yes	Yes	Employee	None	README (form)	Partial
Donor dashboard / history	Yes	N/A	Yes	Yes	Owner-scoped	None	README	Implemented, verification incomplete
Tax certificate	Yes	N/A	Yes + Function	Flag	Owner/Employee; **Function open**	2 (PDF bytes only)	README	Partial / insecure
Project CRUD + updates	Yes	N/A	Yes	Yes	Employee	None	README	Implemented, verification incomplete
Audit log	Unlinked page	N/A	Certificate events only	Yes	Employee	None	None	Partial
Email notifications	—	N/A	SendGrid/NoOp	—	—	None	README (keys)	Implemented, unverified
Registration / login	Yes	N/A	Identity	Yes	—	None	README	Implemented, **insecure role choice**

31. End-to-End Workflows

A. Anonymous donation (critical)

Step	Detail
1. Start	Anonymous visitor on `/Home/Donate?projectId=3`
2. Action	Fills in the form and submits
3. Screen	`Views/Home/Donate.cshtml`
4. Route	POST `/Home/Donate`
5. Logic	Idempotency lookup → transaction → insert Pending → `FundsRaised += Amount` → commit (`HomeController.cs:84–170`)
6. DB	INSERT Donations; UPDATE ReliefProjects (rowversion)
7. External	**None.** PayFast is never invoked
8. Expected	Redirect to PayFast; funds counted only after a verified ITN
9. Feedback	Receipt page says "successful", with a certificate link for logged-in users
10. Failure paths	Invalid currency, type or project → 500; concurrency message invisible; huge amount → 500
11. Tests	None for POST
12. Missing	Checkout redirect, payment return handling, payment status in UI

B. PayFast ITN. Exists (`PaymentsController.cs`) but is unreachable in normal flow. If reached, it double-counts. No integration test.

C. Registration → Employee portal. Register (role radio) → `AddToRoleAsync(user, "Employee")` → sign in → `/Employee/*` . **Complete and working, which is the problem.**

D. Volunteer application → approval. Form → `Volunteers` row (Pending) → employee Approve (POST + anti-forgery) → status saved → email (SendGrid or no-op). No assignment step. No tests.

E. Certificate download. Donor → `/Donor/DownloadTaxCertificate/{id}` → ownership check → audit → PDF. Pending donations are eligible. Employee-only users are blocked on `/Donor/*` (§27) but allowed on `/Donate/Tax-Certificate/{id}/download` . The Function route bypasses everything.

F. Contact. Form → `Inquiries` row → acknowledgement email → confirmation. Employee list exists at `/Employee/Inquiries` with no nav link.

PART IX — QUALITY ENGINEERING

32. Automated Testing Audit

Executed: 19/19 passed (Release, local, 2026-11-02).

File	What it tests	Quality
`HelperCalculationsTests.cs`	Percentage, total/average, certificate number	Good unit tests
`ModelTests.cs`	Extension method; duplicates the helper tests	Redundant
`P1ValidationTests.cs`	`javascript:` URL rejected; enum defaults; PDF magic bytes	OK; the PDF test duplicates `TaxCertificatePdfTests`
`TaxCertificatePdfTests.cs`	PDF magic bytes	Smoke only; donor name and anonymity untested
`PayFastPaymentServiceTests.cs`	Field order preserved in validate POST	Useful; no passphrase, invalid-signature or `INVALID` -response cases
`HomeControllerTests.cs`	`Confirmation` reads TempData	Uses `decimal` where production writes `string` ; the controller is built with `null!` dependencies
`DatabaseContextTests.cs`	InMemory round-trip	Checks EF itself, not app logic; InMemory ignores check constraints, rowversion and FKs

Missing:

- Donate POST (idempotency, funding, invalid project, concurrency)
- Notify (verified, failed, duplicate, amount mismatch)
- Authorization (role and ownership on every controller)
- Registration role restriction
- Employee CRUD
- Migrations applying cleanly to SQL Server
- Any UI, end-to-end or accessibility test
- The Function

No coverage report is produced; the `coverlet.collector` package is referenced but no CI run collects coverage. **Coverage percentage: unable to verify.**

33. Recommended Test Strategy (risk-ordered)

1. **Integration tests with `WebApplicationFactory` against SQL Server.** Use Testcontainers or LocalDB, not InMemory, so constraints and rowversion are real. Cover:
   - Donate happy and failure paths.
   - Funding is unchanged until ITN.
   - ITN idempotency and the amount check.
   - Concurrent donations.
2. **Authorization matrix tests.** Every `/Employee/*` and `/Donor/*` route as anonymous, Donor, Employee and other-donor. Register cannot create Employee.
3. **Validation boundary tests.** Currency, type, amount max, oversize strings: expect 200 with errors, not 500.
4. **Migration test.** `migrations has-pending-model-changes` plus applying all migrations to an empty SQL container in CI.
5. **Function test.** Unauthenticated requests get 401.
6. **Playwright smoke and axe accessibility** on Donate, Volunteer, Contact and Register at mobile and desktop widths.
7. **Contract test for PayFast signature generation** with a passphrase, against PayFast's documented example.

34. Code Quality and Maintainability

- **Positive:** consistent naming; nullable reference types enabled; `.editorconfig` / `.gitattributes`; small files; primary constructors in newer code.
- **Issues:**
  - Mixed constructor styles (classic vs primary).
  - Mixed tabs and spaces in tests (`ModelTests.cs` vs `HelperCalculationsTests.cs`).
  - Misindented block in `HomeController.Contact` (`:329-344`).
  - Fully qualified `System.Security.Claims.ClaimTypes` repeated across controllers.
  - Duplicate certificate actions in two controllers.
  - `P1ValidationTests` has no namespace.
  - Scaffold boilerplate XML docs left in `Register.cshtml.cs` .
  - No analyzers or warnings-as-errors.

35. Dependency and Supply-Chain Audit

Item	Finding	Classification
`System.Security.Cryptography.Pkcs` 9.0.11 (transitive via `Microsoft.Data.SqlClient` 6.1.6)	NU1903 high + NU1902 moderate advisories reported during restore	**Confirmed advisory**
Lock files	Present for all four projects (`RestorePackagesWithLockFile`)	Good, but CI does not use `--locked-mode` , so drift is not enforced
`NuGet.Protocol` 7.9.0 in the web app	No code references (grep)	**Unused dependency**; adds transitive surface
`Microsoft.VisualStudio.Web.CodeGeneration.Design` , `Microsoft.EntityFrameworkCore.Tools`	Design-time packages without `PrivateAssets="all"` (Tools) shipped in the runtime graph	Cleanup
`Microsoft.VisualStudio.Azure.Containers.Tools.Targets`	VS tooling	Fine (build-time)
Package sources	`nuget.org` + private Azure Artifacts feed, `<clear/>` used	Good; no source mapping, which is a dependency-confusion consideration since both feeds are queried. Add `<packageSourceMapping>`
SDK	Pinned 10.0.401, `rollForward: latestFeature` . The machine has 10.0.403; GitHub CI uses `10.0.x`	Acceptable
`xunit` 2.9.3	xUnit v2 line (v3 exists)	Outdated-version note only

PART X — PERFORMANCE, RELIABILITY AND SCALE

36. Performance

- Unbounded list queries: `EmployeeController.Volunteers/Donations/Inquiries` , `HomeController.ReliefProjects` . `EmployeeController.ReliefProjects` eagerly loads all `ProjectUpdates` (`:69-72`) just for counts.

- The employee dashboard runs six sequential queries; the donor dashboard runs four. Fine at this scale.
- PDF generation is synchronous and CPU-bound per request (acceptable).
- App Service F1 cold starts (per README) plus `MigrateAsync` on every start lengthen cold start.
- **No measurements were taken.**

37. Scalability and Concurrency

- **Single hot row:** every donation to a project updates the same `ReliefProjects` row with a rowversion check, so contention grows with traffic and losers receive an error (§24).
- Multiple app instances would all run `MigrateAsync` concurrently at startup, which is unsafe for scale-out.
- TempData is cookie-based by default, so it works across instances.
- Data Protection keys are not configured for persistence. In containers or scale-out, auth cookies and anti-forgery tokens break on restart or across instances. **Medium, High confidence** (no `AddDataProtection` in `Program.cs`).

38. Reliability and Resilience

Scenario	Behaviour
DB unavailable at startup	`MigrateAsync` throws and the app fails to start (no retry or `EnableRetryOnFailure`)
Transient Azure SQL errors	No `EnableRetryOnFailure` configured (`Program.cs:26`). User-initiated transactions would need an execution strategy
SendGrid down	Exception after commit → 500 with data saved → user resubmits → duplicate inquiries
PayFast validate slow	Webhook can hang up to 100 s
Health check	`/health` is liveness only; no DB check

39. Logging, Monitoring and Observability

- Serilog with a console sink only. `Enrich.FromLogContext`, but no request logging (`UseSerilogRequestLogging` absent) and no correlation ID.
- No Application Insights, metrics, tracing or alerting.
- The audit log covers certificate views only.
- **Blind spots:** failed ITNs (warning log only), funding anomalies, privilege changes, staff edits.

PART XI — DEVOPS, CI/CD AND CLOUD

40. Build and Development Workflow

- **Setup:** `setup.ps1` / `setup.sh` → restore, tool restore, `dotnet ef database update`. Docker Compose provides SQL Server + web.
- **Reproducibility:** good (pinned SDK, local tool, lock files). Verified locally: restore, build and test succeed.
- **Friction:**
  - `DATABASE_SETUP.md` instructs editing `appsettings.Development.json` and a **global** `dotnet-ef` install, contradicting README (user-secrets, local tool).
  - `ApplicationDbContextFactory` hard-codes LocalDB, so `dotnet ef` commands ignore user-secrets and env vars. Mac/Linux developers following `setup.sh` with Docker SQL will see migrations target LocalDB and fail. **High confidence.**
- No lint/format step (`dotnet format --verify-no-changes` absent).

41. CI/CD Pipeline Audit

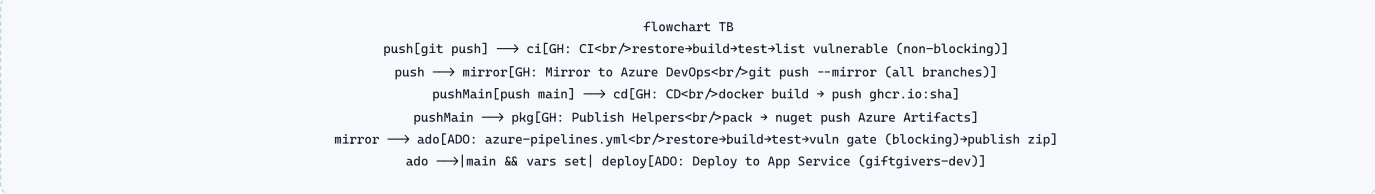


Diagram source (Mermaid) as recorded at the audit date

Finding	Evidence	Severity	Confidence
CD pushes a container image on every `main` push **independently of CI**, so failed tests do not block it	`cd.yml` (no `needs`, separate workflow)	Medium	High
CD/CI declare no `permissions:`. Pushing to GHCR needs `packages: write`; whether it works depends on the repo's default token setting	`cd.yml`, `ci.yml`	Medium	Medium (history shows fix commits `64d343b`, `299ed13`; success unverified)
GitHub CI vulnerability check is non-blocking; ADO's is blocking. With the current Pkcs advisory the **ADO pipeline will fail at "Check vulnerable packages"** (if `dotnet list` reports it, which matches the restore warnings)	`ci.yml:24-25`; `azure-pipelines.yml:79-88`	High	Medium-High (not executed)
Publish-helpers: `dotnet nuget add source --name "giftgivers-helpers"` while `NuGet.config` already defines that name. `add source` fails on a duplicate name, so this step likely errors; `dotnet nuget update source` is the usual fix	`publish-helpers-package.yml:67-73`; `NuGet.config:6`	Medium	Medium-High (not executed; history shows repeated workflow fixes)
`${{ inputs.package_version }}` interpolated directly into a bash script (script-injection pattern; limited to users who can dispatch workflows)	`publish-helpers-package.yml:42-43`	Low	High
`--store-password-in-clear-text` writes the PAT into the runner's NuGet config (ephemeral runner)	`:73`	Low	High
Mirror runs `push --mirror` for every push, including deletions; ADO branch protection is recommended in CONTRIBUTING but not enforced	`mirror-to-azure-devops.yml`	Low	High
No migration step, smoke test, slot swap or rollback in ADO deploy; schema changes happen on app start	`azure-pipelines.yml:111-132`; `DbSeeder.cs:23`	Medium	High
CI does not restore with `--locked-mode`, run `dotnet format`, collect coverage or scan secrets	`ci.yml`	Low/Medium	High
Duplicate CI systems (GitHub and ADO) run the same build/test	both files	Low (cost and confusion)	High
Image tagged only by SHA; no `latest`/semver; nothing deploys the image	`cd.yml:28`	Low	High

Pipeline success: unable to verify. No run logs were available.

42. Git and Collaboration

- Branch naming is documented. PRs are used (#5–#11 in history).
- There are no PR or issue templates, no CODEOWNERS and no tags/releases.
- Commit messages are often uninformative ("good", "donate page" ×2, "improvements").
- Feature branches merge back into themselves, and Copilot agent commits are interleaved.
- `.gitignore` is comprehensive (VS template plus `.env`, `appsettings.Development.json`) but misses `local.settings.json` and the scratch `.txt` files.
- `CONTRIBUTING.md` defers branch protection "until the pipeline is green".

43. Infrastructure and Deployment

- **Dockerfile:** multi-stage, non-root (`USER $APP_UID`), installs `libfontconfig1` for QuestPDF (good). `EXPOSE 8081` is unused.
- **docker-compose (dev only):**
  - Exposes SQL Server on host `1433` with the `sa` login and `Encrypt=False`.
  - Runs the app in **Development** (developer exception page and migrations endpoint).
  - Uses `mssql/server:2022-latest` (floating tag).
- **No IaC.** The App Service, Azure SQL, Function App, firewall rules and identities are all configured by hand, so the configuration is **unable to verify**.
- No Data Protection key persistence, no forwarded-headers config, and `AllowedHosts: *`.

44. Cloud Architecture and Cost

- **Configured in repo:** none (no IaC).
- **Documented:** App Service F1 (Free), Azure SQL, Azure DevOps, Azure Artifacts, GHCR.
- F1 has no Always On, no slots and a CPU quota, so it is unsuitable beyond a demo.
- No backups, alerts or budget configuration in evidence.
- Costs cannot be estimated without the SQL tier and Function plan (missing inputs).

45. Environments, Secrets and Configuration

- **Secrets handling is good in principle:** user-secrets documented; `.env` git-ignored; no real secrets in history (verified with redacted scans).
- **Risks:**
  - `PayFast:NotifyUrl/ReturnUrl/CancelUrl` default to `https://localhost:7052`. A production deployment that forgets to override them silently breaks payment callbacks.
  - `Sandbox: true` is the default. Production needs the flag flipped, plus real credentials and a passphrase, with no startup validation.
  - Missing `SendGrid:ApiKey` silently falls back to the NoOp sender (no log warning).
  - `ApplicationDbContextFactory` ignores configuration.
- **Recommendation:** `ValidateOnStart()` options validation that rejects localhost URLs, sandbox mode and an empty passphrase outside Development.

46. Release, Rollback and Migration Safety

- No versioning of the web app, no release notes and no tags.
- Migrations run at app start, so a rollback of code does not roll back schema.
- Migrations include destructive operations (`DropForeignKey` / `DropCheckConstraint` /column changes in `SyncCurrentModel` and `P1 …`).
- No backward-compatibility policy. As a server-rendered app with no external clients, old-client risk is limited to the PayFast webhook contract.
- **Needed before releases:**
  - Generate an idempotent migration script in CI (`dotnet ef migrations script --idempotent`) and apply it as a gated step.
  - Use deployment slots (requires a paid tier).
  - Record a release tag per deploy.

47. Backup and Disaster Recovery

There is no backup configuration, restore procedure, RPO/RTO or DR documentation in the repo. Azure SQL has built-in point-in-time restore, but the tier and retention are **unable to verify**. Restoration has never been documented or tested.

PART XII — DOCUMENTATION AND DEVELOPER EXPERIENCE

48. README Audit — Rating: **Adequate**

Strengths: clear purpose, badges, setup Options A/B, user-secrets guidance, Docker instructions, role and route tables, and an honest prototype disclaimer.

Problem	Evidence
Overclaims: "Recurring donation management", "Donation oversight and reporting", "Clean, accessible forms", "Success/confirmation pages for all submissions" (project edits redirect with TempData only)	README §Features/§UI vs code
Says demo login works on the live site, but the seeder only runs in Development	`README.md:15–22` vs `DbSeeder.cs:25`
Project structure omits Functions, Helpers, Tests and the CI/CD files	`README.md:247–256`
CI/CD instructions (Azure mirror, Artifacts, Pipelines) placed under "Getting Started" before Prerequisites, which confuses newcomers	`README.md:82–132`
No architecture, testing, troubleshooting, security-reporting or known-limitations sections	—
Payments (PayFast) configuration not documented beyond one user-secret line	`README.md:209`
`DATABASE_SETUP.md` contradicts README; `GIT_SETUP_GUIDE.md` is obsolete	§40

49. Documentation Audit

Document	Status
Architecture / ERD	Missing (the code references a "Section B ERD" that is not in the repo)
API / webhook docs	Missing
Database docs	Minimal, contradictory
Deployment / runbook	Partial (CI/CD setup only)
Testing guide	Missing
Security policy (`SECURITY.md`)	Missing
Release procedure	Missing
User documentation	Missing

A new developer would need to reverse-engineer:

- the payment design intent,
- the role provisioning model,
- which environment the live site runs in,
- the purpose of the Function and how it is deployed,
- the migration history (orphan and raw-SQL constraints).

50. Developer Experience

Can a new developer…	Answer
Understand the purpose	Yes
Install toolchain and dependencies	Yes (pinned SDK, local tool; verified)
Configure local environments	Yes on Windows; **partially** on Mac/Linux (design-time factory forces LocalDB)
Run the app / services	Yes (dotnet run or Compose; not executed in this audit)
Create and migrate the database	Windows yes; Mac/Linux via CLI no (see above), via app startup yes
Run meaningful tests	Tests run; low value
Build web	Yes
Debug failures	Limited (console logs only)
Understand API contracts	Partly (no docs)
Contribute safely	Partly (no PR template, no branch protection)
Package a release	No documented process

PART XIII — LONG-TERM HEALTH

51. Ongoing Development Readiness

Can another developer add a substantial feature without first rewriting large parts of the application? Yes, with caveats.

- The codebase is ~2,000 lines of first-party C# plus views, uses conventional ASP.NET Core patterns, and builds and tests cleanly. Input models, EF configuration and service abstractions give clear extension points.
- **Before** adding donation- or payment-adjacent features, a developer must fix the funding logic (§24) and add integration tests. Otherwise new features will inherit wrong totals.
- **Before** adding staff features, role provisioning must be fixed.
- The schema-drift suppression must be removed to keep migrations trustworthy.

52. Technical Debt Register

Item	Category	Class	Fix before features?
Self-assignable Employee role	Security	Critical	**Yes**
Anonymous certificate Function	Security	Critical	**Yes**
FundsRaised pre-increment / double count / mixed currency	Data integrity	Critical	**Yes**
Payment checkout not wired	Functionality	High	Yes (before any real donations)
Vulnerable Pkcs transitive	Security / dependency	High	Yes
No privacy notice or consent	Privacy / UX	High	Yes (before real data)
No integration or authorization tests	Testing	High	Yes
Pending-model-changes warning suppressed; orphan migration; duplicate raw-SQL constraints	Database	Medium	Yes (cheap)
Validation gaps → 500s; validation summary missing	UX / backend	Medium	Soon
No rate limiting, security headers or Data Protection persistence	Security / ops	Medium	Soon
Startup migrations in prod; no deploy migration step	Deployment	Medium	Before scale-out
CD not gated on CI; workflow permission gaps; helpers publish step likely broken	CI/CD	Medium	Soon
Design-time factory hard-coded	DX	Medium	Soon
Functions→Web project reference	Architecture	Medium	Before Function work
Mutations not audited	Security / compliance	Medium	Roadmap
No pagination	Performance	Low	Roadmap
Fake stats and organisation details; Privacy placeholder	UX / content	Medium	Before public launch
Stale docs and scratch files	Documentation	Low	Roadmap
Mixed `DateTime.Now` / `UtcNow`	Correctness	Low	Roadmap
Unused `NuGet.Protocol` ; design packages in runtime	Dependency	Low	Roadmap

53. Demo, Placeholder and Dead-Code Audit

Finding	Location	Classification
Seeded demo accounts and passwords	`DbSeeder.cs:41–44` , README, setup scripts	Intentional dev content → **production risk** if Development runs live
Seeded projects, volunteers, donations	`DbSeeder.cs`	Harmless dev content (Development only)
"This is a prototype. No actual payment will be processed."	`Donate.cshtml`	Intentional disclaimer (accurate)
"Portfolio of Evidence Part 1 prototype — symbolic values, not a valid tax document."	`TaxCertificatePdf.cs:107`	Intentional; must change before real certificates
"Tax Registration: 123-456-789", fake phone and address	`Donor/TaxCertificate.cshtml:84` , `_Layout.cshtml` footer	Placeholder → production risk
Hard-coded impact statistics	`Index.cshtml` , `About.cshtml`	Placeholder
Privacy template text	`Privacy.cshtml`	Unfinished
"PayPal" option	`Donate.cshtml:114`	Unfinished / misleading
`BuildCheckoutRedirectUrl` unused	`PayFastPaymentService.cs:16`	Unfinished implementation
Orphan migration; empty migration	`Data/Migrations`	Dead code
`NuGet.Protocol` reference	csproj	Dead dependency
`allfiles.txt` , `helpers.txt` , `testlog.txt` , `GIT_SETUP_GUIDE.md`	repo root	Temporary artifacts
Unused VM fields (`FirstName` / `LastName` , `DonorName`) and `ViewBag.DonorName/DonorEmail`	VMs, `HomeController.cs:186–187`	Dead code
TODO/FIXME/lorem	grep: none	—

Finding	Location	Classification
Test doubles (`RecordingHttpMessageHandler` , `ReceiptTempDataProvider`)	Tests	Legitimate test infrastructure

54. Governance and Delivery Risks

- No roadmap, backlog, acceptance criteria, risk owners or versioning in the repo.
- Phase labels in migration names (P0, P1) suggest a prioritised fix list existed, but it is not committed.
- The academic context (Part 1, Section B) implies fixed deadlines and possible rubric-driven scope. **The rubric was not supplied**, so alignment cannot be assessed.
- A single active maintainer, plus AI-agent commits, raises bus-factor and review-quality risk.

PART XIV — FINAL REPORT AND ACTION PLAN

55. Feature and Engineering Completeness Matrix

Area	Implemented	Tested	Securely implemented	Documented	Production ready	Main gap
Web application	Mostly	No	Partly	Partly	No	Validation UX, placeholders
Mobile application	N/A	N/A	N/A	N/A	N/A	Not in scope
Backend	Mostly	Minimal	Partly	No	No	Funding logic, validation → 500
HTTP endpoints / webhook	Yes	Minimal	Partly	No	No	Function auth, ITN checks
Database	Yes	No (InMemory only)	Yes	Minimal	Partly	Drift suppression, duplicate constraints
Authentication	Yes	No	Partly	Yes	No	No email confirmation, no staff MFA
Authorization	Yes	No	**No**	Yes	No	Self-assigned Employee role
Integrations (PayFast, SendGrid)	Partial	1 test	Partly	Minimal	No	Checkout unwired
File storage	N/A	N/A	N/A	N/A	N/A	—
Automated testing	Minimal	—	—	No	No	No integration or auth tests
CI/CD	Yes (4 + ADO)	—	Partly	Yes	Partly	CD ungated, failing gates likely
Infrastructure	Docker only	No	Dev-grade	Partly	No	No IaC, no DP keys
Monitoring	Console logs	No	—	No	No	No APM or alerts
Backup and recovery	No	No	—	No	No	Nothing documented
Security	Partial	No	No	Partial	No	§28
Accessibility	Partial	No	—	No	Unknown	No testing
Privacy / POPIA	No	No	No	No	No	No notice or consent
README	Yes	—	—	—	—	Overclaims, outdated parts
Developer documentation	Partial	—	—	—	—	Architecture, testing, runbooks

56. Risk Register

ID	Risk	Severity	Likelihood	Impact	Evidence	Mitigation	Priority
R1	Public user self-registers as Employee → PII breach, data tampering	Critical	Very high (UI offers it)	Severe	`Register.cshtml:31` , `.cs:131–154`	Remove role choice; admin assignment	P0
R2	Anonymous Function leaks all certificates	Critical	High if deployed	Severe	`GenerateTaxCertificateFunction.cs:18`	Auth + ownership; do not deploy until fixed	P0
R3	Funding totals inflated by unpaid / fake donations	Critical	High	Public figures wrong; trust and reputation	`HomeController.cs:143`	Count only verified payments	P0

ID	Risk	Severity	Likelihood	Impact	Evidence	Mitigation	Priority
R4	Double count on ITN; no reversal on failure	High	Certain once payments are wired	Wrong totals	`PaymentsController.cs:52,61`	Single increment path	P0 (with R3)
R5	Mixed currencies summed	High	High	Wrong totals	`HomeController.cs:143` ; `Employee/Donations.cshtml`	Normalise currency	P1
R6	Payment never collected	High	Certain	No revenue; misleading receipts	grep `BuildCheckoutRedirectUrl`	Wire checkout + return page	P1
R7	Certificates issued for unpaid donations	High	High	Fraudulent tax documents (if made real)	`TaxCertificate` actions; no status check	Require `Verified`	P1
R8	Vulnerable Pkcs dependency	High	Unknown exploitability	Security	Restore output	Pin patched version	P1
R9	No privacy notice or consent for sensitive PII	High	Certain	Regulatory (POPIA)	`Privacy.cshtml` ; Volunteer form	Notice, consent, retention	P1
R10	Live site may run in Development with public staff credentials	High	Unknown	Full compromise of demo data	README vs `DbSeeder.cs:25`	Verify env; rotate or remove demo accounts	P0 (verify)
R11	Schema drift hidden	Medium	Medium	Failed or partial deploys	`Program.cs:27` ; migrations	Remove suppression; CI check	P1
R12	Data Protection keys not persisted	Medium	High in containers / scale-out	Logouts, anti-forgery failures	`Program.cs`	Persist keys (Blob/DB)	P2
R13	Startup migrations on scale-out	Medium	Medium	Locking / failed starts	`DbSeeder.cs:23`	Pipeline migration step	P2
R14	CD publishes untested images	Medium	Medium	Bad artifacts	`cd.yml`	Gate on CI	P2
R15	Form spam / abuse (no rate limit)	Medium	High (public)	DB bloat, inflation	`Program.cs`	Rate limiter / CAPTCHA	P1
R16	Validation gaps produce 500s and invisible errors	Medium	High	Lost donations, poor UX	§10, §17	Allowed-values validation + summary	P1
R17	No backups / DR documented	Medium	Low	Data loss	—	Document PITR, test restore	P2
R18	Misleading content (fake stats, tax reg no., contact details) under a real NGO's name	Medium	Certain on public site	Reputational / legal	Views	Replace or label as demo	P1

57. Prioritised Engineering Roadmap

Effort figures are rough and assume one developer familiar with ASP.NET Core. The biggest uncertainty is the PayFast production onboarding.

P0 — Critical blockers

P0-1 Remove self-service Employee role.

- **Problem:** R1.
- **Outcome:** registration always creates Donor; Employee is granted by an existing Employee/Admin or a seeded admin from config.
- **Files:** `Register.cshtml(.cs)` , `DbSeeder.cs` , new `Admin` or `EmployeeController.ManageUsers` .
- **Applies to:** web.
- **DB:** none (roles exist).
- **Dependencies and risks:** need a bootstrap admin path (config-seeded in all environments). Existing self-registered Employee accounts must be reviewed and demoted.
- **Effort:** 0.5–1 day.
- **Acceptance:** POST with `Input.Role=Employee` yields a Donor account; only admins can grant Employee.
- **Tests:** integration test for register; authorization matrix.
- **Docs:** README §How registration works.

P0-2 Secure or disable the certificate Function.

- **Problem:** R2.
- **Outcome:** no anonymous access. Use `AuthorizationLevel.Function` as a minimum and App Service authentication with an ownership check as the target, and look up by `TransactionReference` GUID.
- **Files:** `GenerateTaxCertificateFunction.cs` , `local.settings*` .

- **Applies to:** Functions.
- **Dependencies:** confirm whether it is deployed (Azure portal); if it is, take it offline first.
- **Effort:** 0.5 day to lock down; 1–2 days for proper user auth.
- **Acceptance:** unauthenticated request → 401; another user's donation → 403/404.
- **Tests:** function unit test with a fake request.
- **Docs:** README Functions section (new).

P0-3 Correct funding integrity.

- **Problem:** R3, R4.
- **Outcome:** `FundsRaised` changes only when a donation becomes `Verified`, exactly once. Remove the increment in `Donate`. Add a reconciliation query (or admin "recalculate").
- **Files:** `HomeController.cs:132–145`, `PaymentsController.cs`, `ReliefProject.cs` comment.
- **DB:** a one-off data fix that recalculates `FundsRaised` from verified donations. This **changes displayed figures** and needs stakeholder sign-off.
- **Dependencies:** decide the currency rule (P1-2).
- **Risks:** seed data shows inflated numbers by design.
- **Effort:** 1 day + tests.
- **Acceptance:** Pending donation → total unchanged; COMPLETE ITN → +amount once; duplicate ITN → no change; FAILED → no change.
- **Tests:** integration tests on SQL Server.
- **Docs:** ADR-1.

P0-4 Verify live environment and demo credentials.

- **Problem:** R10.
- **Outcome:** `ASPNETCORE_ENVIRONMENT=Production` on App Service; demo accounts removed or passwords rotated on any public instance; README stops publishing staff credentials for the live site.
- **Applies to:** infrastructure and docs.
- **Effort:** 1–2 hours.
- **Acceptance:** `/ApplyDatabaseMigrations` is unavailable and the developer exception page is not shown on the live site.

P1 — High-priority stability

ID	Task	Files	Effort	Acceptance / tests
P1-1	Patch `System.Security.Cryptography.Pkcs` (explicit PackageReference to a patched version, or SqlClient update); regenerate lock files; remove unused `NuGet.Protocol`	csproj ×3, lock files	2–4 h	`dotnet list package --vulnerable --include-transitive` clean; ADO gate passes
P1-2	Currency handling: store ZAR-equivalent (rate + source + timestamp) or per-currency totals; fix `ToString("C")` to explicit currency formatting	`Donation`, migration, views	1–2 d	Mixed-currency test totals correct; display shows the correct symbol
P1-3	Wire PayFast checkout: Donate → `BuildCheckoutRedirectUrl` → return page that loads by `m_payment_id` (not TempData) and shows status; ITN amount and merchant checks; mandatory passphrase and production URL validation (`ValidateOnStart`)	`HomeController`, `PaymentsController`, `PayFastPaymentService`, options	2–4 d (+ PayFast sandbox testing)	Sandbox end-to-end: Pending → Verified; amount mismatch rejected
P1-4	Certificates only for `Verified` donations; real certificate numbering; remove fake registration number	Donor/Home controllers, `TaxCertificatePdf`, view	0.5–1 d	Pending donation → no certificate link, 404 on route
P1-5	Donate validation: allowed values for Currency/Type/Method, realistic max amount, active-project check before insert, catch unique violation for idempotency, validation summary in all forms, `asp-for` on name/email, preserve radio/select state	`DonateViewModel`, `Donate.cshtml`, `HomeController`	1 d	Invalid inputs → 200 with messages (no 500s)
P1-6	Privacy: real privacy notice, consent checkbox and purpose text on Volunteer/Contact/Donate, DOB handling fix, retention policy	Views, VMs, `Volunteer`	1–2 d + legal review	Consent stored with timestamp; Privacy page has real content
P1-7	Integration test harness (WebApplicationFactory + SQL container) plus authorization matrix tests	Tests	2–3 d	CI runs them; covers P0-1..3
P1-8	Schema hygiene: delete the orphan migration, remove `PendingModelChangesWarning` suppression, add a migration dropping the duplicate `CK_ReliefProjects_*` / `CK_Donations_Amount_Positive` constraints, add `has-pending-model-changes` to CI	`Program.cs`, Migrations, `ci.yml`	0.5–1 d	CI fails on drift; DB has one set of constraints
P1-9	Rate limiting on public POSTs and Identity login/register; replace or label placeholder content (stats, contact details)	`Program.cs`, views	0.5–1 d	Load test shows 429 beyond threshold

P2 — Engineering quality

ID	Task	Effort
P2-1	Gate CD on CI (`workflow_run` or merge the jobs); add `permissions:` blocks; fix `nuget add source` → `update source` ; use `--locked-mode` in CI	0.5 d
P2-2	Migrations as a deploy step (idempotent script) instead of startup in Production; `EnableRetryOnFailure`	1 d
P2-3	Persist Data Protection keys; `UseForwardedHeaders` ; security headers; `AllowedHosts`	0.5–1 d
P2-4	Audit all staff mutations (project create/edit, update edits, volunteer decisions, inquiry handled) with actor and before/after	1 d
P2-5	Extract a `GiftOfTheGivers.Data` project (DbContext, models, PDF); point Functions at it; move donation logic into a `DonationService`	1–2 d
P2-6	Design-time factory reads configuration (user-secrets/env)	1 h
P2-7	Docs: reconcile `DATABASE_SETUP.md` , delete `GIT_SETUP_GUIDE.md` and scratch files, add architecture/testing/runbook/SECURITY.md, correct README feature claims	1 d
P2-8	Link Inquiries and Audit Log in employee navigation; fix certificate back-links; pagination on staff lists	0.5–1 d
P2-9	Request logging + Application Insights (or OpenTelemetry); DB health check	0.5–1 d
P2-10	Accessibility pass (fieldsets, error announcements, contrast) + axe in CI	1–2 d

P3 — Strategic enhancements

- Real recurring donations (PayFast subscriptions).
- Volunteer-to-project assignment UI.
- Reporting and export.
- Staff MFA.
- Email confirmation.
- IaC (Bicep) for App Service, SQL and Functions.
- Paid tier with deployment slots.
- Backup and restore runbook with a tested restore.
- Consolidate on one CI system.

Ordering rationale. P0-1/2/4 are independent and quick. P0-3 depends on the P1-2 currency decision for the data fix, but the code change (stop pre-incrementing) can ship first. P1-7 tests should land alongside P0-3 and P1-3, not after them. Refactoring (P2-5) waits until P1-7 tests pin the behaviour.

58. What Should Be Preserved

- **EF model configuration** (`ApplicationDbContext.cs`): constraints, indexes, delete behaviours, rowversion. This is the strongest part of the codebase.
- **Input models** for every staff form (no over-posting) and the `AllowedImageUrl` validator.
- **Server-side ownership checks** in donor queries and certificate routes.
- **Global anti-forgery filter** and Razor encoding discipline (no `Html.Raw`).
- `IPaymentGateway` / `IEmailService` **abstractions** with a no-op fallback, and the PayFast verification implementation (field order, fixed-time compare, server validate).
- **Idempotency token design** on donations: keep it, and harden the race.
- **Reproducible toolchain:** `global.json` , local `dotnet-ef` , lock files, setup scripts, and Docker Compose with a SQL health check.
- **Non-root multi-stage Dockerfile.**
- **QuestPDF certificate layout**, once fed only verified data.
- The intro's **reduced-motion and skip** handling.

59. Professional Scorecard

Scoring:

- 0–2: absent or dangerous.
- 3–4: present with serious defects.
- 5–6: workable prototype quality.
- 7–8: solid.
- 9–10: exemplary.

Weighting (sums to 100):

- 10 each: Security, Authorization, Business logic.
- 5 each: Product completeness, Architecture, Backend, Database, Automated testing, Production readiness, Web frontend, Maintainability.
- 3 each: Authentication, CI/CD, Documentation, Reliability, Dependency management.
- 2 each: UI/UX, Accessibility, Performance, Infrastructure, Monitoring, Backup, README, Developer experience, Scalability, API design.

Mobile is excluded (not applicable, not scored as zero). Critical security and integrity findings cap the overall result at **≤ 4.5** regardless of the weighted sum.

Area	Score	Evidence confidence	Main reason
Product completeness	5	Medium	Core CRUD works; payments, recurring and reporting are missing
Architecture	6	High	Clean monolith; Functions→Web coupling; logic in controllers
Web frontend	5	Medium (no browser)	Consistent UI; validation display gaps; placeholders
Mobile application	N/A	High	Not present
Backend	4	High	Funding defect, 500s on invalid input
API design (routes/webhook)	5	High	Reasonable routes; duplicate certificate routes; ITN checks incomplete
Database	7	High	Strong constraints; drift suppression and duplicates
Authentication	5	High	Identity defaults; no confirmation or MFA
Authorization	2	High	Self-assigned staff role; anonymous Function
Security	3	High	Two critical issues, vulnerable dependency, no headers or rate limits
Business logic	3	High	Funding, currency and certificate eligibility wrong
UI/UX	5	Low-Medium	Clean Bootstrap; misleading copy
Accessibility	5	Low	Basics present; untested
Performance	6	Medium	Fine at current scale; no pagination
Reliability	4	Medium	No retries, health check shallow, post-commit email failures
Automated testing	2	High	19 shallow tests; no critical-path coverage
Dependency management	6	High	Lock files and pinning; known advisory unresolved
CI/CD	5	Medium	Many pipelines; ungated CD; likely-failing steps
Infrastructure	4	Medium	Docker good; no IaC, DP keys, forwarded headers
Monitoring and logging	3	High	Console only
Backup and recovery	1	Medium	Nothing documented
Documentation	4	High	README decent; others stale or contradictory
README	6	High	Detailed; overclaims
Developer experience	6	High	Scripts work on Windows; Mac/Linux migration friction
Maintainability	6	High	Small, readable, consistent
Scalability	4	Medium	Hot row, startup migrations, DP keys
Production readiness	2	High	Multiple P0 blockers

Weighted overall: ≈ 3.9 / 10 (below the 4.5 cap).

Separate measure	Estimate	Uncertainty
Feature completion	~65%	No specification; judged against the README
Evidence-backed testing confidence	Low	19 passing unit tests; none on critical paths
Documentation completeness	~55%	Subjective; key ops and security docs absent
Maintainability	Good for its size (6/10)	Will degrade without tests as features grow
Production readiness	~20%	Live environment configuration unverified

60. Final Professional Verdict

What the project actually is

A well-structured **functional prototype** of an NGO management site: a clean ASP.NET Core MVC monolith with a thoughtfully designed database. It is presented publicly, but its money, privilege and privacy flows are prototype-grade.

What is working well

- EF schema design with real integrity constraints.
- Input-model discipline.
- Server-side ownership checks on donor data.
- Global CSRF protection.
- A reproducible toolchain (pinned SDK, lock files, setup scripts, Compose).
- A correct PayFast signature-verification component.
- A codebase that builds without errors and passes its tests.

What is incomplete

- Payment checkout and return.
- Recurring donations.
- Emailed certificates.
- Volunteer-project assignment UI.
- Reporting and export.
- Audit coverage.
- Privacy notice and consent.
- Pagination.
- Monitoring, backup and IaC.
- Meaningful tests.

What is dangerous

1. Anyone can become staff.
2. Anyone can download any donor's certificate via the Function, if it is deployed.
3. Anyone can inflate public funding totals without paying, and real payments would be double-counted in mixed currencies.
4. Certificates labelled "Section 18A" are issued for unpaid donations.
5. Sensitive volunteer PII is collected with no privacy notice or consent.
6. A known high-severity transitive dependency advisory.
7. Possibly a live site in Development mode with published staff credentials (unverified).

What prevents production release

R1–R4, R6–R10 in §56. Specifically: role provisioning, Function authentication, funding integrity, payment wiring, certificate eligibility, the dependency patch, privacy compliance, and live-environment configuration.

What prevents efficient ongoing development

- No integration or authorization tests to protect refactors.
- Suppressed migration-drift detection and a messy migration history.
- A design-time DbContext factory that ignores configuration (Mac/Linux).
- Contradictory setup docs.
- CI/CD that may be failing or ungated.
- Domain logic embedded in controllers and duplicated across them.

What should be fixed first (smallest set, most risk reduced)

1. Force Donor role at registration and add an admin-granted Employee path (P0-1).
2. Lock down or undeploy the Function (P0-2).
3. Delete the `FundsRaised` increment in `Donate`, and recalculate totals from verified donations (P0-3).
4. Confirm the live environment is Production and remove published staff credentials (P0-4).
5. Patch the Pkcs dependency (P1-1).

Together these are roughly 2–3 developer-days.

What should be built next

1. The SQL-backed integration test harness (P1-7).
2. Then the PayFast checkout and return flow with amount verification (P1-3) and currency normalisation (P1-2).
3. Then the privacy notice and consent (P1-6).

Those are the prerequisites for accepting a single real donation or volunteer record.

What should not be rewritten

The EF model and migrations (clean them, don't regenerate them), the input models, the PayFast service, the PDF generator, the Identity integration (only the role step changes), the layout and views, and the Docker/tooling setup.

Recommended engineering strategy

Strengthen security and data integrity first, then improve testing and CI/CD, then continue incremental development. No architectural redesign is justified. A modest extraction of a shared data project and a donation service is enough once tests exist.

Final maturity classification: **Functional but incomplete** (functional prototype)

- **Why:** every user-facing area renders and persists data, and the build and tests are green. But the payment flow is disconnected, authorization can be bypassed by design, and financial figures are not trustworthy.
- **Remaining uncertainty:**
  - Live-site environment and configuration.
  - Whether the Function is deployed.
  - Actual CI/CD run results.
  - Runtime UI and accessibility behaviour (no browser testing).
  - The unsupplied academic requirements, which may intentionally scope some of these gaps as later "parts".

Gift of the Givers · Relief Management System

DELIVERABLE 08-A

Audit Addendum — Remediation Review

AUD-2026-11-01-R1 — what was changed after the baseline audit, how each fix was verified, what was newly discovered, the re-scored scorecard, and the audit programme for future reviews.

DOCUMENT	VERSION	DATE	STATUS
D-08-A	1.0	2026-11-02	Remediation review — code in commit 921e5d8

CONTENTS

1. Summary
2. What was changed
3. Verification evidence
4. Findings status
5. New findings discovered during remediation
6. Re-scored scorecard
7. Remaining work before production
8. Audit vision — how future audits work

1. Summary

The baseline audit **AUD-2026-11-01** (document 08) classified the system as a *functional prototype* (normalised score 4.07/10). It found three Critical defects: anyone could make themselves staff, an anonymous function exposed donor certificates, and anyone could inflate funding totals without paying.

This remediation review covers the changes made on 2026-11-02 on branch `feature/erd-schema-and-pdf`. The application changes were committed and pushed as `921e5d8` **"audit fixes"**. Final documentation and diagram polish was made after that commit.

> **Scope note.** An Android project (`app/` , Gradle/Kotlin) was staged in the working tree while this review was in progress. It is **not covered** by this review. Section 13–16 of the baseline (mobile) still reflects the repository at `efa2b02` . The next audit must include the mobile app.

0	98	67.9%	0
CRITICAL FINDINGS OPEN (WAS 3)	AUTOMATED TESTS (WAS 19)	LINE COVERAGE, HAND-WRITTEN CODE	VULNERABLE PACKAGES (WAS 1, THEN 2)

Measure	Baseline	Now
Weighted score (normalised, weights sum 105)	4.07	**7.03** (reported **6.5**, capped while High finding F-23 is open)
Maturity	Functional prototype	**Beta** — functional, tested and secured; not yet a production candidate
Findings tracked	39 at baseline	46 (7 new found during remediation)
Status	—	32 fixed · 7 partially fixed · 7 open

Correction to the baseline. The baseline scorecard said its weights "sum to 100", but they sum to **105**. Its reported overall score of about 3.9 should have been **4.07**. The register stores the corrected value. The baseline document itself is kept unchanged as a frozen record.

2. What was changed

2.1 Security and access

- **Registration** (`Areas/Identity/Pages/Account/Register.cshtml(.cs)`): the role picker was removed and every self-registration becomes a Donor.
- **Staff Access** page (`EmployeeController.StaffAccess / GrantEmployee / RevokeEmployee` , `Views/Employee/StaffAccess.cshtml`): employees grant or revoke staff access by email. Self-revoke is blocked, the security stamp is rotated, and every change is audited.
- **Bootstrap staff** through `StaffAccess:BootstrapEmployeeEmails` (`DbSeeder`). Roles are created in every environment; demo accounts are created only in Development or with `Seed:DemoData` .
- **Azure Function** (`GiftOfTheGivers.Functions`): function key, lookup by transaction reference, verified donations only, issuer from configuration.
- **Per-action roles** in `DonorController` : employees can open certificates; donors still own their data.
- **Hardening** (`Program.cs` , `Infrastructure/SecurityHeaders.cs`):
  - per-IP POST rate limiting, with the webhook exempt
  - CSP, `X-Frame-Options` , `nosniff` , Referrer-Policy and Permissions-Policy
  - SRI on the CDN stylesheet
  - Data Protection keys stored in the database
  - opt-in forwarded headers
  - password length 8, lockout settings, unique email
  - `IdentityEmailSender` for Identity emails

- **Dependencies:**
  - `System.Security.Cryptography.Pkcs` pinned to 10.0.14.
  - Microsoft packages aligned on 10.0.14, which also resolves `Cryptography.Xml`.
  - Unused `NuGet.Protocol` and the scaffolding package removed.
  - `NuGet.config` has package source mapping and audit sources.

2.2 Money integrity and payments

- `Services/DonationService.cs` (new) is the only place donations are captured and payments applied:
  - pending capture with no change to totals
  - idempotency with race handling
  - active-project check
  - rand conversion
  - ITN application with merchant and amount checks, replay safety, a single increment, and a rowversion retry
  - `RecalculateFundsRaisedAsync`
- **Currency** (`Services/CurrencyConverter.cs`, `Currency:RatesToZar`): `AmountZar` and `ExchangeRateToZar` are stored per donation, and PayFast is charged in rand.
- **PayFast** (`PayFastPaymentService`, `HomeController.Donate`): checkout is wired, with per-donation return, cancel and notify URLs, `email_address`, and a 15-second timeout. `PayFastOptionsValidator` refuses unsafe live settings at startup.
- **Receipts** by unguessable reference, with live status (`Views/Home/DonationSuccess.cshtml`).
- **Certificates** only for verified donations (`DonorController`). The issuer comes from `Organisation` config, with a demonstration watermark (`TaxCertificatePdf`). Legacy routes redirect.
- **Offline funds and Funding Reconciliation** (`ReliefProject.OfflineFundsZar`, `Views/Employee/FundingReconciliation.cshtml`): recalculate, or accept the difference as offline funds, both audited.

2.3 Data and schema

- **Migration** `20261102112731_AuditRemediationNov2026`**:**
  - new columns, constraints and an index
  - historical backfill
  - normalisation of `PaymentMethod`
  - removal of the duplicate raw-SQL constraints
  - a `Down()` method that re-creates them
- **Historical migration fixed:** `P1PaymentsEmailAndStatuses` no longer alters a rowversion column (F-21).
- **Orphan migration deleted;** design-time factory deleted; `PendingModelChangesWarning` suppression removed; Identity key length aligned (F-22).
- **Volunteer** `DateOfBirth` and `PhoneNumber` are now optional and never invented. Consent timestamps were added to donations, volunteers and inquiries. Audit columns are sized and have `Details`.

2.4 Staff and public experience

- **Validation:**
  - `AllowedValues` / `Range` rules on the donation form
  - validation summaries and field messages on every form
  - radio and select state preserved after errors
  - fieldset and legend on the donation-type radios
  - a live region for the rand equivalent
- **Content:**
  - live impact statistics (`ViewComponents/ImpactStatsViewComponent.cs`)
  - organisation details from configuration
  - fabricated statistics, phone numbers, office hours and the tax number removed
  - illustrative stories labelled as such
  - a real privacy notice
  - consent on every form
  - honest "monthly pledge" copy
  - a demonstration badge in the footer
- **Staff portal:**
  - attention cards on the dashboard
  - paged and filtered Donations (with CSV export, formula-injection safe), Volunteers, Inquiries and Audit Log
  - volunteer assignments
  - offline funds on project forms
  - complete navigation
  - `data-confirm` instead of inline JavaScript
- **Money formatting:** a culture-independent `Money` helper replaces every `ToString("C")`.
- **Accent button contrast** fixed: it measured 2.71:1 and now measures 5.77:1.

2.5 Tests, CI/CD and documentation

- **Tests:** 98 in total. Integration tests run through the real pipeline (`WebApplicationFactory`, test auth, fake gateway) plus unit tests. The obsolete TempData test was replaced.
- **CI:**
  - locked restore, coverage, a blocking vulnerability gate and an EF drift check

    - a SQL Server 2022 migrations job (empty-database apply, rollback and re-apply, idempotent script)
- **CD:** runs only after CI succeeds.
- **Publish workflow** fixed: `update source`, inputs passed through env, SemVer check.
- **Azure Pipelines:** locked restore, drift check, and a migration script artifact.
- **Docker Compose:** SQL Server port bound to localhost only.
- **Docs:**
    - README, DATABASE_SETUP and CONTRIBUTING rewritten
    - `SECURITY.md` and a PR template added
    - deliverables 01–08 produced as HTML and PDF from Markdown sources (`docs/tools/build_deliverables.py`)
    - audit programme in `docs/audits/`
    - stale files removed: `GIT_SETUP_GUIDE.md` , scratch `.txt` files, and the committed `local.settings.json`

3. Verification evidence

Check	Command / method	Result
Build	`dotnet build GiftOfTheGivers.slnx -c Release --no-incremental`	Succeeded, **0 warnings, 0 errors**
Tests	`dotnet test GiftOfTheGivers.slnx -c Release`	**98 passed**, 0 failed
Coverage	`--collect "XPlat Code Coverage"` , excluding migrations and compiled views	Lines 67.9% (2394/3528), branches 52.5%; `DonationService` , `PaymentsController` , `DonorController` , `CurrencyConverter` , `AuditService` 100%
Vulnerabilities	`dotnet list GiftOfTheGivers.slnx package --vulnerable --include-transitive`	No vulnerable packages in all 4 projects
Restore	`dotnet restore --force-evaluate`	Clean, no NU19xx warnings (private feed no longer queried)
Model drift	`dotnet ef migrations has-pending-model-changes`	No changes
Migrations on SQL Server	Throwaway LocalDB database: old chain → legacy rows (ZAR; USD with PayPal; EUR) → upgrade → `Down` → re-apply	All steps succeeded; rand backfill R 100 / R 185 / R 100; PayPal → Card; 3 duplicate constraints removed; 11 check constraints remain
App start on migrated DB	Development, LocalDB	Started; seeding OK; 0 errors in the log
Public smoke test	`curl` on 12 public routes	All 200; `/health` returned `Healthy` (DB check)
Headers	`curl -I /Home/Privacy`	CSP, X-Frame-Options DENY, nosniff, Referrer-Policy and Permissions-Policy present
Donation over HTTP	Real form post (USD 10)	302 to `sandbox.payfast.co.za` with `amount=185.00` ; stored as Pending (R 185); receipt shows *Awaiting payment*; project total unchanged
Staff smoke test	Cookie login as the demo employee; 14 staff routes	All 200; Donor dashboard correctly denied (302 to Access Denied); reconciliation flagged the legacy R 715 inflation; CSV export correct
PayFast sandbox	Signed requests to the public sandbox (GET/POST, with and without the documented passphrase)	**Rejected:** "signature does not match". Unsigned request accepted. Our signature string matches PayFast's PHP algorithm byte for byte → see F-23

Not verified in this review:

- GitHub and Azure DevOps pipeline runs (the definitions changed; first runs pending)
- the live Azure environment and its database
- an end-to-end PayFast payment (F-23)
- browser accessibility tooling
- load testing

4. Findings status

Generated from `docs/audits/findings-register.json` . Findings are sorted by severity, then ID.

ID	Finding	Severity	Baseline ref	Status	Evidence / verification	Remaining action
F-01	Anyone can self-register as Employee (staff)	Critical	R1	Fixed	Register always assigns Donor; Employee is granted from Staff Access (audited) or StaffAccess:BootstrapEmployeeEmails. Tests: Registration_AlwaysCreatesDonor_EvenWhenEmployeeRoleIsPosted, RegisterPage_NoLongerOffersAnEmployeeOption, StaffAccess_GrantIsAuditedAndSelfRevokeIsRefused.	Review existing live accounts that se registered as Employee and revoke needed.
F-02	Anonymous Azure Function returns any donor certificate by sequential id	Critical	R2	Fixed	AuthorizationLevel.Function, lookup by 32-character TransactionReference, Verified donations only (GenerateTaxCertificateFunction.cs). Builds in CI.	Confirm in Azure whether an old anonymous version is deployed; rec or delete it.
F-03	Project totals inflated by unpaid or fake donations	Critical	R3	Fixed	DonationService.CaptureAsync never changes FundsRaised; Funding Reconciliation detects and corrects legacy inflation. Tests: Donate_RecordsPendingDonation_RedirectsToPayFast_AndDoesNotTouchProjectTotal, FundingReconciliation_RecalculateUsesOfflinePlusVerifiedOnly. LocalDB smoke test flagged a legacy R 715 difference.	Run Funding Reconciliation on the l database after deployment.

ID	Finding	Severity	Baseline ref	Status	Evidence / verification	Remaining action
F-04	Payment confirmation double-counts and failures never reverse	High	R4	Fixed	Single increment on Verified, replay-safe, Failed never touches totals, rowversion retry. Tests: Notify_Complete_VerifiesOnceAndAddsRandAmountOnce_EvenIfReplayed, Notify_Failed_MarksDonationFailedWithoutChangingTotals.	—
F-05	USD/EUR amounts summed 1:1 with rand	High	R5	Fixed	AmountZar and ExchangeRateToZar stored at capture; all totals in rand; migration backfills history (verified on LocalDB). Test: Donate_ForeignCurrency_IsChargedInRandAtConfiguredRate.	Rates are indicative configuration; s 46.
F-06	Donation flow never sends the donor to PayFast	High	R6	Partially fixed	Checkout wired with return/cancel/notify URLs; smoke test on LocalDB produced a correctly signed sandbox redirect for the rand amount (USD 10 → amount=185.00).	End-to-end payment not yet verifie blocked by F-23.
F-07	Tax certificates issued for unpaid donations; fake registration number	High	R7	Fixed	Certificates only for Verified donations; issuer from Organisation configuration; demonstration watermark until a PBO reference is configured. Tests: Certificates_OnlyForOwnersOfVerifiedDonations, Issuer_IsOfficialOnlyWhenNotDemoAndPboConfigured.	Configure real PBO details and set Organisation:IsDemo=false only wh legally entitled.
F-08	Known-vulnerable System.Security.Cryptography.Pkcs 9.0.11	High	R8	Fixed	Pinned 10.0.14 (first patched 10.0.13). dotnet list package --vulnerable --include-transitive reports none for all four projects.	—
F-09	No privacy notice or consent for volunteer and donor personal data (POPIA)	High	R9	Partially fixed	Privacy notice written; consent checkbox and timestamp on Donate, Volunteer and Contact. Tests: Volunteer_WithoutConsent_IsNotSaved, Contact_RecordsConsentTimestamp, PrivacyPage_IsARealNoticeNotTheTemplate.	Legal review; appoint an Informatio Officer; automate retention and del
F-10	Live site may run in Development with published staff credentials	High	R10	Partially fixed	Demo accounts are seeded only in Development or with Seed:DemoData=true; README no longer advertises a live staff login.	Owner must confirm ASPNETCORE_ENVIRONMENT=Proc on App Service and remove or rotat demo accounts there (cannot be ve from the repository).
F-21	Historical migration fails on every empty SQL Server database (rowversion ALTER)	High	New	Fixed	Found while verifying migrations on LocalDB. The AlterColumn on ReliefProjects.RowVersion in P1PaymentsEmailAndStatuses was removed (SQL Server cannot alter a rowversion column). Full chain now applies to an empty database; CI migrations job guards it.	—
F-22	Identity key-length drift (AddDefaultIdentity 128 vs schema 450) crashed startup once the warning was unsuppressed	High	New	Fixed	Stores.MaxLengthForKeys = 0 keeps the runtime model equal to the existing schema; app verified starting on the migrated LocalDB database.	—
F-23	Shared PayFast sandbox merchant rejects every signed request; payments unverified end to end	High	New	Open	Signature string matches PayFast's documented PHP algorithm byte for byte; the sandbox answers 'signature does not match' for every variant (GET/POST, with/without the documented passphrase) but accepts unsigned requests, so the shared merchant's passphrase is unknown.	Register an own PayFast sandbox merchant, set PayFast:Passphrase, ru sandbox payment with a public Not and record the evidence.
F-24	Transitive System.Security.Cryptography.Xml advisories (three High)	High	New	Fixed	Surfaced when Data Protection storage was added; Microsoft packages aligned on 10.0.14; audit clean.	—
F-39	No automated tests on critical paths	High	§32	Fixed	98 tests (19 at baseline) including integration tests through the real pipeline; 67.9% line / 52.5% branch coverage of hand-written code; DonationService and PaymentsController 100%.	Add Playwright and axe UI tests.
F-11	Model/migration drift hidden by suppressed PendingModelChangesWarning	Medium	R11	Fixed	Suppression removed; design-time factory removed; duplicate raw-SQL constraints dropped; orphan migration deleted; CI drift check. has-pending-model-changes reports none.	—
F-12	Data Protection keys not persisted	Medium	R12	Fixed	PersistKeysToDbContext with a DataProtectionKeys table.	—
F-13	Migrations run at startup on every instance	Medium	R13	Partially fixed	Database:MigrateOnStartup switch; idempotent migrate.sql produced by CI and Azure Pipelines.	Set Database:MigrateOnStartup=fal production and add the script step release.
F-14	CD publishes images without CI passing	Medium	R14	Fixed	cd.yml runs on workflow_run of a successful CI run on main, with explicit permissions.	—
F-15	No rate limiting on public forms and login	Medium	R15	Fixed	Per-IP fixed-window limiter on POST requests, webhook exempt. Test: PublicPosts_AreLimitedPerClient_ButTheWebhookIsNot.	Consider CAPTCHA if abuse is obse
F-16	Invalid input causes 500s and errors are not displayed	Medium	R16	Fixed	AllowedValues/Range validation, validation summaries on all forms, project checked before insert. Test: Donate_InvalidInput_RedisplaysFormInsteadOfServerError (5 cases).	—
F-17	No backups or disaster recovery procedure	Medium	R17	Open	Procedure documented in deliverable 07; no restore has been tested.	Confirm the Azure SQL retention tie perform and record a test restore.
F-18	Fabricated statistics and organisation details on a public site	Medium	R18	Fixed	Live ImpactStats view component; Organisation configuration; demonstration badge; illustrative stories labelled. Test: HomePage_ShowsLiveFiguresInsteadOfHardCodedStatistics.	—
F-19	Missing security headers, Subresource Integrity and host filtering	Medium	§28	Partially fixed	CSP, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy; SRI on the CDN stylesheet. Test: Responses_CarrySecurityHeaders.	CSP still allows 'unsafe-inline'; set AllowedHosts per environment.

ID	Finding	Severity	Baseline ref	Status	Evidence / verification	Remaining action
F-20	PayFast ITN lacks amount/merchant checks; passphrase optional	Medium	§20	Fixed	Amount and merchant validated; live mode refuses to start without a passphrase or with the sandbox merchant. Tests: Notify_AmountMismatch_IsRefusedAndDonationStaysPending, Notify_WrongMerchant_IsRefused, PayFastOptionsValidatorTests.	—
F-25	Private Azure Artifacts feed breaks clean restore; dependency-confusion exposure	Medium	New	Fixed	NU1301 401 reproduced on restore of a new package; packageSourceMapping (only GiftOfTheGivers.Helpers from the private feed) and auditSources = nuget.org added.	—
F-28	Staff changes not audited; wrong audit action names	Medium	§17	Fixed	Audit entries with details for projects, updates, volunteer decisions and assignments, inquiries, staff access, exports, reconciliation and payment events.	—
F-31	Culture-dependent money formatting	Medium	§10	Fixed	Money helper; no ToString("C") left in views. Tests: MoneyTests.	—
F-32	Volunteer date of birth fabricated when not supplied	Medium	§17	Fixed	DateOfBirth and PhoneNumber nullable and never invented. Test: Volunteer_WithoutDateOfBirth_StoresNullInsteadOfInventingOne.	—
F-36	Login lockout-on-failure not enabled by the default Identity UI; no staff MFA	Medium	§27	Open	Lockout options configured and POST rate limiting mitigates brute force, but the default Login page does not count failures.	Scaffold the Login page with lockoutOnFailure: true; require MFA the Employee role.
F-37	Recurring donations are only a label	Medium	§5	Partially fixed	Relabelled 'Monthly pledge' with honest copy on the form, receipt and README.	Implement PayFast subscriptions wi payment child donations.
F-38	Monitoring limited to console logs	Medium	§39	Partially fixed	Serilog request logging; /health (database) and /health/live probes.	Application Insights or OpenTeleme alerts on failed ITNs and health.
F-46	Exchange rates are manual configuration	Medium	New	Open	Indicative USD 18.50 and EUR 20.00 in appsettings; also used by the historical backfill.	Integrate a rates provider or a staff-maintained rate table with effective
F-26	Developer database contains migration 20260924223317_SnapshotSync that is not in the repository	Low	New	Open	Observed read-only in __EFMigrationsHistory of the local GiftOfTheGiversDb.	Confirm whether any shared or live database contains it; recreate local databases from repository migratio
F-27	Class-level Donor role blocked employees from certificate actions	Low	§27	Fixed	Roles declared per action. Test: DonorPortal_ForbidsEmployeeOnlyUsersButCertificatesAllowThem.	—
F-29	Azure Function references the whole web project	Low	§7	Open	Deferred: low risk once the function is key-protected.	Extract a shared data project (implementation plan T-2.3).
F-30	Unbounded staff lists (no pagination)	Low	§36	Fixed	Server-side paging for donations, volunteers, inquiries and the audit log.	—
F-33	Design-time DbContext factory hard-coded to LocalDB	Low	§40	Fixed	Factory removed; dotnet ef builds the model from the app host and its configuration.	—
F-34	Helpers publish workflow fails on duplicate source; script-injection pattern	Low	§41	Fixed	dotnet nuget update source; inputs passed via env; SemVer validation; tests before pack.	—
F-35	Committed local.settings.json, scratch files, stale and contradictory docs	Low	§6	Fixed	local.settings.json untracked and ignored; scratch files and GIT_SETUP_GUIDE removed; README, DATABASE_SETUP and CONTRIBUTING rewritten; SECURITY.md and a PR template added.	—
F-40	Certificate GET request changed state	Low	§17	Fixed	TaxCertificateIssued is set when the payment is verified, not when a page is viewed.	—
F-41	Inquiries and Audit Log unreachable from the UI; broken certificate back-links	Low	§10	Fixed	Employee menu and dashboard links; controller-qualified back-links.	—
F-42	Azure Function has no tests or deployment pipeline	Low	§41	Open	The function builds in CI only.	Add a deployment job and tests (implementation plan T-2.3).
F-43	Mixed local and UTC timestamps	Low	§17	Fixed	UTC used for all new writes and model defaults; staff views label UTC.	—
F-44	No forwarded-headers handling behind proxies	Low	§27	Fixed	Opt-in ForwardedHeaders:Enabled configuration.	Enable ForwardedHeaders:Enabled Linux App Service or containers.
F-45	Email failure after commit returned a 500 (invites duplicate submissions)	Low	§38	Fixed	Email delivery errors are logged, not thrown.	—

5. New findings discovered during remediation

ID	How it was found	Why it matters
F-21	Applying the repository's migrations to an empty SQL Server database for verification	Every fresh environment (new developer, Docker, CI, disaster recovery) would crash at startup on `MigrateAsync`. The baseline audit did not apply migrations, so it missed this
F-22	Removing the suppressed `PendingModelChangesWarning` made the app refuse to start	`AddDefaultIdentity` changes Identity key lengths at runtime, so design-time and runtime models disagreed. Earlier migrations had flip-flopped these columns, and the suppression hid it
F-23	Following the PayFast redirect against the real sandbox	The checkout cannot be proven end to end with shared credentials
F-24	NuGet audit after adding Data Protection storage	Fixing one advisory can introduce another; the CI gate now catches this
F-25	`restore` failed with 401 once a new package was needed	Clean CI agents and new developers could not restore
F-26	Read-only check of the local dev database before risky EF commands	A migration exists in a database but not in source control: evidence of past drift
F-46	Designing currency conversion	Correct foreign totals depend on maintained rates

6. Re-scored scorecard

Same areas, scale and weights as the baseline (weights sum to 105, normalised). Before uses the baseline score. After reflects the evidence above.

Area	Weight	Before	After	Reason for change
Security	10	3	7	Criticals fixed, headers, rate limit, clean dependencies; CSP inline and no MFA remain
Authorization	10	2	8	Staff access controlled and audited; role matrix tested
Business logic	10	3	8	Money rules enforced in one service with tests; manual FX rates remain
Product completeness	5	5	6.5	Payments wired, reconciliation, assignments, export; recurring billing absent
Architecture	5	6	7	Service layer for donations; function coupling remains
Backend	5	4	7.5	Validation, idempotency, concurrency retries
Database	5	7	8.5	Chain applies to an empty DB, no drift, backfills, constraints
Automated testing	5	2	7	98 tests, 67.9% coverage; no UI tests yet
Production readiness	5	2	5	Payment E2E, live environment, backups and MFA still open
Web frontend	5	5	7	Validation UX, honest states, paging
Maintainability	5	6	7.5	Smaller controllers, shared partials, documented standards
Authentication	3	5	6	Password and lockout options; Login lockout and MFA pending
CI/CD	3	5	8	Real gates and a migrations job; runs not yet observed
Documentation	3	4	8	Full deliverable set, setup docs fixed
Reliability	3	4	6	Retry strategy, DP keys, health checks, email failures contained
Dependency management	3	6	8.5	Source mapping, audit sources, aligned versions
UI/UX	2	5	6.5	Honest content and status
Accessibility	2	5	6	Fieldsets, live regions, contrast fix; untested with tools
Performance	2	6	6.5	Paging; not load tested
Infrastructure	2	4	5	Compose hardened; no IaC
Monitoring and logging	2	3	4.5	Request logs and DB health; no APM or alerts
Backup and recovery	2	1	2	Procedure documented; never tested
README	2	6	8	Accurate and complete
Developer experience	2	6	8	Migrations work everywhere; restore clean
Scalability	2	4	6	Shared keys, configurable startup migration
API design	2	5	7	Webhook contract validated; receipts by reference
Weighted overall	**105**	**4.07**	**7.03**	Reported **6.5** (cap: an open High finding)

7. Remaining work before production

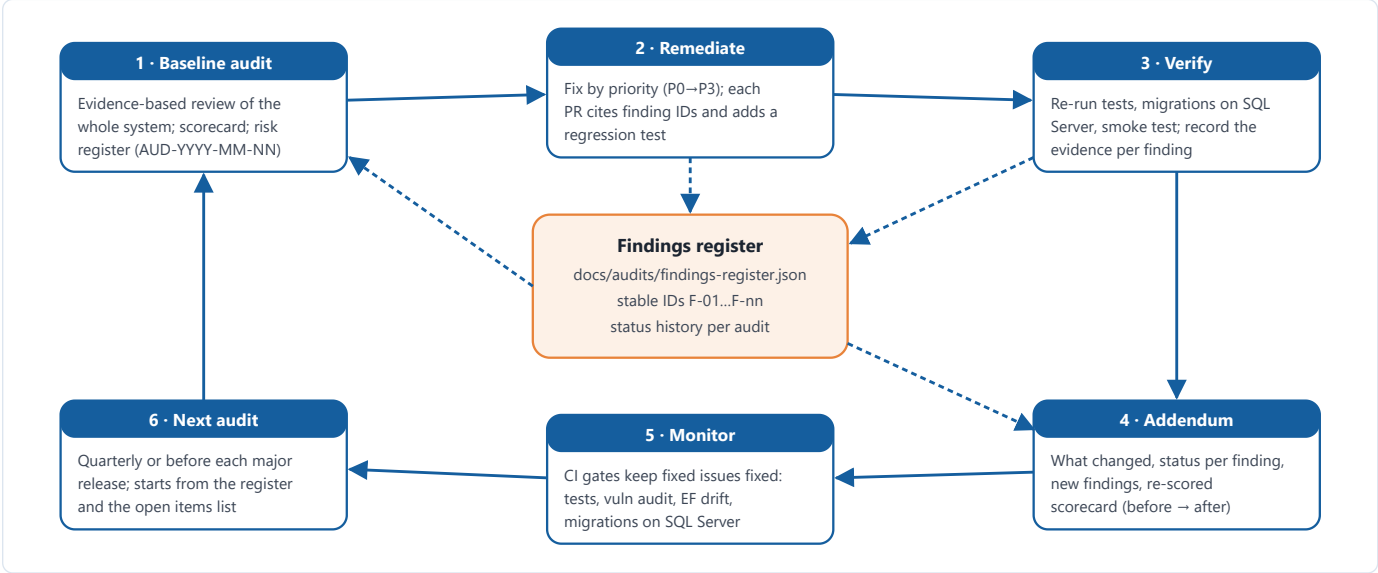
The smallest set of actions that moves the system to *production candidate* (details in 06 — Implementation Plan, Phase 1):

1. **T-1.1 / F-23:** run a real sandbox payment with your own PayFast merchant and passphrase.
2. **T-1.2 / F-10, F-02:** verify the live App Service (Production environment, no demo accounts, no anonymous function).
3. **T-1.3 / F-36:** Login lockout and staff MFA.
4. **T-1.4 / F-17:** a tested database restore.
5. **T-1.5 / F-38:** monitoring and alerts.
6. **T-1.6 / F-13:** migrations applied by the pipeline, not at startup.
7. **T-1.7 / F-09:** legal sign-off of the privacy notice.
8. **Confirm CI** on commit `921e5d8` and later, including the first green run of the SQL Server migrations job, and protect `main` with those checks.
9. **Audit the new Android app** (`app/`) before it ships.

All findings that are not yet fixed:

ID	Finding	Severity	Baseline ref	Status	Evidence / verification	Remaining action
F-06	Donation flow never sends the donor to PayFast	High	R6	Partially fixed	Checkout wired with return/cancel/notify URLs; smoke test on LocalDB produced a correctly signed sandbox redirect for the rand amount (USD 10 → amount=185.00).	End-to-end payment not yet verified: blocked by F-23.
F-09	No privacy notice or consent for volunteer and donor personal data (POPIA)	High	R9	Partially fixed	Privacy notice written; consent checkbox and timestamp on Donate, Volunteer and Contact. Tests: Volunteer_WithoutConsent_IsNotSaved, Contact_RecordsConsentTimestamp, PrivacyPage_IsARealNoticeNotTheTemplate.	Legal review; appoint an Information Officer; automate retention and deletion.
F-10	Live site may run in Development with published staff credentials	High	R10	Partially fixed	Demo accounts are seeded only in Development or with Seed:DemoData=true; README no longer advertises a live staff login.	Owner must confirm ASPNETCORE_ENVIRONMENT=Production on App Service and remove or rotate demo accounts there (cannot be verified from the repository).
F-23	Shared PayFast sandbox merchant rejects every signed request; payments unverified end to end	High	New	Open	Signature string matches PayFast's documented PHP algorithm byte for byte; the sandbox answers 'signature does not match' for every variant (GET/POST, with/without the documented passphrase) but accepts unsigned requests, so the shared merchant's passphrase is unknown.	Register an own PayFast sandbox merchant, set PayFast:Passphrase, run a sandbox payment with a public NotifyUrl and record the evidence.
F-13	Migrations run at startup on every instance	Medium	R13	Partially fixed	Database:MigrateOnStartup switch; idempotent migrate.sql produced by CI and Azure Pipelines.	Set Database:MigrateOnStartup=false in production and add the script step to the release.
F-17	No backups or disaster recovery procedure	Medium	R17	Open	Procedure documented in deliverable 07; no restore has been tested.	Confirm the Azure SQL retention tier, perform and record a test restore.
F-19	Missing security headers, Subresource Integrity and host filtering	Medium	§28	Partially fixed	CSP, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy; SRI on the CDN stylesheet. Test: Responses_CarrySecurityHeaders.	CSP still allows 'unsafe-inline'; set AllowedHosts per environment.
F-36	Login lockout-on-failure not enabled by the default Identity UI; no staff MFA	Medium	§27	Open	Lockout options configured and POST rate limiting mitigates brute force, but the default Login page does not count failures.	Scaffold the Login page with lockoutOnFailure: true; require MFA for the Employee role.
F-37	Recurring donations are only a label	Medium	§5	Partially fixed	Relabelled 'Monthly pledge' with honest copy on the form, receipt and README.	Implement PayFast subscriptions with per-payment child donations.
F-38	Monitoring limited to console logs	Medium	§39	Partially fixed	Serilog request logging; /health (database) and /health/live probes.	Application Insights or OpenTelemetry; alerts on failed ITNs and health.
F-46	Exchange rates are manual configuration	Medium	New	Open	Indicative USD 18.50 and EUR 20.00 in appsettings; also used by the historical backfill.	Integrate a rates provider or a staff-maintained rate table with effective dates.
F-26	Developer database contains migration 20260924223317_SnapshotSync that is not in the repository	Low	New	Open	Observed read-only in __EFMigrationsHistory of the local GiftOfTheGiversDb.	Confirm whether any shared or live database contains it; recreate local databases from repository migrations.
F-29	Azure Function references the whole web project	Low	§7	Open	Deferred: low risk once the function is key-protected.	Extract a shared data project (implementation plan T-2.3).
F-42	Azure Function has no tests or deployment pipeline	Low	§41	Open	The function builds in CI only.	Add a deployment job and tests (implementation plan T-2.3).

8. Audit vision — how future audits work



Audit programme lifecycle

The goal is that **every future audit is faster, comparable, and builds on this one**:

Principle	How it is implemented
One register, stable IDs	`docs/audits/findings-register.json` : IDs are never reused or renumbered; each audit appends a history entry per finding
Evidence-based status	*Fixed* requires a test, a CI gate or a recorded verification; *Partially fixed* means an operational step remains
Comparable scoring	The same 26 areas and weights (sum 105, normalised). Caps: open Critical → ≤ 4.5; open High → ≤ 6.5
Regression-proof	Each Critical/High fix has a test or CI gate (role matrix, money tests, vuln gate, drift check, SQL Server migrations)
Reproducible reports	Markdown sources plus `python docs/tools/build_deliverables.py` produce identical HTML and PDF
Trend visibility	`docs/audits/AUDIT-HISTORY.md` tracks score, maturity, open counts, tests, coverage and vulnerabilities per audit

Cadence

Trigger	Audit type
Before the first production release (M1)	Full audit **AUD-2027-xx** using the master brief, starting from the open register items
Quarterly	Remediation review (`-R2` , `-R3` …) of open and partial findings + CI evidence
After any security incident, new payment feature, or schema overhaul	Targeted audit of the affected areas

Score trend

Audit	Weighted	Reported	Maturity	Critical open	Tests	Coverage	Vulnerable packages
AUD-2026-11-01 (baseline)	4.07	3.9	Functional prototype	3	19	not measured	1
AUD-2026-11-01-R1 (this review)	7.03	6.5	Beta	0	98	67.9%	0
Next (target, M1)	≥ 8.0	—	Production candidate	0	+ UI tests	≥ 70%	0

Running the next audit

1. `python docs/tools/findings.py open` lists the starting point.
2. Run the verification commands in section 3 and record the results.
3. Append the audit to the register (`audits[]`) and a history entry for each finding re-checked; add new findings with the next free ID.
4. Copy this addendum's source as `src/08-…-addendum-R2.md` (or write a new full audit for a baseline), then run `python docs/tools/build_deliverables.py` .
5. Add a row to `docs/audits/AUDIT-HISTORY.md` .

Gift of the Givers · Relief Management System

DELIVERABLE 09

Part 3 — Testing, Deployment and Handover

What was added and changed in Part 3, the evidence for each rubric criterion, the load and stress analysis, the continuous-deployment design and the handover package.

DOCUMENT	VERSION	DATE	STATUS
D-09	1.0	2026-11-02	Testing complete (local + CI-ready); Azure deployment configured, awaiting first run

CONTENTS

1. Summary
2. What changed in Part 3
3. A.1 Unit tests
4. A.2 Integration tests
5. A.3 Load and stress testing
6. A.4 Selenium UI tests
7. B. Tests in Azure Pipelines
8. C. Continuous deployment
9. D. Handover package
10. Screenshots to capture after the Azure deployment
11. Known limitations
12. Declaration of AI assistance

Appendix: run everything locally

1. Summary

Part 3 turns the Part 2 solution into a tested, deployable product with a handover pack:

Area	Result	Evidence
Unit tests (A.1)	**48 unit tests pass**, including 12 new `DonationService` tests with stubbed dependencies	`docs/evidence/part3/tests/unit.trx`
Integration tests (A.2)	**80 integration tests pass**, including a full *web form → payment notification → Azure Function → database* workflow	`docs/evidence/part3/tests/integration.trx`
Code coverage	**89.9 % lines, 67.9 % branches** (merged unit + integration)	`docs/evidence/part3/tests/coverage-summary.md`
Load test (A.3)	20 and 100 concurrent users, **0 errors**, p95 6–11 ms	`docs/evidence/part3/load-*`
Stress test (A.3)	10 → 800 users; bottleneck found (home page), **mitigated with caching + index**, re-measured, recovery verified	`docs/evidence/part3/stress-*`, section 5
Selenium UI tests (A.4)	**12 scenarios × 2 browsers (Chrome, Edge) = 24/24 pass**, desktop + mobile sizes, negative paths	`docs/evidence/part3/ui/*.trx`, 20 screenshots
Pipeline with tests (B)	Azure Pipeline runs unit → integration → Selenium, publishes 3 test runs, a merged coverage report and a **≥ 70 % coverage gate**	`azure-pipelines.yml`
CD (C.1–C.3)	Stages for Azure SQL (idempotent migrations + rollback script), App Service (staging slot, smoke test, swap, automatic swap-back) and Function App (config from secrets, 3-case live test)	`azure-pipelines.yml`, `pipelines/templates/*`
Handover (D)	User manual (deliverable 10), Azure deployment guide for the team member doing the set-up (deliverable 11), team contribution report (deliverable 12)	`docs/deliverables/10-12`

> **Evidence gate — action required.** The live site still shows the Azure "*waiting for your content*" page, and the deployment stages have not yet run in Azure DevOps. A PAT shared for this purpose was rejected by Azure DevOps (HTTP 401) when tested on 2026-11-02. Before submission, a team member must follow deliverable 11 (about 2 hours) and capture the screenshots listed in section 10. Until then, sections 7 and 8 describe configuration, not a completed deployment.

2. What changed in Part 3

2.1 New projects

Project	Purpose
`GiftOfTheGivers.UiTests`	Selenium WebDriver tests (xUnit). Starts the real app on a free port with Kestrel and an in-memory database, or targets a deployed site through `UI_BASE_URL`. Supports Chrome, Edge and Firefox (`UI_BROWSER`), headless or visible (`UI_HEADLESS`), and saves labelled screenshots (`UI_SCREENSHOT_DIR`).
`GiftOfTheGivers.LoadTests`	Console load/stress runner, run from Visual Studio or `dotnet run`. Virtual users follow the donor journey; it writes `requests.csv`, `timeline.csv`, `summary.json` and `summary.md`.

2.2 New and changed tests (`GiftOfTheGivers.Tests`)

File	Change
`DonationServiceTests.cs`	**New**, 12 unit tests: currency conversion through a stub converter, idempotency, inactive projects, payment verification, replayed and tampered notifications, recalculation. A recording mock checks the audit trail.
`EmployeeWorkflowTests.cs`	**New**, 8 integration tests: create project (valid + invalid dates), post and edit a project update, validation, 404s, volunteer approve → assign → unassign → reject, inquiry handling, offline-funds reconciliation.
`DonorPortalTests.cs`	**New**: donor dashboard totals, donation history ownership, public pages, and `DbSeederTests` (demo seed is complete and idempotent).
`TaxCertificateFunctionTests.cs`	**New**, 6 integration tests calling the Azure Function in-process against the web app's database.
All existing test classes	Tagged `[Trait("Category", "Unit")]` or `"Integration"` so the pipeline runs and reports them separately.
`Infrastructure/TestApplicationFactory.cs`	Turns off the new cache in tests (`Caching:PublicPageSeconds = 0`).
`GiftOfTheGivers.Tests.csproj`	References the Functions project (aliased `functions`, because both projects declare `Program`).

2.3 Application changes

File	Change	Why
`ViewComponents/ImpactStatsViewComponent.cs`	Five aggregate queries cached for `Caching:PublicPageSeconds` (default 30 s)	Stress-test bottleneck (section 5.3)
`Controllers/HomeController.cs`	Featured projects on the home page cached the same way	Same
`Services/AppOptions.cs`, `Program.cs`, `appsettings.json`	`CachingOptions`, `AddMemoryCache()`, `"Caching"` section	Configurable; 0 disables
`Data/ApplicationDbContext.cs`	`ReliefProject.Status` limited to 20 characters; index `IX_ReliefProjects_Status_CreatedDate`	Home and project pages filter by status and sort by date
`Data/Migrations/202611021132424_Part3PerformanceIndexes.cs`	**New migration.** Drops and recreates the `CK_ReliefProject_Status` check constraint around the column change (SQL Server cannot alter a column a constraint depends on), and has a full `Down()`	Schema update deployed by CD (C.3)
`Controllers/EmployeeController.cs`, `Data/DbSeeder.cs`	`RowVersion` initialised on new projects	Found by the new integration tests: project creation failed on non-SQL providers
`Controllers/HomeController.cs`	`Home/VolunteerConfirmation` and `Home/ContactConfirmation` routes	Found by the new public-pages test: the conventional URLs returned 404

2.4 Pipelines and tooling

File	Change
`azure-pipelines.yml`	Rewritten: separate unit, integration and Selenium steps; three published test runs; merged coverage; coverage gate; migration **and rollback** scripts; function packaging; CD stages for SQL, App Service and Functions; live Selenium smoke tests; `rollbackDatabase` run parameter.
`pipelines/templates/variables.yml`	**New.** Non-secret settings and resource names, plus `deployEnabled`.
`pipelines/templates/dotnet-setup.yml`	**New.** Reusable SDK + cache + feed authentication + locked restore.
`pipelines/templates/smoke-test.yml`	**New.** Reusable post-deployment health and page checks.
`coverage.runsettings`	**New.** Coverage of hand-written code only (migrations and compiled Razor views excluded).
`.github/workflows/ci.yml`	Runs the Selenium suite in headless Chrome and writes a coverage table into the GitHub job summary.
`docs/tools/load_charts.py`	**New.** Draws the load/stress charts from the measured data.

2.5 Commits

The Part 3 work is in commits `b5d1179` (*part 3*), `d45db0e` (*updates*) and `9e7787c` (*test*) on `feature/erd-schema-and-pdf`. The UI test project, pipeline templates, coverage settings and documents are in the working tree for the next commit.

3. A.1 Unit tests

Where: `GiftOfTheGivers.Tests` (xUnit), run in Visual Studio's **Test Explorer** or with `dotnet test --filter Category=Unit`.

Class	Tests	Critical logic covered
`DonationServiceTests`	12	Donation capture, USD/EUR → ZAR conversion, idempotency, project availability, payment verification, replay protection, amount/merchant tampering, totals recalculation
`HelperCalculationsTests`	7	Donation totals, funding percentage, certificate numbers (NuGet helper package)
`CsvTests`, `MoneyTests`	9	CSV injection protection, currency formatting
`CurrencyConverterTests`	3	Rates, unknown currency, rounding
`PayFast*Tests`	5	Signature, options validation, sandbox vs live
`ModelTests`, `P1ValidationTests`	8	Validation attributes (amount range, consent, image URL)
`TaxCertificate*Tests`	3	Certificate PDF and issuer details
`DatabaseContextTests`	1	EF Core persistence
Total	**48**	**All pass** (`unit.trx`)

Mocks and stubs. `DonationServiceTests` replaces the `ICurrencyConverter` with a stub that returns rates set by each test and records every call, and replaces `IAuditService` with a recording mock that the tests assert on, for example "*a verified payment writes exactly one `PaymentVerified` audit entry*". The integration tests use a `FakePaymentGateway`, so no test ever calls PayFast.

```
[Fact]
public async Task Capture_ConvertsWithTheInjectedRate_AndStoresPendingDonation()
{
    _converter.Rates["USD"] = 18m;
    var form = Form(amount: 10m, currency: "USD", type: "Recurring", anonymous: true);

    var result = await CreateService().CaptureAsync(form, donorUserId: null);

    Assert.Equal(180m, result.Donation!.AmountZar);
    Assert.Equal(PaymentStatus.Pending, result.Donation.PaymentStatus);
    Assert.Equal(("USD", 10m), _converter.Calls.Single());
}
```

Coverage: 89.9 % of lines and 67.9 % of branches in the application, function and helper assemblies, from unit and integration tests together. Generated code (EF migrations, compiled Razor views) is excluded by `coverage.runsettings`.

4. A.2 Integration tests

Where: same project, `dotnet test --filter Category=Integration`. `TestApplicationFactory` hosts the **real** application (routing, anti-forgery, authorisation, model binding, views, EF Core) in memory, with an isolated database per test class.

Class	Tests	Flow
`TaxCertificateFunctionTests`	6	**Form → function → database**: donate (USD, monthly, anonymous) → function refuses before payment (404) → PayFast notification verifies → function returns a PDF; plus malformed (400), unknown (404) and failed-payment cases
`DonationFlowTests`	20	Donation capture, currency, idempotency, validation errors, payment notifications (valid, replayed, wrong amount, wrong merchant), certificate ownership
`EmployeeWorkflowTests`	8	Employee: create project, post/edit update, volunteer lifecycle, inquiries, reconciliation, each with audit-trail checks
`AuthorizationTests`	36	Every employee page × anonymous / donor / employee
`DonorPortalTests`, `DbSeederTests`, `PublicFormsTests`, `RateLimitingTests`	10	Donor portal, public pages, contact and volunteer forms, demo seeding, rate limiting
Total	**80**	**All pass** (`integration.trx`)

The certificate workflow test, abridged:

```
var submitted = await TestApplicationFactory.PostFormAsync(client, "/Home/Donate", "/Home/Donate", form);
var donation  = await _factory.WithDbAsync(db ⇒ db.Donations.SingleAsync(d ⇒ d.IdempotencyKey == token));
Assert.Equal(370m, donation.AmountZar);                                  // 20 USD × 18.50

Assert.Equal(HttpStatusCode.NotFound, (await CallFunctionAsync(donation.TransactionReference!)).StatusCode);
await client.PostAsync("/Payments/Notify", /* COMPLETE, amount_gross = 370.00 */);
var response = await CallFunctionAsync(donation.TransactionReference!);   // Azure Function, same DB
Assert.Equal("application/pdf", response.Headers.GetValues("Content-Type").Single());
```

Writing these tests exposed two real defects, both fixed (section 2.3): project creation failed when no SQL Server rowversion was generated, and two confirmation pages were unreachable at their conventional URLs.

5. A.3 Load and stress testing

5.1 Configuration

Setting	Load test	Stress test
Tool	`GiftOfTheGivers.LoadTests` (.NET 10 console, `HttpClient`, one connection pool)	same
Target	Release build of the web app, Production environment, SQL Server LocalDB with demo data, `http://localhost:5080`	same
Journey per virtual user	`/` → `/Home/ReliefProjects` → `/Home/Donate` → `/health/live`, repeated	same
Think time	1.0–1.5 s between requests (realistic visitors)	none (maximum pressure)
Users	20 for 60 s; 100 for 60 s	stages of 10, 50, 100, 200, 400, 800 users, 20 s each
Timeout	30 s	10 s (counts as an error)
Machine	Windows 11 developer PC, app and load generator on the same machine	same

Commands:

```
dotnet run --project GiftOfTheGivers.LoadTests -c Release -- load   --base-url http://localhost:5080 --users 20 --duration 60
dotnet run --project GiftOfTheGivers.LoadTests -c Release -- stress --base-url http://localhost:5080 --stages 10,50,100,200,400,80
```

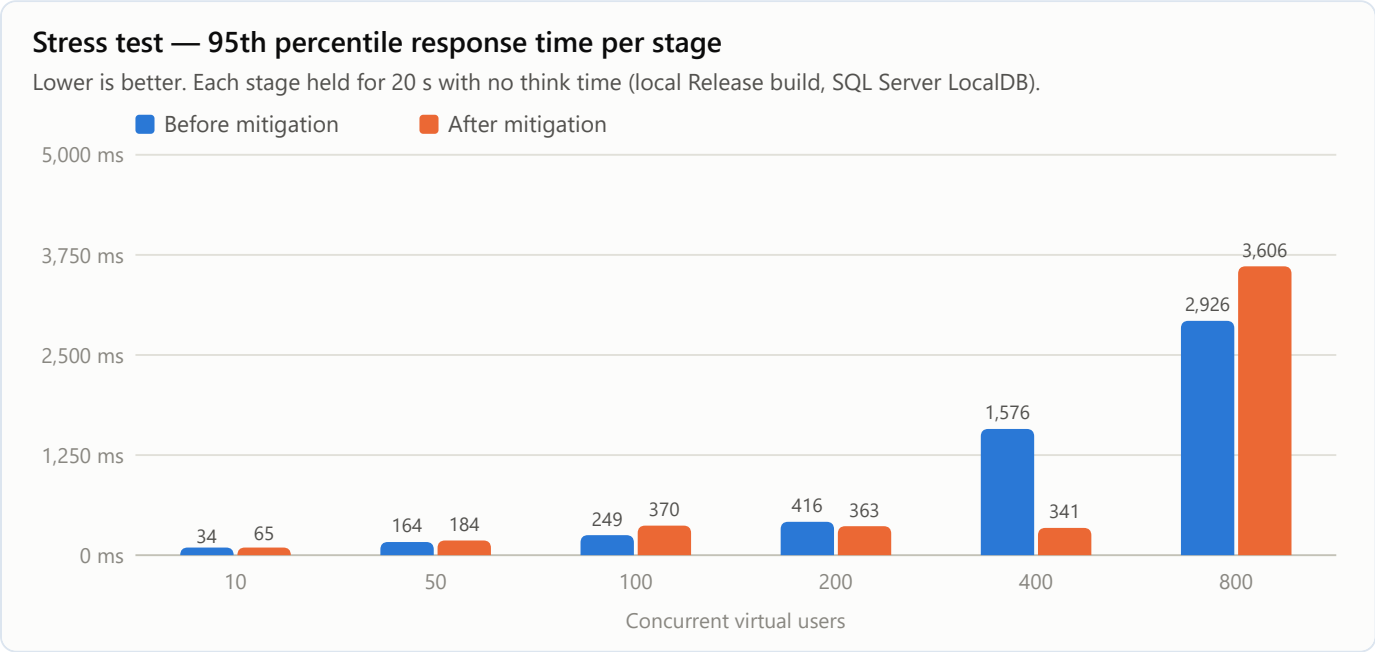
The same commands work against Azure with `--base-url https://<site>`. On a B1 plan, keep the stress stages to 50 users or fewer, so the test does not affect other users.

5.2 Load test results

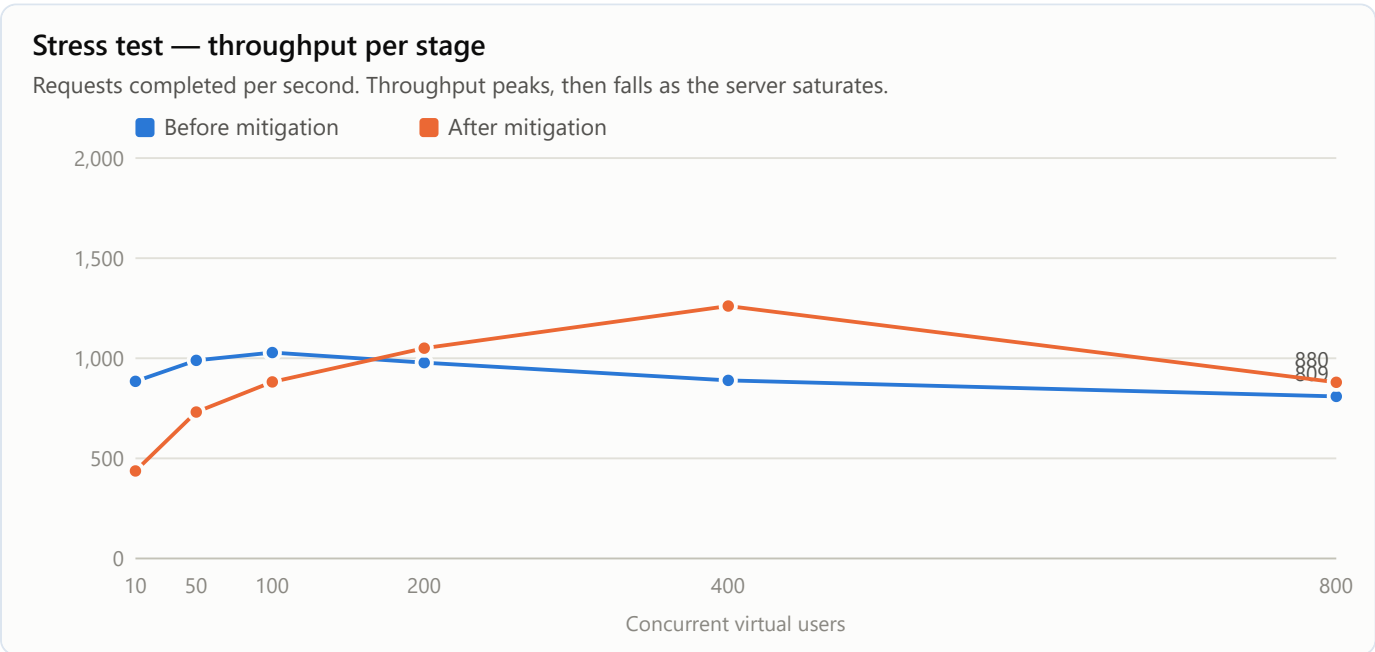
Run	Users	Requests	Throughput	Avg	p95	p99	Max	Errors
Before mitigation	20	951	15.8 req/s	5 ms	11 ms	33 ms	189 ms	0
After mitigation	20	952	15.9 req/s	3 ms	6 ms	12 ms	37 ms	0
After mitigation	100	4,761	79.3 req/s	4 ms	11 ms	44 ms	132 ms	0

With realistic think time, 100 concurrent visitors are served with no errors and a p95 of 11 ms. Throughput rises in proportion to users, which shows the server is not saturated at this level.

5.3 Stress test: finding the bottleneck



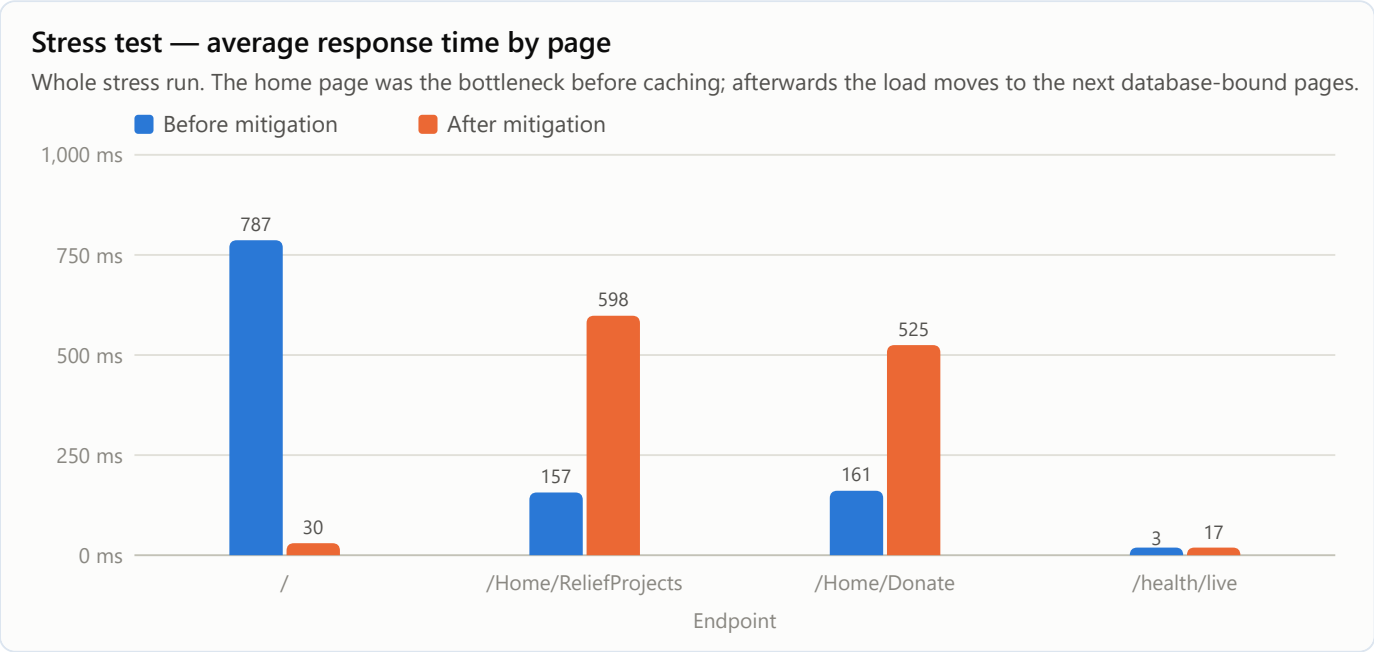
Stress test p95 per stage, before and after mitigation



Stress test throughput per stage

Users	Before: req/s	Before: p95	Before: errors	After: req/s	After: p95	After: errors
10	885	34 ms	0 %	437	65 ms	0 %
50	990	164 ms	0 %	732	184 ms	0 %
100	**1,028** (peak)	249 ms	0 %	881	370 ms	0 %
200	978	416 ms	0.5 %	1,051	363 ms	0.6 %
400	889	1,576 ms	0 %	**1,261** (peak)	341 ms	1.2 %
800	809	2,926 ms	0 %	880	3,606 ms	3.7 %

Before. Throughput peaked at about **1,030 requests/s at 100 users**, then fell while latency rose sharply (p95 1.6 s at 400 users, 2.9 s at 800). The first timeouts (102 requests over 10 s) appeared at 200 users. The per-page breakdown pointed to one cause:



Average response time by page, whole stress run

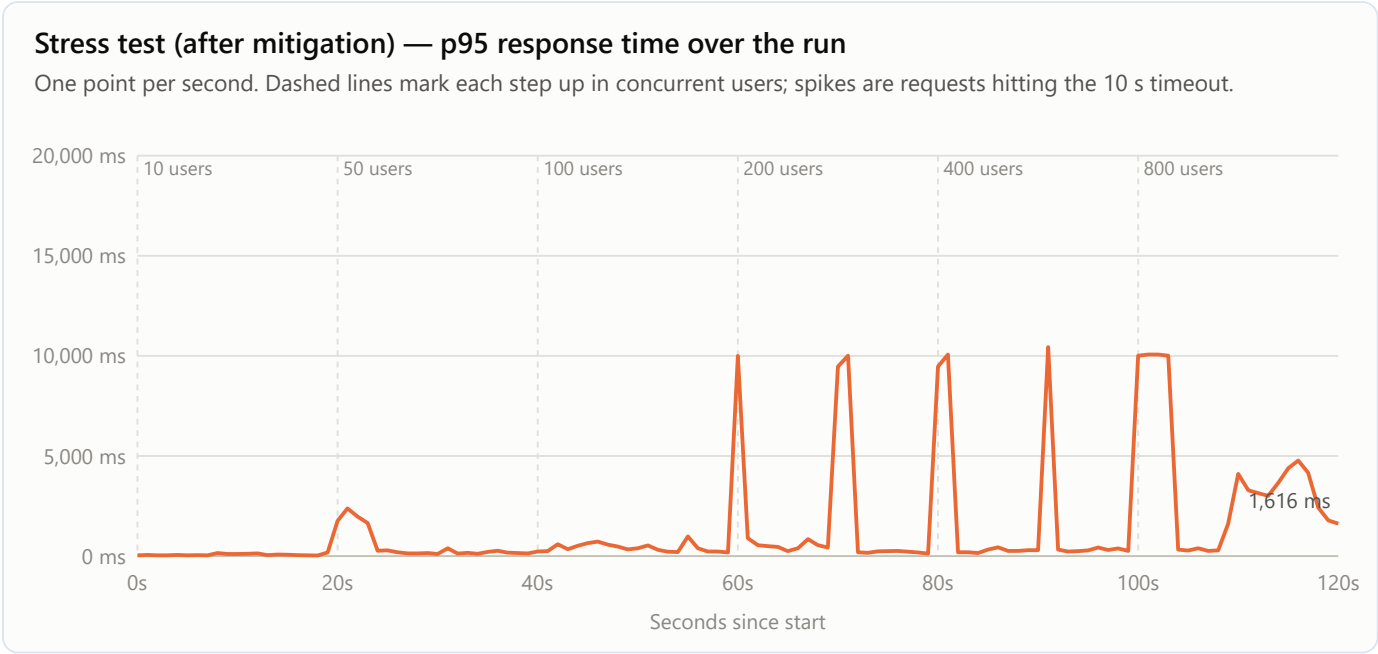
The **home page averaged 787 ms**, five times slower than the other pages (≈ 160 ms). Each home-page view runs **six database queries**: five aggregates for the impact figures (`ImpactStatsViewComponent`) and one for the featured projects. Under concurrency these queries compete for the database connection pool (100 connections by default), so requests queue for a connection.

5.4 Mitigation and re-test

1. **Cache** the impact figures and featured projects in memory for 30 s (`IMemoryCache` , configurable through `Caching:PublicPageSeconds`). The figures are at most 30 s out of date, which is acceptable for public statistics.
2. **Index** `ReliefProjects(Status, CreatedDate)` , which both public project queries filter and sort on (migration `Part3PerformanceIndexes`).
3. The existing **retry logic** (`EnableRetryOnFailure`) and **rate limiting** of form posts (20 per minute per IP, HTTP 429) already protect the database and forms from bursts and abuse. `RateLimitingTests` proves the limiter works.

After. The home page fell from **787 ms to 30 ms** on average. Peak throughput rose by 23 %, from 1,028 to **1,261 req/s**, and moved from 100 to 400 users. At 400 users the p95 improved from 1,576 ms to 341 ms.

The bottleneck moved. With the home page cheap, the next database-bound pages (`/Home/ReliefProjects` 598 ms, `/Home/Donate` 525 ms on average) took the load, and timeouts at 400–800 users rose to 1.2–3.7 %. The first stage was also slower in the second run (437 vs 885 req/s), most likely because of the cold start after redeploying on the same PC. The timeline shows the spikes are individual requests hitting the 10 s timeout once more than 200 users run with no think time:



p95 over the stress run, after mitigation

Recovery. Straight after the 800-user stage, `/health` answered **HTTP 200 in 49 ms**, and a 20-user load test immediately afterwards ran with **0 errors and p95 6 ms**. The application recovers without a restart (`stress-after/recovery-health.txt` , `load-20users-after`).

5.5 Conclusions and recommendations

Finding	Recommendation
Capacity with realistic users is far above the Foundation's expected traffic (100 users: p95 11 ms, 0 errors).	No change needed for normal operation.
Under extreme load the database connection pool is the limit.	Cache the public project list the same way (it changes rarely), and use `AsNoTracking` projections for list pages.
Load tests ran on a developer PC with LocalDB, not on Azure B1 (1 core, 1.75 GB).	Repeat the 20-user load test against the Azure URL after deployment and add the result to this report. Expect higher latencies because of network distance and the smaller plan.
During a disaster appeal, traffic can spike 10× or more.	Scale out to 2+ instances (S1) during appeals; Azure SQL *serverless* or S0 for the database.

6. A.4 Selenium UI tests

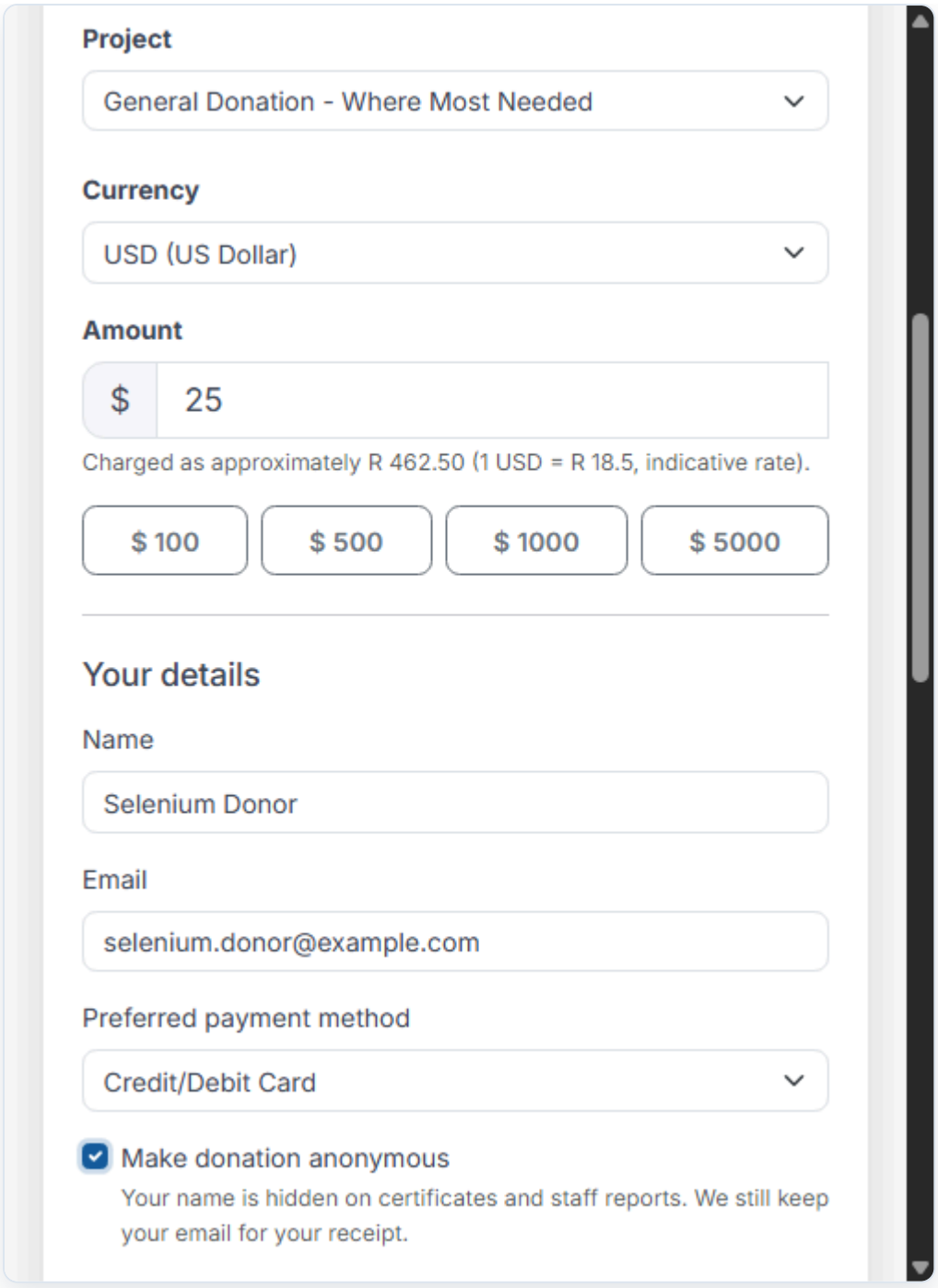
Where: `GiftOfTheGivers.UiTests/UserFlowTests.cs`. Visual Studio: **Test Explorer → GiftOfTheGivers.UiTests → Run**. Set `UI_HEADLESS=false` to watch the browser.

#	Scenario	Type	Sizes
1	Home page loads with Donate and Volunteer links (intro animation skipped)	Smoke	Desktop 1366×900 **and** mobile 390×844
2	Employee logs in → Employee Dashboard	Happy path	Desktop
3	Wrong password → *Invalid login attempt*, still signed out	**Negative**, Smoke	Desktop
4	Anonymous visitor opens `/Employee/Dashboard` → redirected to login	**Negative**, Smoke	Desktop
5	Anonymous **monthly** donation in **USD**, live rand preview (R 462.50), receipt *Awaiting payment*	Happy path	Desktop **and** mobile
6	Donation with invalid email, no amount, no consent → validation errors, nothing saved	**Negative**, Smoke	Desktop
7	Volunteer registration → confirmation page	Happy path	Desktop
8	Volunteer form with required fields empty → rejected	**Negative**, Smoke	Desktop
9	Employee posts a project update → success message	Happy path	Desktop
10	Donor logs in → dashboard → My Donations → tax certificate with Download link	Happy path	Desktop

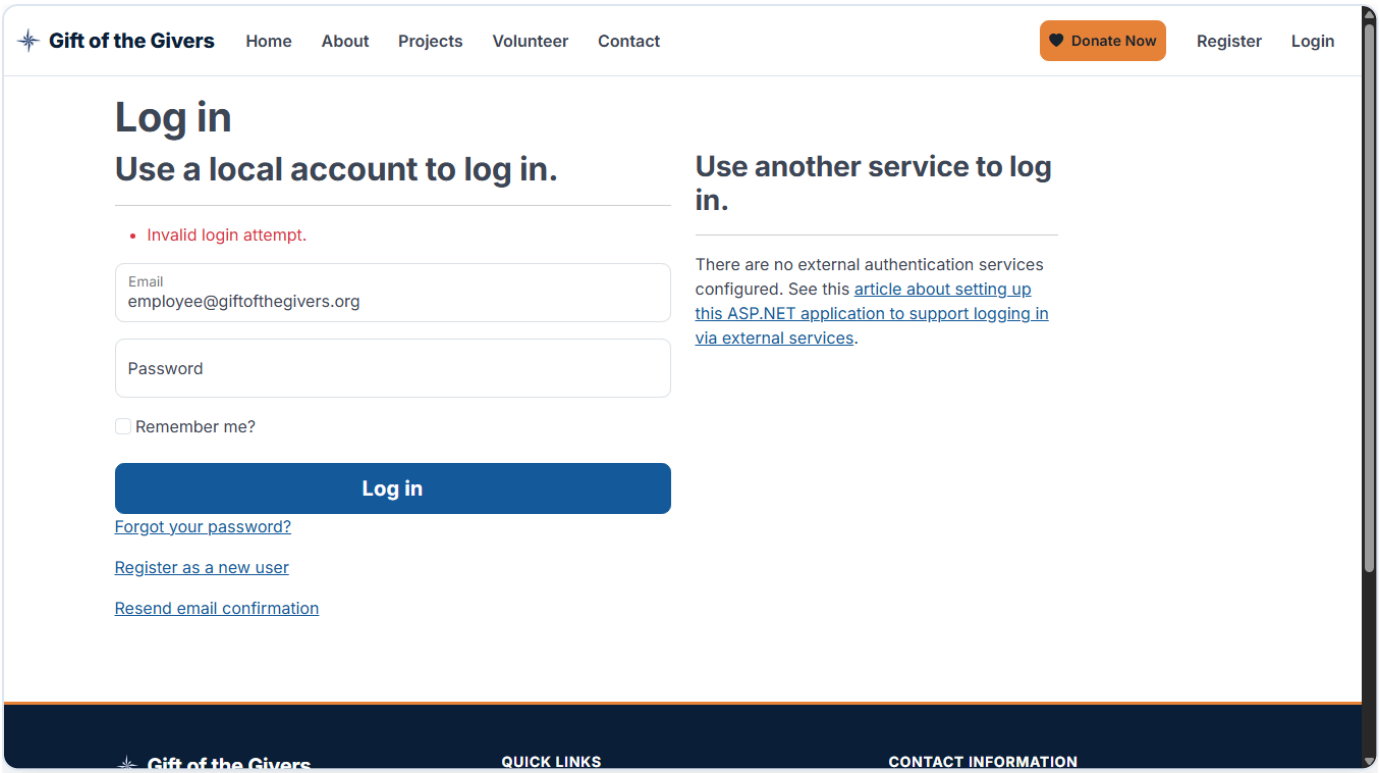
Results: 12 test cases (10 scenarios, 2 at two screen sizes), all passing in Chrome 156 and Microsoft Edge 156: 24 of 24 (`docs/evidence/part3/ui/ui-chrome.trx` , `ui-edge.trx` , `ui-*-console.txt`).

Robustness measures: the tests wait for conditions instead of sleeping; they retry stale elements during page changes; they scroll elements into view and fall back to a script click when a floating element covers the target; they load pages *eagerly*, so slow external fonts cannot time them out; and each test runs in a fresh browser.

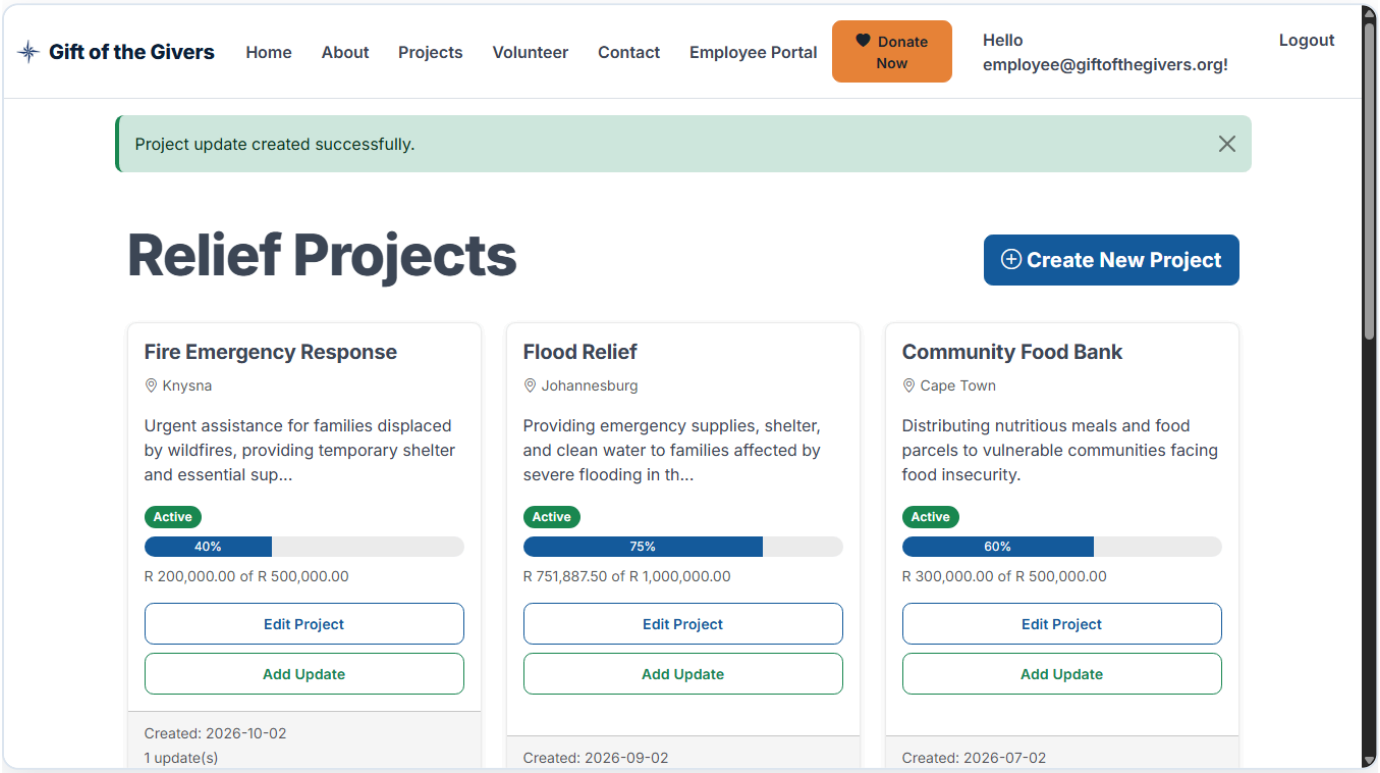
Screenshots. Each step saves a labelled PNG. The 20 Chrome screenshots are in `docs/deliverables/assets/screenshots/part3/` and are used in the user manual. Three examples:



Selenium: anonymous USD monthly donation, mobile size



Selenium: wrong password (negative path)



Selenium: employee posted a project update

7. B. Tests in Azure Pipelines

The `Build` stage of `azure-pipelines.yml` runs on every commit to `main` and `feature/*` and on every pull request into `main`:

Step	Detail
`dotnet-setup.yml` template	.NET SDK from `global.json`, NuGet cache, Azure Artifacts authentication, locked restore
Build	Release, whole solution
Unit tests	`--filter Category=Unit`, coverage collected, `unit.trx`
Integration tests	`--filter Category=Integration`, coverage collected, `integration.trx`
Selenium UI tests	headless Chrome, desktop + mobile; screenshots published as artifact `ui-screenshots`
Publish test results ×3	Three named runs (*Unit tests*, *Integration tests*, *Selenium UI tests*) in the run's **Tests** tab; any failure fails the build
Publish code coverage	`PublishCodeCoverageResults@2` merges both Cobertura files into the **Code Coverage** tab and the build summary
Coverage gate	Fails the build if merged line coverage < 70 % (currently 89.9 %)
Quality gates	Vulnerable-package scan, EF model-drift check
Packaging	`migrate.sql`, `rollback.sql`, `schema-version.txt`, `webapp.zip`, `functions.zip`

The GitHub workflow `ci.yml` runs the same tests on GitHub and writes a coverage table into the job summary, so every push is also checked before it is mirrored to Azure DevOps.

Status: the YAML is valid and every command in it was run locally. Screenshots of the first Azure DevOps run must be added after the set-up in deliverable 11 (section 10 lists them).

8. C. Continuous deployment

All deployment stages run only on `main`, wait for the approval on the `giftgivers-production` environment, and read secrets from the variable group `giftgivers-azure`, where they are marked secret and masked in logs. No secret is stored in the repository.

8.1 C.1 Web app (App Service)

Step	Purpose
`AzureAppServiceSettings@1`	Writes `Database__MigrateOnStartup=false`, `ForwardedHeaders__Enabled=true` and the connection string (from the secret group)
`AzureWebApp@1`	Deploys `webapp.zip`, to the **staging slot** when `useStagingSlot: 'true'`
Smoke test (template)	Waits for `/health/live`, then checks `/health`, `/`, projects, donate, volunteer and login for HTTP 200
`AzureAppServiceManage@0` Swap	Swaps staging into production with no downtime, then smoke-tests production
`on: failure` hook	**Automatic rollback**: swaps back, restoring the previous build that is still in staging

Deployment strategy. Blue/green with slots when the plan is Standard (S1) or higher. On Free/Basic plans, slots are not available: the pipeline deploys straight to production, and rollback means re-running *DeployWebApp* from the last good run, which redeploys that run's own artifact. Both paths are documented in deliverable 11, section 9.

8.2 C.2 Function app

Step	Purpose
`AzureFunctionApp@2`	Deploys `functions.zip` (.NET 10 isolated worker)
`AzureAppServiceSettings@1`	`ConnectionStrings__DefaultConnection` from the secret group, organisation settings, `FUNCTIONS_WORKER_RUNTIME`
Live test, 3 inputs	no key → **401** (function-level authorisation); malformed reference → **400** (validation); valid but unknown reference → **404** (database lookup works). Each call is retried while the function warms up.

The function key is a secret pipeline variable (`functionKey`), never in the repository. The same three cases, plus the success path, run in-process on every build (`TaxCertificateFunctionTests`).

8.3 C.3 SQL database

Mechanism	Detail
Migrations	EF Core migrations are the source of truth; 8 versions, the newest is `20261102132424_Part3PerformanceIndexes`
Automated script	The build produces an **idempotent** `migrate.sql` (each migration is guarded by `__EFMigrationsHistory` , so it is safe on any database version) and `rollback.sql` (newest → previous)
Deployment	`SqlAzureDacpacDeployment@1` runs `migrate.sql` on Azure SQL, then lists `__EFMigrationsHistory` in the log, so the **schema version is visible** in every run
App start-up	`Database:MigrateOnStartup=false` in Azure, so only the pipeline changes the schema
Rollback	Run the pipeline with **rollbackDatabase** ticked: it runs `rollback.sql` and skips app deployment. For data loss: Azure SQL point-in-time restore (deliverable 11, section 9)
Drift protection	The build fails if the model changed without a migration

Evidence that schema updates and rollback work (SQL Server LocalDB, 2026-11-02, `docs/evidence/part3/sql/migration-rollback-evidence.txt`): all migrations applied; `Part3PerformanceIndexes` rolled back to `AuditRemediationNov2026` ; re-applied; `__EFMigrationsHistory` lists all 8 versions; `IX_ReliefProjects_Status_CreatedDate` present. The generated `forward-` and `rollback-Part3PerformanceIndexes.sql` are stored alongside. The GitHub CI job *Apply migrations to SQL Server* repeats apply → roll back → re-apply on SQL Server 2022 on every push.

```
-- rollback-Part3PerformanceIndexes.sql (generated, abridged)
DROP INDEX [IX_ReliefProjects_Status_CreatedDate] ON [ReliefProjects];
ALTER TABLE [ReliefProjects] DROP CONSTRAINT [CK_ReliefProject_Status];
ALTER TABLE [ReliefProjects] ALTER COLUMN [Status] nvarchar(max) NOT NULL;
ALTER TABLE [ReliefProjects] ADD CONSTRAINT [CK_ReliefProject_Status] CHECK ([Status] IN ('Active','Completed','Suspended'));
DELETE FROM [__EFMigrationsHistory] WHERE [MigrationId] = N'20261102132424_Part3PerformanceIndexes';
```

8.4 GitHub → Azure synchronisation

Every push to GitHub runs `.github/workflows/mirror-to-azure-devops.yml`, which mirrors all branches and tags to Azure Repos with a PAT held in GitHub secrets. Azure Pipelines triggers on the mirrored commit. Deliverable 11, section 5, gives the full set-up and test procedure.

9. D. Handover package

Item	Location
User manual: login (employee / donor / anonymous), posting updates, donations (once-off, monthly, anonymous, ZAR/USD/EUR), tax certificates, volunteering, accessibility, FAQ. 17 labelled screenshots.	Deliverable 10
Azure deployment and GitHub sync guide, for the member doing the set-up	Deliverable 11
Team contribution report with git evidence	Deliverable 12 (**names and roles to be completed by the team**)
Earlier deliverables (requirements, design, schema, plan, CI/CD, audits)	Deliverables 01–08
Raw evidence (TRX, coverage, load CSV/JSON, SQL scripts, screenshots)	`docs/evidence/part3/`

10. Screenshots to capture after the Azure deployment

These cannot be produced until the deployment in deliverable 11 has run. Label each one, for example *"Figure C.1-2 — Swap completed"*.

For	Screenshot
B.1	YAML in Azure DevOps; a run summary showing *Build and test* green
B.1/B.2	**Tests** tab with the three runs (unit 48, integration 80, UI 12) all passing
B.2	**Code Coverage** tab; the *Coverage gate* log line
B.2	Commit on GitHub → mirror Action green → triggered run in Azure DevOps (same SHA)
C.1	DeployWebApp log (deploy, smoke test, swap); the live site in a browser; App Service *Deployment slots* page
C.2	DeployFunctions log with the 401/400/404 results; a browser download of a certificate PDF; Function App → *Monitor* invocation log
C.3	DeployDatabase log with the `__EFMigrationsHistory` listing; portal *Query editor* showing the same table
A.3	Optional: 20-user load test against the Azure URL (`summary.md`)

11. Known limitations

- **Not yet deployed to Azure** (section 1). This is the main risk to the marks for C.1–C.3.
- Load and stress figures come from a developer PC, not from Azure B1.

- Monthly pledges are recorded but not billed automatically (PayFast subscriptions are not integrated). The user manual says so.
- Some mobile-API work (`Controllers/Api`, `GiftOfTheGivers.Mobile`) was still in progress in the working tree when this report was written. It did not compile at that time and is not covered by this report.

12. Declaration of AI assistance

The team must review and complete this declaration so that it is accurate before submission. IIE guidelines allow AI for idea generation, grammar and debugging, but not for writing full code or solutions, or for producing images, diagrams or tables.

- An AI coding assistant (Claude Code) and the GitHub Copilot coding agent (11 commits, visible in the history) were used during this project. In Part 3, AI assistance was used for: *[the team lists exactly what, e.g. drafting test cases, the load-test runner, pipeline YAML, this report, the chart script]*.
- All AI-assisted work was reviewed, run and verified by the team. Every figure in this report comes from test runs whose raw output is in `docs/evidence/part3/`.
- *[Each member states how they personally used AI tools.]*

Appendix: run everything locally

```
# Unit + integration tests with coverage (Visual Studio: Test Explorer → Run All)
dotnet test GiftOfTheGivers.Tests --settings coverage.runsettings --collect "XPlat Code Coverage"

# Selenium (Chrome by default; UI_BROWSER=edge for Edge; UI_HEADLESS=false to watch)
dotnet test GiftOfTheGivers.UiTests

# Selenium against a deployed site, read-only checks only
UI_BASE_URL=https://<site> dotnet test GiftOfTheGivers.UiTests --filter Category=Smoke

# Load / stress (start the app first: dotnet run --project GiftOfTheGivers -c Release --urls http://localhost:5080)
dotnet run --project GiftOfTheGivers.LoadTests -c Release -- load --users 20 --duration 60
dotnet run --project GiftOfTheGivers.LoadTests -c Release -- stress --stages 10,50,100,200 --stage-seconds 20

# Charts from the latest stress results
python docs/tools/load_charts.py

# Rebuild these documents (HTML + PDF)
python docs/tools/build_deliverables.py
```

Gift of the Givers · Relief Management System

DELIVERABLE 10

User Manual

How to sign in, post project updates, make donations, get a tax certificate and register as a volunteer, for staff, donors and volunteers of the Gift of the Givers Foundation.

DOCUMENT	VERSION	DATE	STATUS
D-10	1.0	2026-11-02	Handover edition

CONTENTS

1. About this manual
2. Signing in
3. Getting staff (employee) access
4. Employees: posting a project update
5. Making a donation
6. Getting a tax certificate (Section 18A)
7. Registering as a volunteer
8. Accessibility and good practice
9. Frequently asked questions
10. Getting help

1. About this manual

This manual explains how to use the Gift of the Givers relief website. It is written for Foundation staff with no technical background. Every task is a short list of numbered steps with a picture of the screen.

If you are…	Read
A **staff member** (employee)	Sections 2, 3 and 4
A **donor** with an account	Sections 2, 5 and 6
Giving **without an account** (anonymous)	Sections 5 and 6.3
Someone who wants to **volunteer**	Section 7

What you need: a phone, tablet or computer with an up-to-date web browser (Chrome, Edge, Safari or Firefox) and an internet connection. Nothing needs to be installed.

Website address: `https://gift-relief-managemen-gdbsf9bbena9epgw.southafricanorth-01.azurewebsites.net`. Your administrator will confirm the address once the site is live. Bookmark it.

Screens in this manual were captured automatically from the real website during testing on 2 November 2026, using demonstration data and test names. Your screen may show different projects and amounts. On a phone the menu is folded behind the ☰ button at the top right; everything else works the same.

> **Payments on the demonstration site.** Until the Foundation switches on live payments, the site records donations as *Awaiting payment* and does not charge anyone. The steps are the same once payments are live.

2. Signing in

There are three ways to use the site:

Type of user	Account needed?	What you can do
Anonymous visitor	No	Browse projects, donate (your name can be hidden), apply to volunteer, contact the Foundation
Donor	Yes (free, self-registration)	All of the above, plus see your donation history and download tax certificates
Employee (staff)	Yes, and an administrator must give you staff access	Manage projects, post updates, review volunteers, check donations, issue certificates

2.1 Anonymous visitors

You do not need to sign in. Open the website and use the menu at the top: **Projects**, **Volunteer**, **Contact** or the orange **Donate Now** button.

The first time you open the home page in a browser session, a short welcome animation plays and then the home page appears (Figure 1).

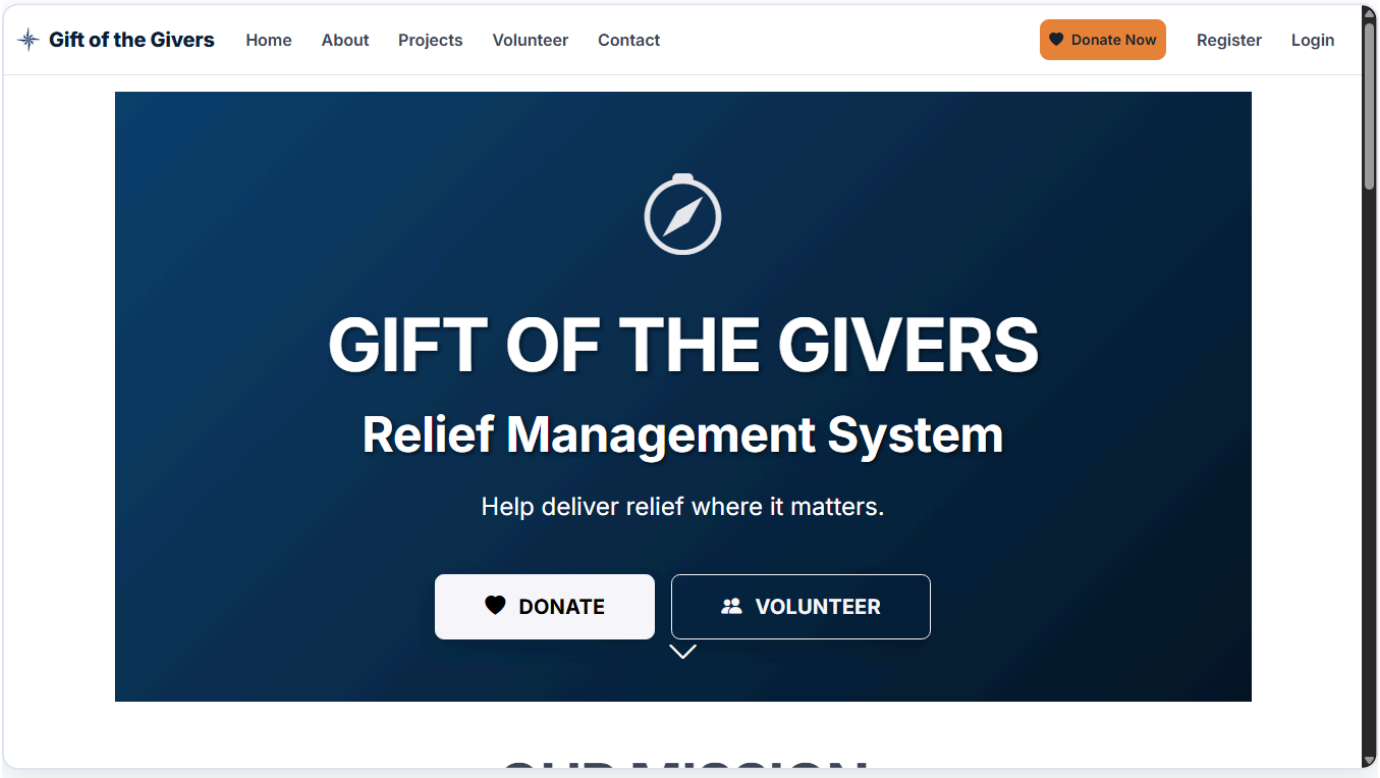


Figure 1 — Home page (desktop). The orange Donate Now button and the main menu are at the top.

2.2 Donors: create an account

1. Click **Register** at the top right.

2. Enter your full name, email address and a password of at least 8 characters. Confirm the password.
3. Click **Register**. You are signed in straight away as a donor.

Donations you make while signed in are linked to your account, so you can download their tax certificates later.

2.3 Signing in (donors and employees)

1. Click **Login** at the top right. The sign-in page opens (Figure 2).
2. Type your **email** and **password**.
3. Tick **Remember me?** only on your own device.
4. Click **Log in**.

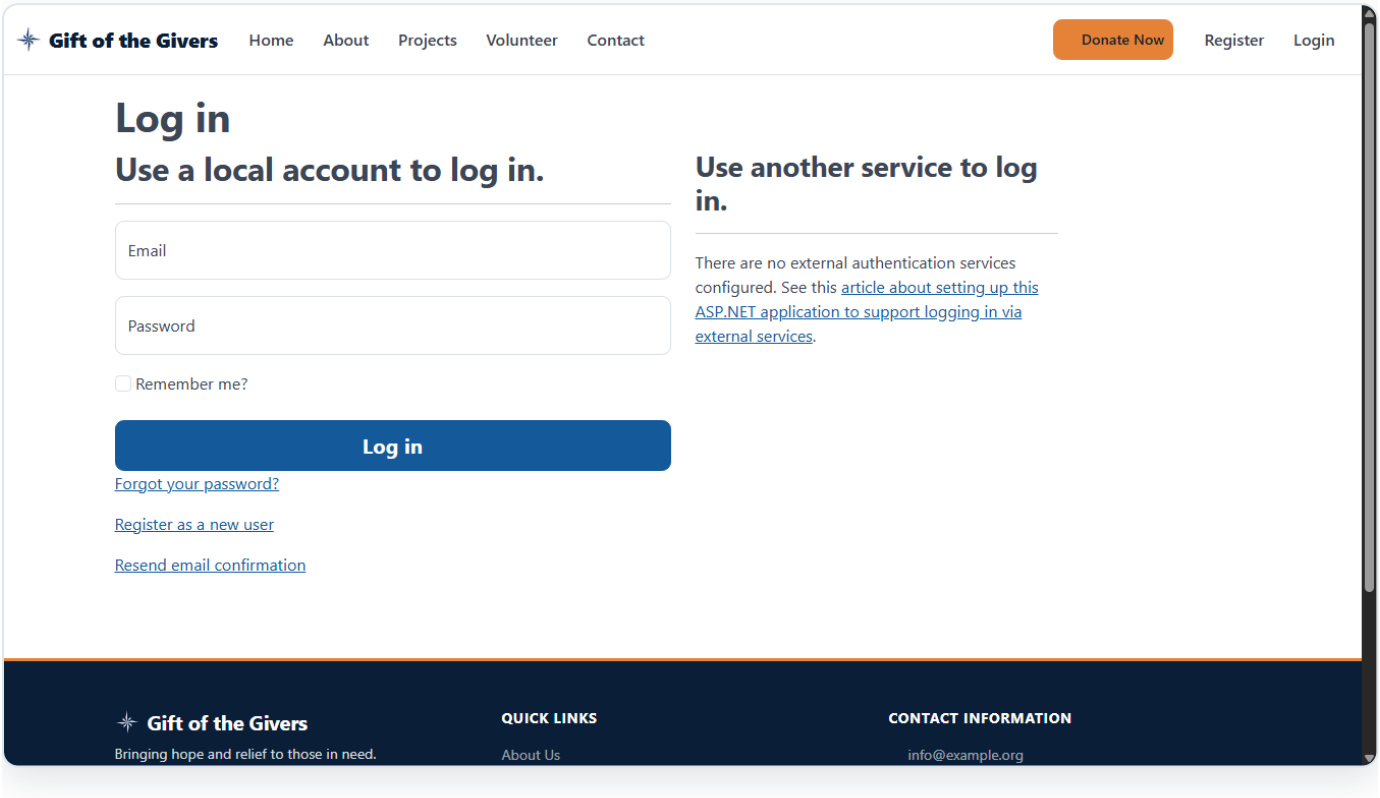


Figure 2 — The sign-in page

After signing in, your email appears at the top right with a **Logout** button.

- **Employees** see an extra menu item, **Employee Portal**. Choose **Dashboard** to reach the Employee Dashboard (Figure 3).
- **Donors** see **Donor Portal**, with **Dashboard**, **My Donations** and **Make Donation**.

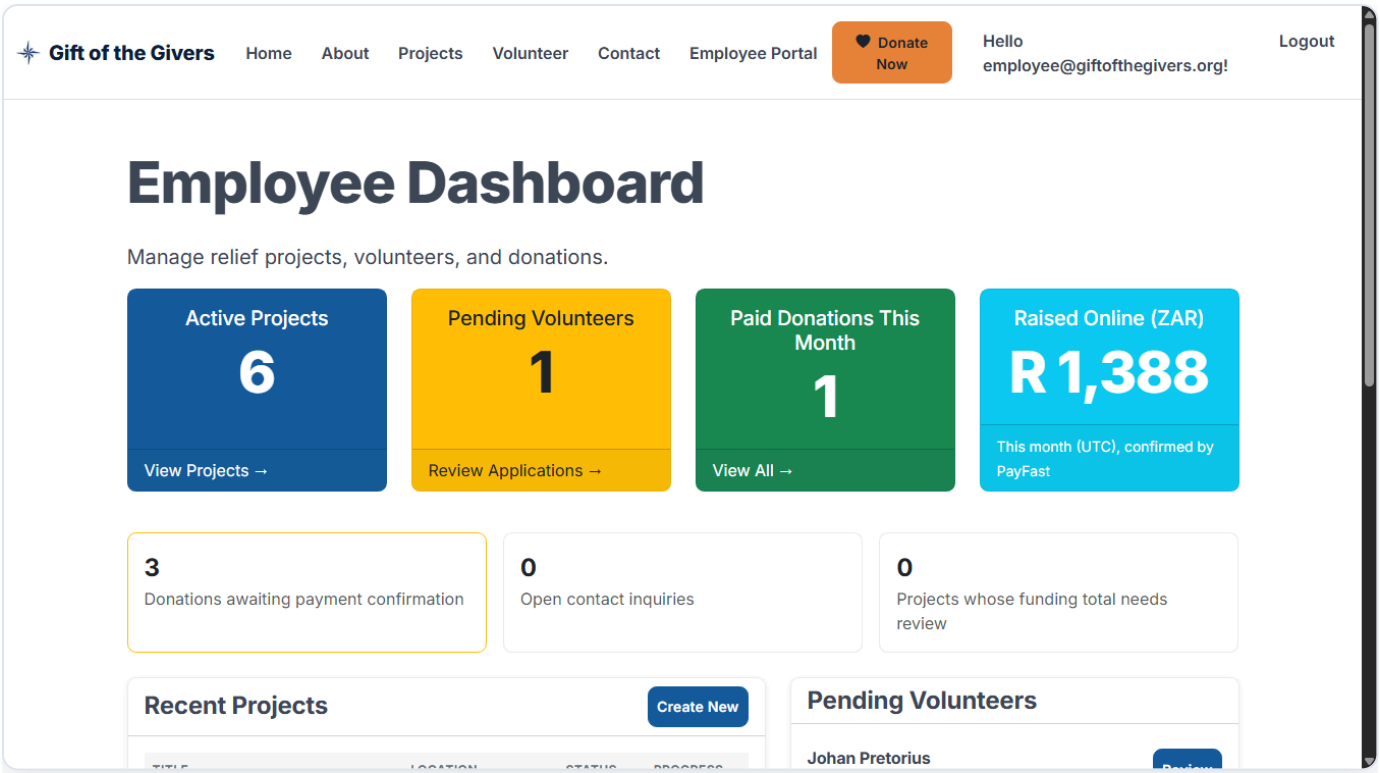


Figure 3 — Employee Dashboard after signing in as a staff member

2.4 If sign-in fails

If the email or password is wrong, the page says **Invalid login attempt** (Figure 4). Check:

- the email has no spaces;
- Caps Lock is off (passwords are case-sensitive).

After **5 wrong attempts** the account is locked for **15 minutes**. This protects it from people guessing passwords. Wait, then try again. If you have forgotten your password, ask your administrator to reset it.

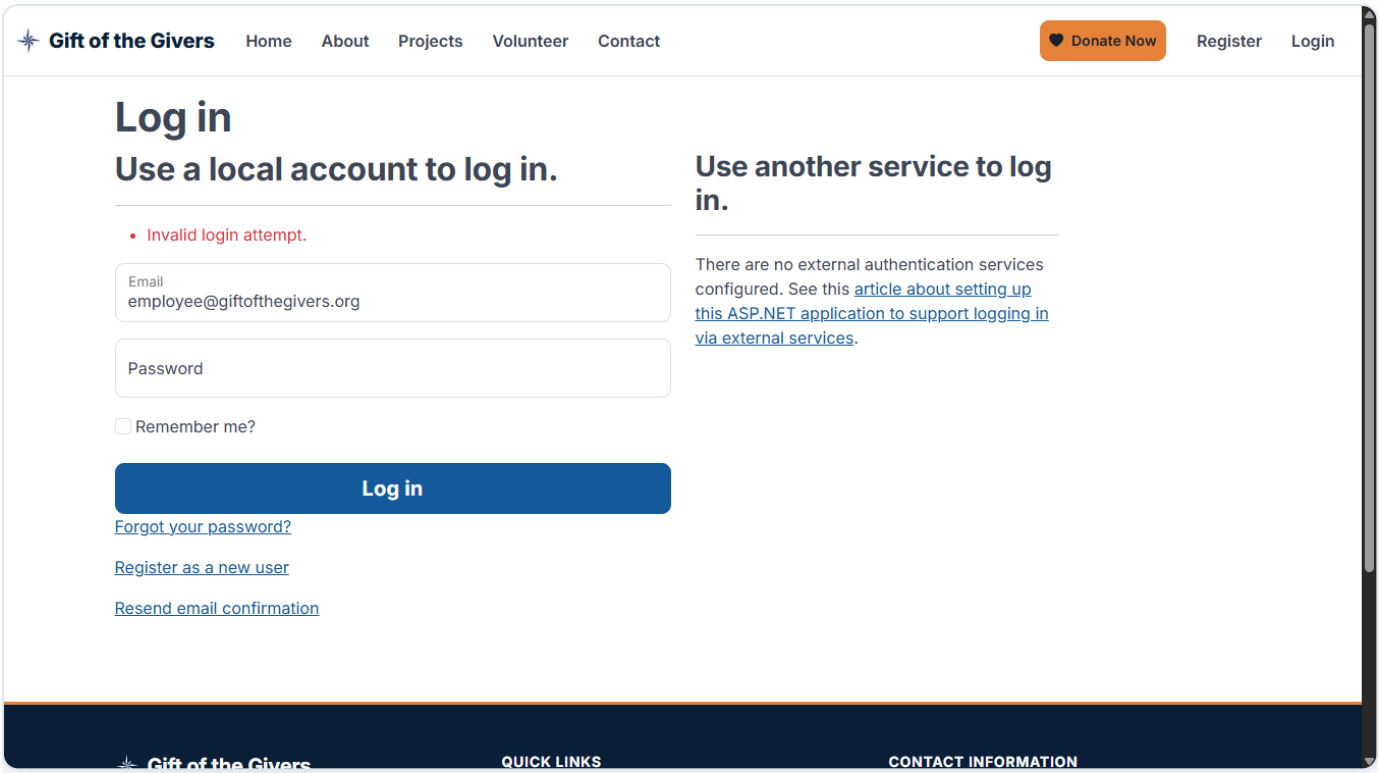


Figure 4 — Wrong password: the page shows "Invalid login attempt" and you stay signed out

2.5 Signing out

Click **Logout** at the top right. Always sign out on shared computers.

3. Getting staff (employee) access

For security, nobody can make themselves an employee.

1. The new staff member **registers** as in 2.2.
2. An existing employee opens **Employee Portal** → **Staff Access**, types the new person's email under *Grant staff access*, and clicks **Grant access**.
3. The new staff member signs out and in again. **Employee Portal** now appears.

To remove access when someone leaves, open **Staff Access** and click **Remove** next to their name, then confirm. Every grant and revoke is recorded in the **Audit Log**.

4. Employees: posting a project update

Updates tell donors what their money achieved: deliveries, progress, challenges. They appear on the public project pages.

1. Sign in as an employee (2.3).
2. Open **Employee Portal** → **Manage Projects**. The list of relief projects appears (Figure 5).
3. Find the project and click **Add Update**.

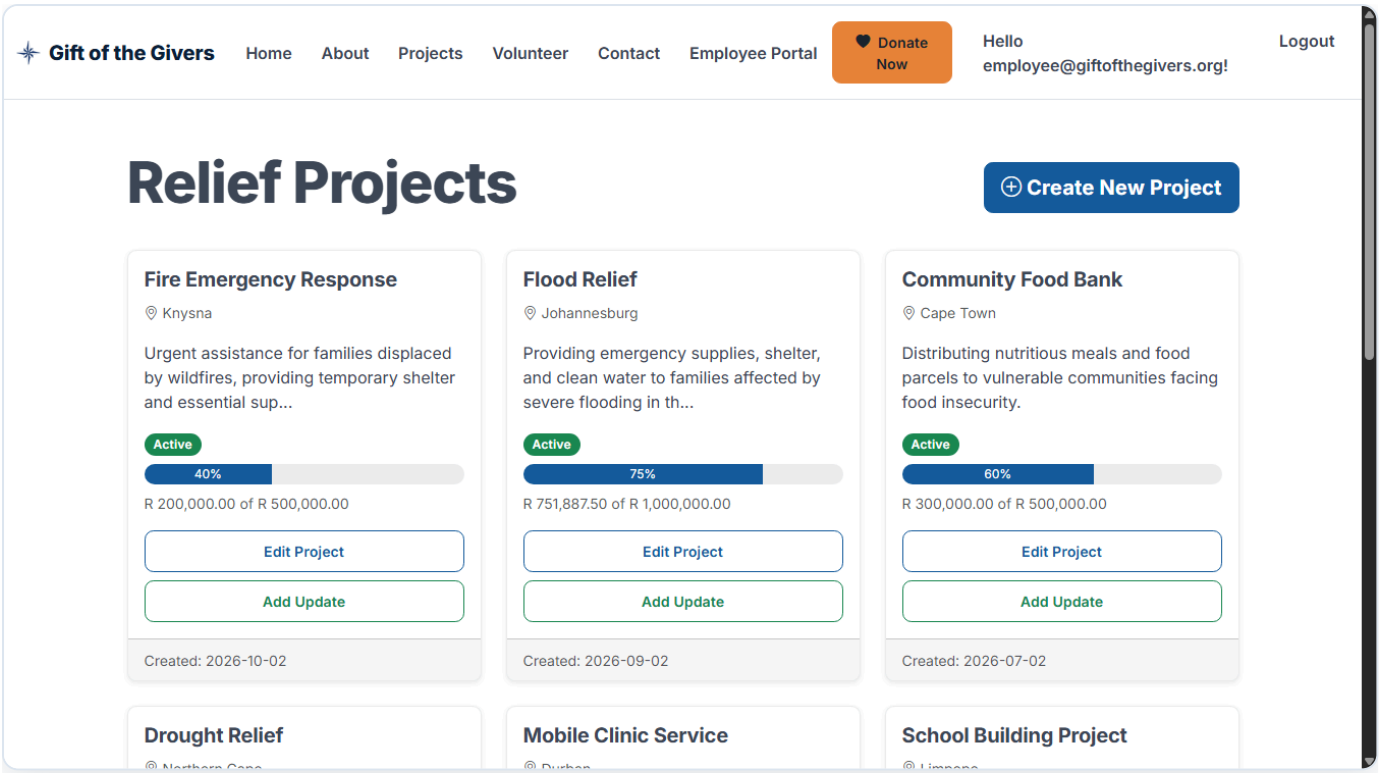


Figure 5 — Relief Projects list for staff. Each project has an Add Update button.

4. Fill in the form (Figure 6):
   - **Title**: a short headline, for example *Day 3: 400 food parcels delivered*.
   - **Content**: what happened, in plain words (up to 4,000 characters).
   - **Image URL** (optional): the web address of a photo already published online. Leave it empty if you have none.
5. Click **Publish Update**.

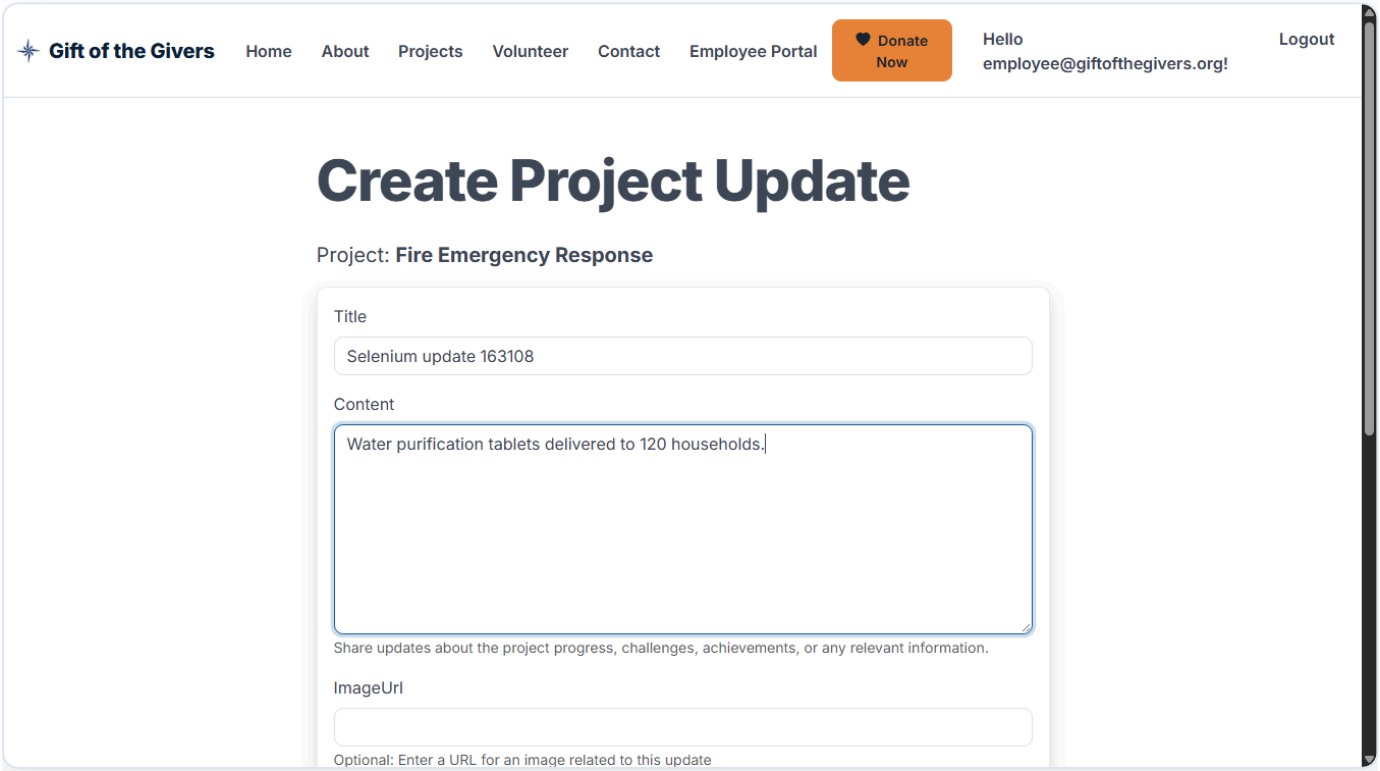


Figure 6 — Writing a project update

6. You return to the project list with the green message **Project update created successfully** (Figure 7). The update is now visible to the public.

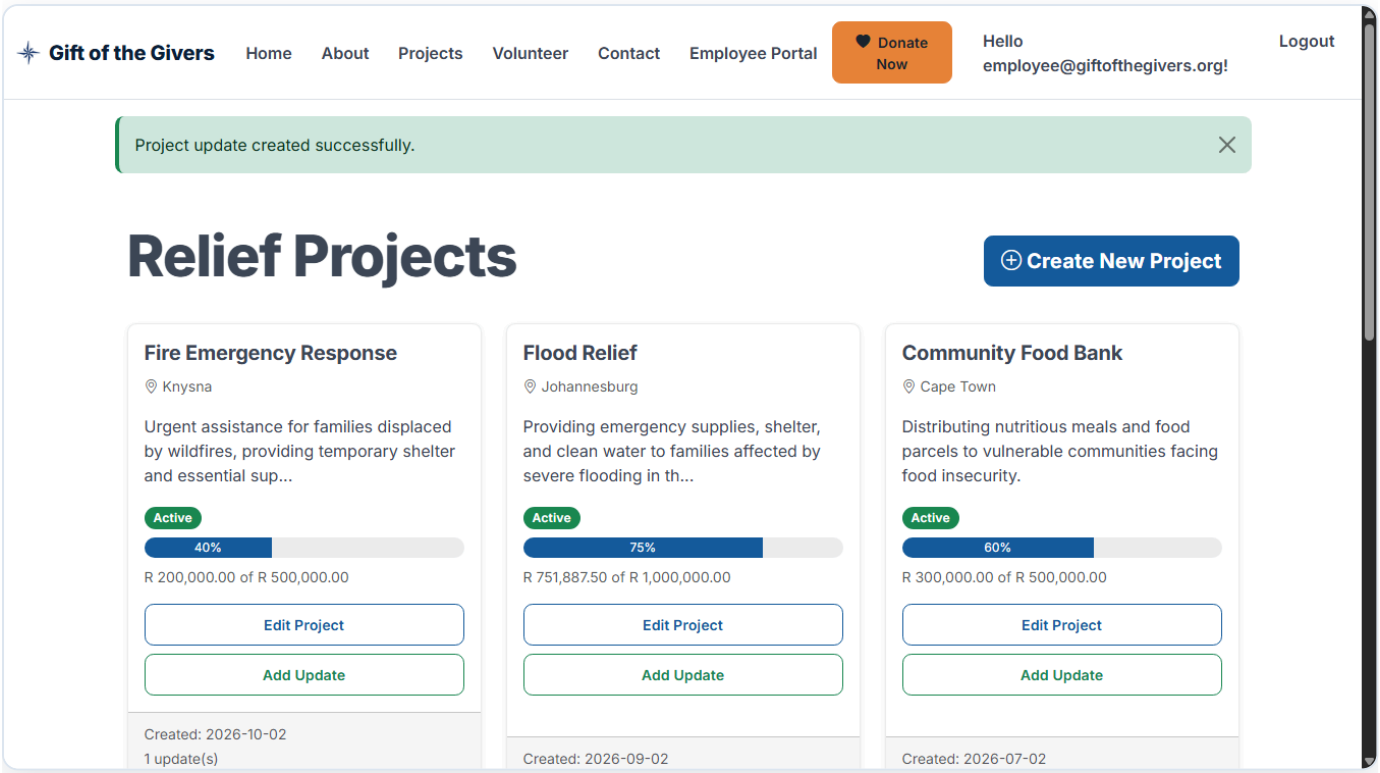


Figure 7 — Confirmation that the update was published

To correct an update after publishing, ask the system administrator; editing published updates is not yet available from the menu.

If the form shows red messages: a required field (Title or Content) is empty or too long. Fix it and click **Publish Update** again. Nothing is saved until the form is accepted.

5. Making a donation

Anyone can donate, with or without an account.

5.1 Step by step

1. Click the orange **Donate Now** button (top of every page), or **Donate** on a project.
2. Choose the **donation type** (Figure 8):
   - **One-time donation**: a single gift.
   - **Monthly pledge** (recurring): records that you intend to give every month. Automatic monthly billing is not yet available, so each month's payment is made separately with the same form.
3. Choose a **Project**, or leave **General Fund** so the Foundation can use the money where it is most needed.
4. Choose the **Currency** (see 5.2) and enter the **Amount**, or click one of the quick-amount buttons.
5. Enter your **Name** and **Email**. The receipt is sent to this email.
6. Choose a **Preferred payment method**: *Credit/Debit Card* or *EFT*.
7. To give **anonymously**, tick **Make donation anonymous** (see 5.3).
8. Optionally add a **Message**.
9. Tick **I have read the privacy notice...**. This is required.

10. Click **Continue to payment** (on the demonstration site the button reads **Record donation**) (Figure 9).

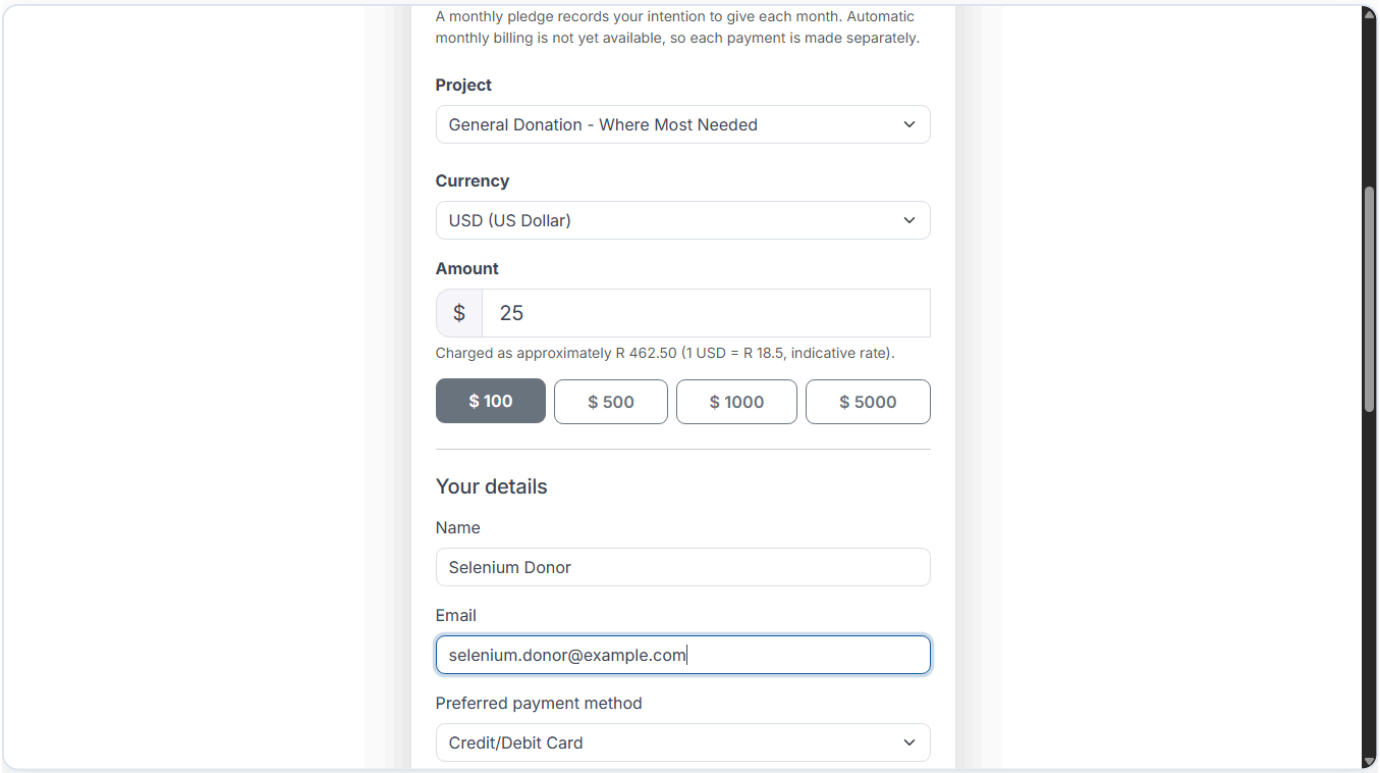


Figure 8 — Top of the donation form: donation type, project, currency and amount

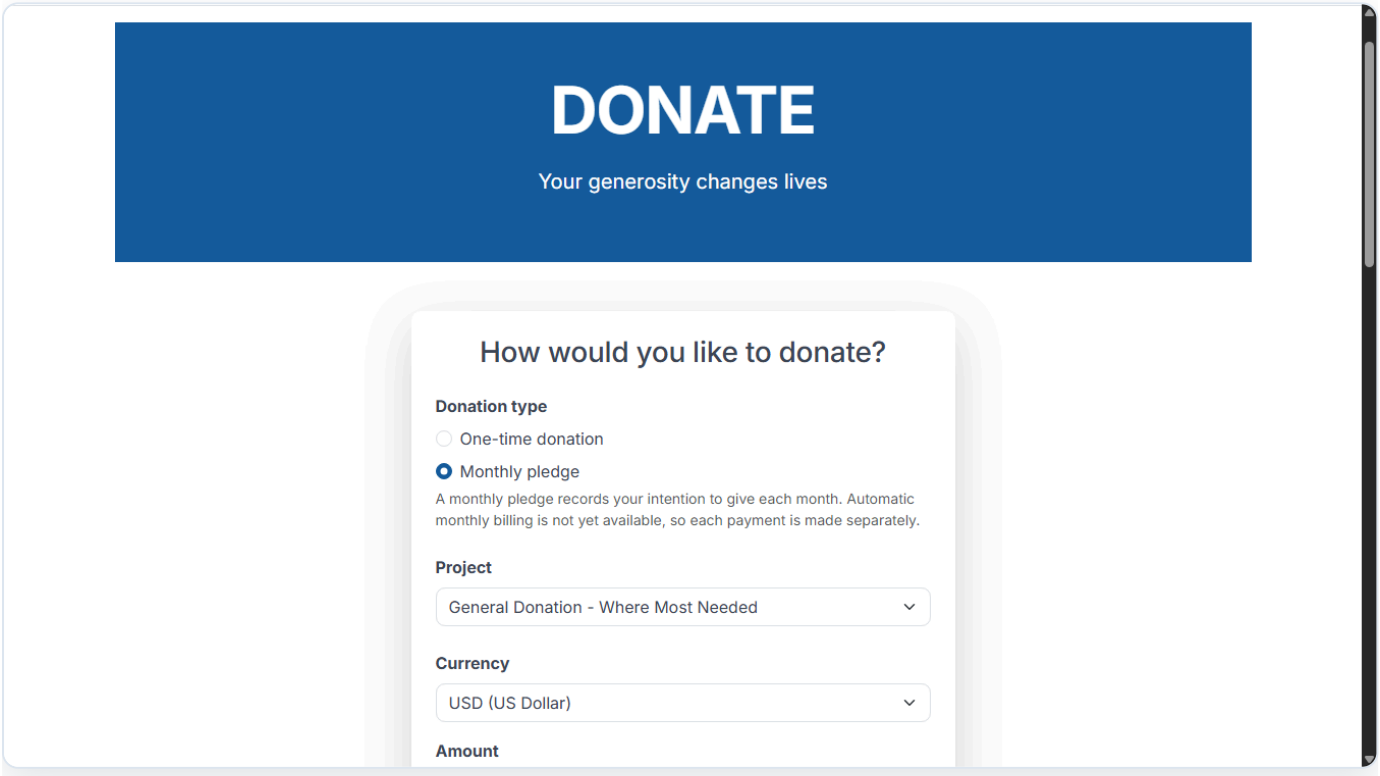


Figure 9 — Bottom of the donation form: anonymous option, privacy consent and the Donate button

11. When payments are live, you are taken to **PayFast**, South Africa's secure payment page, to pay by card or EFT. The Foundation never sees your card details.
12. You return to the **Thank You** receipt page (Figure 10). It shows the amount, the rand amount charged, the type, the project, the **status** and your **reference number**. Keep the reference number.

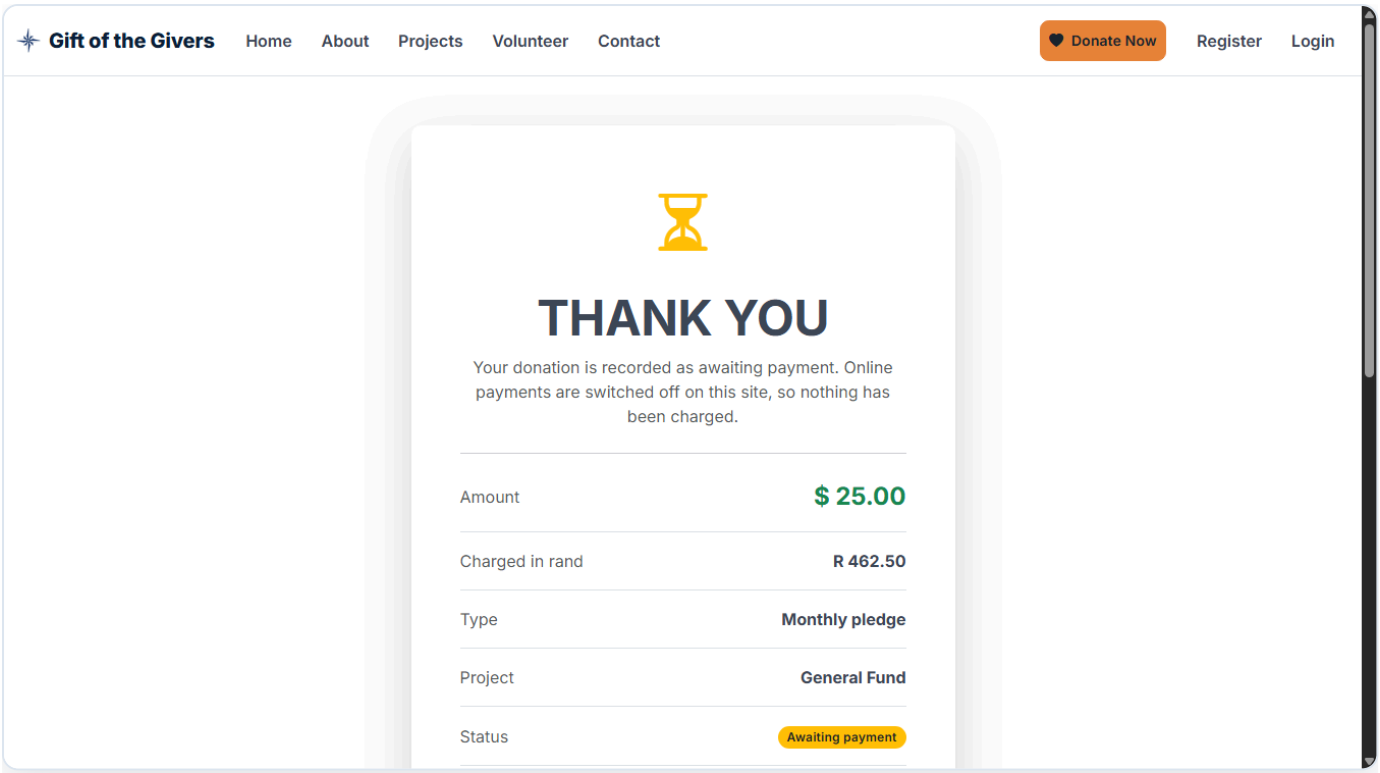


Figure 10 — Receipt page for an anonymous US-dollar monthly pledge. Status "Awaiting payment" changes to "Paid" when PayFast confirms the payment.

Payment status explained

Status	Meaning
Awaiting payment	The donation is recorded; the payment is not yet confirmed.
Paid	PayFast confirmed the payment. It now counts towards the project total and a tax certificate is available.
Failed	The payment was cancelled or declined. Nothing was charged. Click **Try again**.

Clicking the button twice does not create two donations. The site recognises the same form.

5.2 Giving in another currency

You can give in **South African rand (ZAR)**, **US dollars (USD)** or **euro (EUR)**.

- When you choose USD or EUR and type an amount, the form shows the rand equivalent, for example *Charged as approximately R 462.50 (1 USD = R 18.50)*.
- PayFast charges in rand, so the rand amount is what your bank charges. Your bank may add a small foreign-transaction fee.
- The receipt shows both your amount (e.g. **$ 25.00**) and the **Charged in rand** amount (Figure 10).

5.3 Giving anonymously

Tick **Make donation anonymous**. Your name is then hidden from staff reports and printed as *Anonymous Donor* on the tax certificate. The Foundation still keeps your email privately, so it can send your receipt.

You can also give **without an account**, as an anonymous visitor. In that case keep your reference number, because you will need it to request a tax certificate (6.3).

5.4 If the form shows errors

Red messages appear under any field that needs attention (Figure 11). Common ones:

Message	What to do
Amount must be between 1 and 1,000,000	Enter an amount of at least 1.
The Your Email field is not a valid e-mail address	Check the email for typos.
You must accept the privacy notice	Tick the privacy box.
This project is not accepting donations	The project has closed. Choose another project or the General Fund.

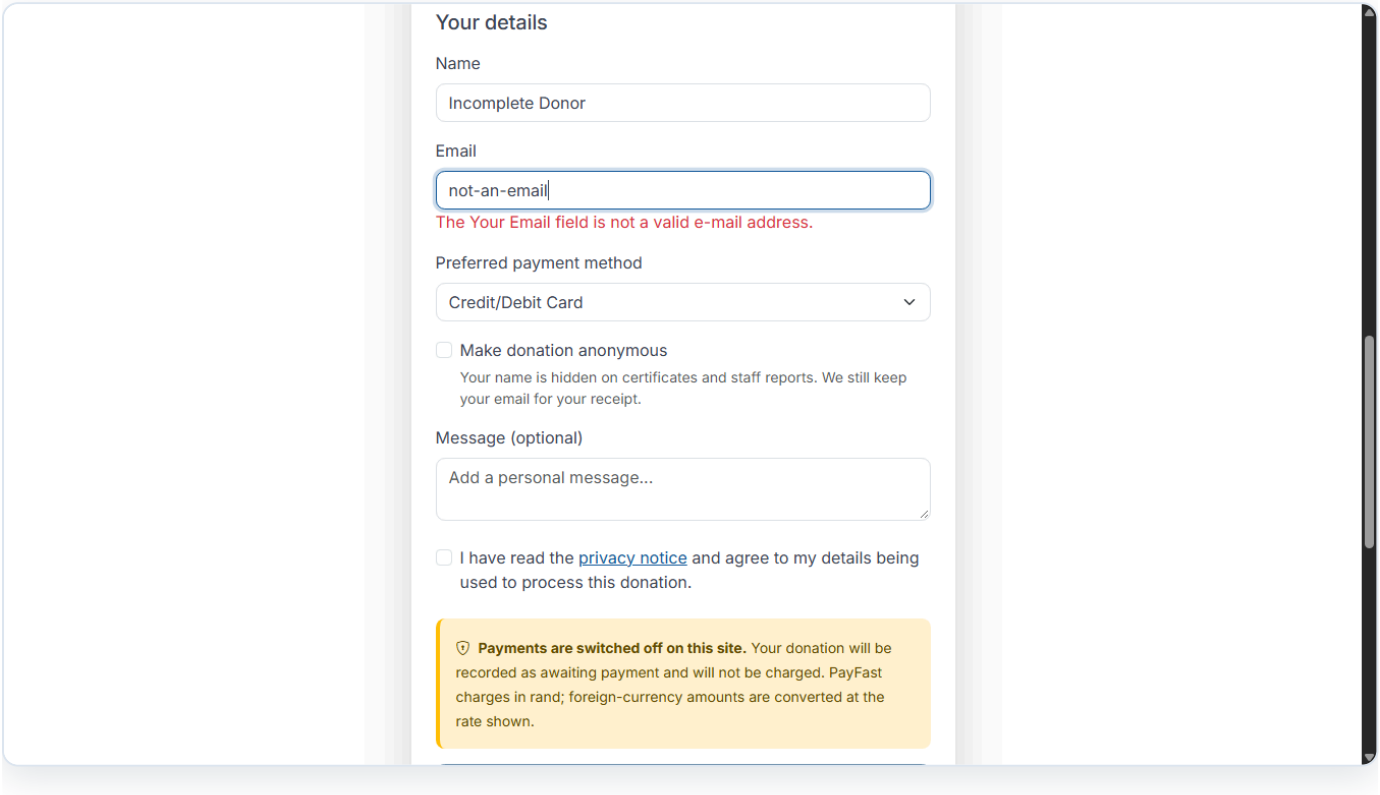


Figure 11 — The form will not submit until the highlighted fields are corrected

6. Getting a tax certificate (Section 18A)

A tax certificate is available for every **paid** donation. It is a PDF that you can save or print.

6.1 Donors with an account

1. Sign in (2.3) and open **Donor Portal** → **Dashboard** (Figure 12). It shows your confirmed donations and total given.

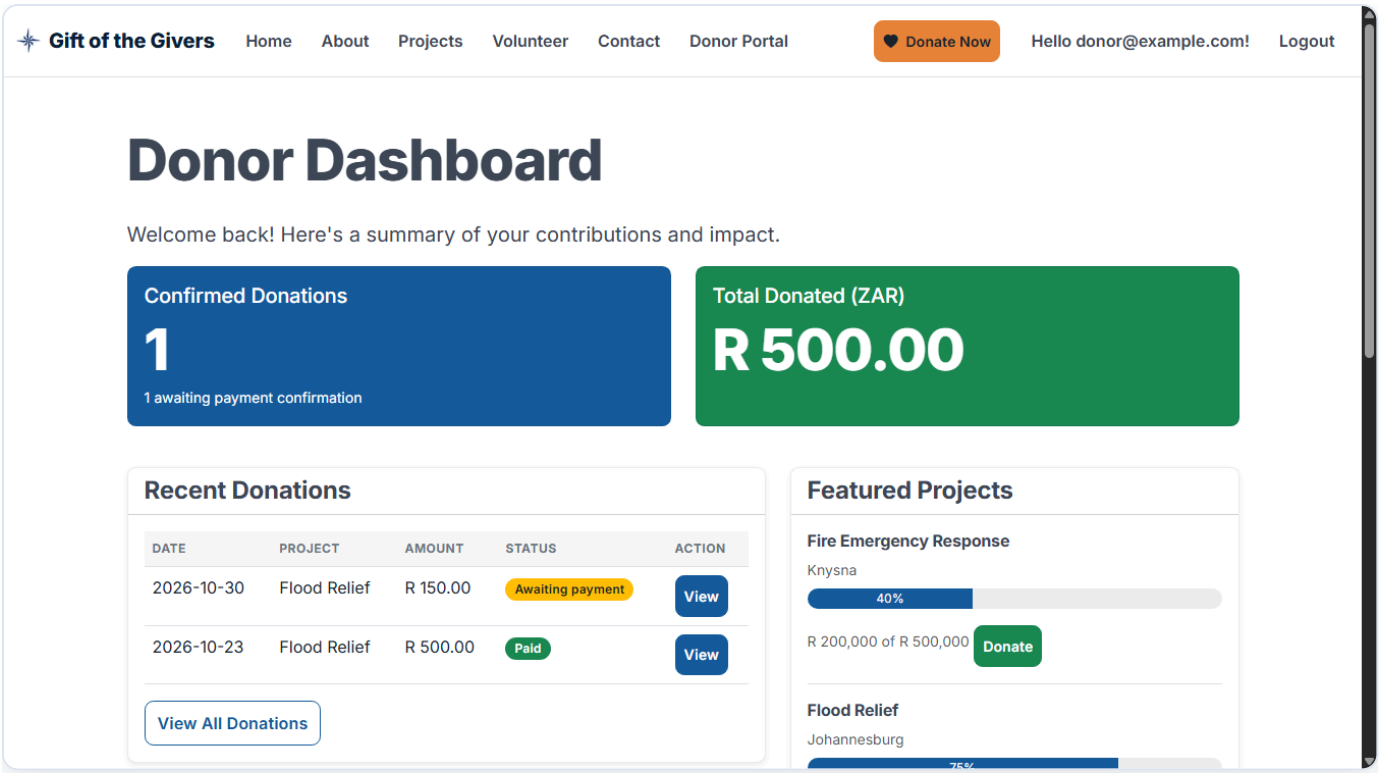


Figure 12 — Donor Dashboard

2. Open **Donor Portal** → **My Donations** (Figure 13). Each paid donation has a **Certificate** button.

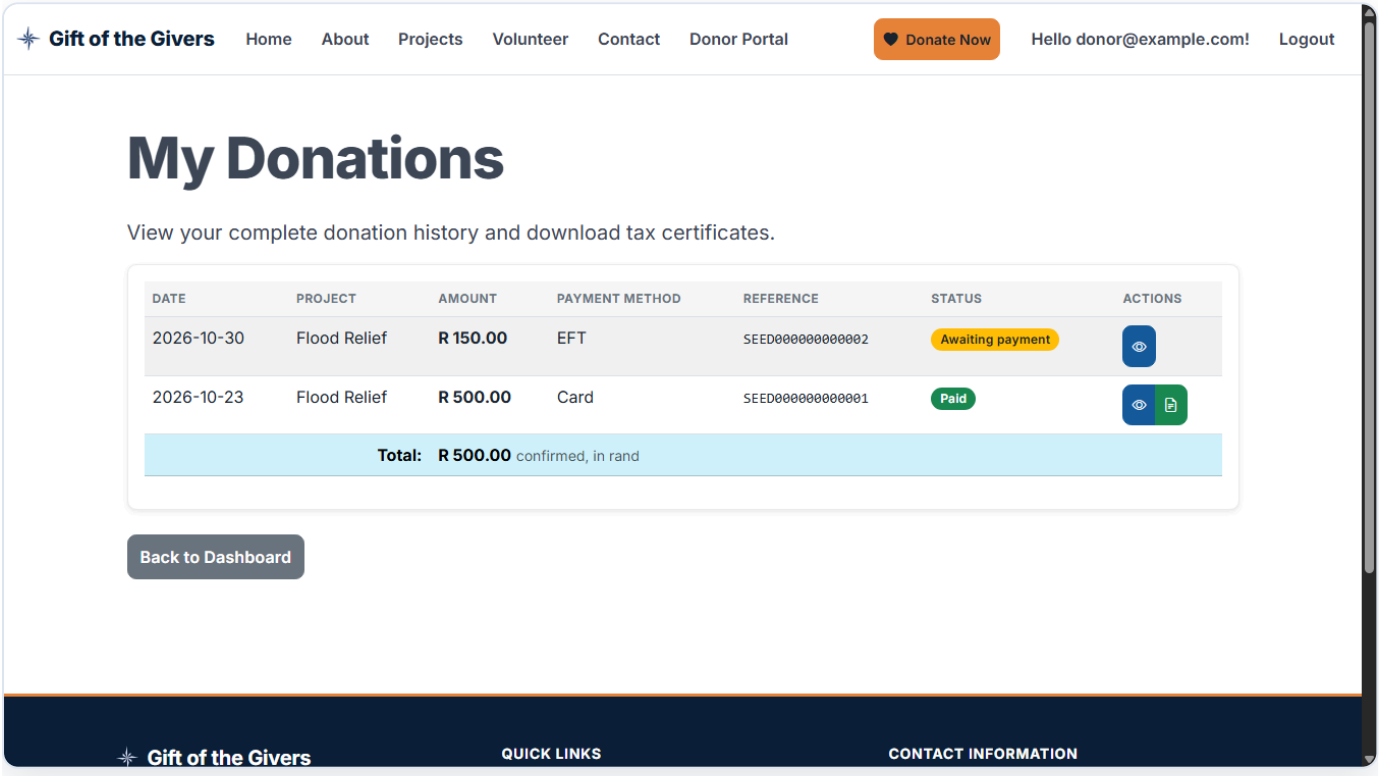


Figure 13 — My Donations: the Certificate button appears for paid donations only

3. Click **Certificate**. The certificate opens on screen (Figure 14).
4. Click **Download PDF** to save it, then print it or attach it to your tax return.

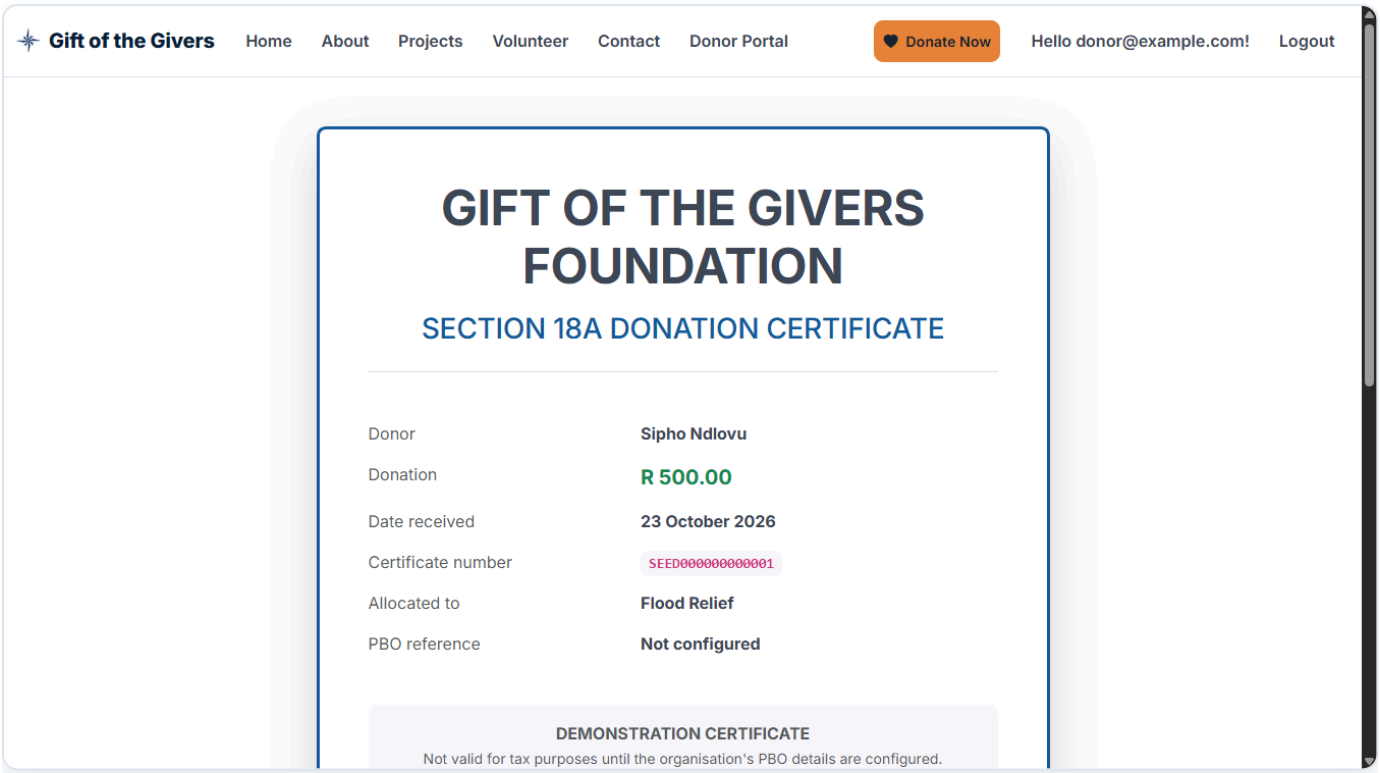


Figure 14 — The tax certificate on screen, with the Download PDF button

No Certificate button? The donation is still *Awaiting payment*, or it failed. Certificates appear automatically once PayFast confirms the payment, usually within minutes.

6.2 Straight after donating

If you were signed in when you donated, the receipt page shows **View Tax Certificate** as soon as the payment is confirmed.

6.3 Donors without an account, and staff issuing certificates

If the donor gave without an account, they contact the Foundation with their **reference number**. A staff member then:

1. Signs in and opens **Employee Portal → Donations**.
2. Clicks the **Paid** filter and finds the donation by the reference number or date.
3. Clicks **Certificate → Download PDF**, and emails the PDF to the donor.

Each time a certificate is viewed or downloaded, this is recorded in the Audit Log.

> **Demonstration certificates** are clearly marked *DEMONSTRATION — not a valid Section 18A certificate* until the Foundation's PBO registration number is entered by the administrator.

7. Registering as a volunteer

1. Click **Volunteer** in the top menu.
2. Fill in the form (Figure 15):
   - **Full name** and **email**: required.
   - **Phone number** and **city**: optional, but they help the team contact you.
   - **Skills**: for example *first aid, driving, cooking, logistics*. Required.

- **Availability**: for example *Weekends* or *Weekdays after 17:00*. Required.
- Date of birth and emergency contact: optional. Volunteers must be at least 16.

3. Tick the **privacy notice** box.
4. Click **Register interest**.

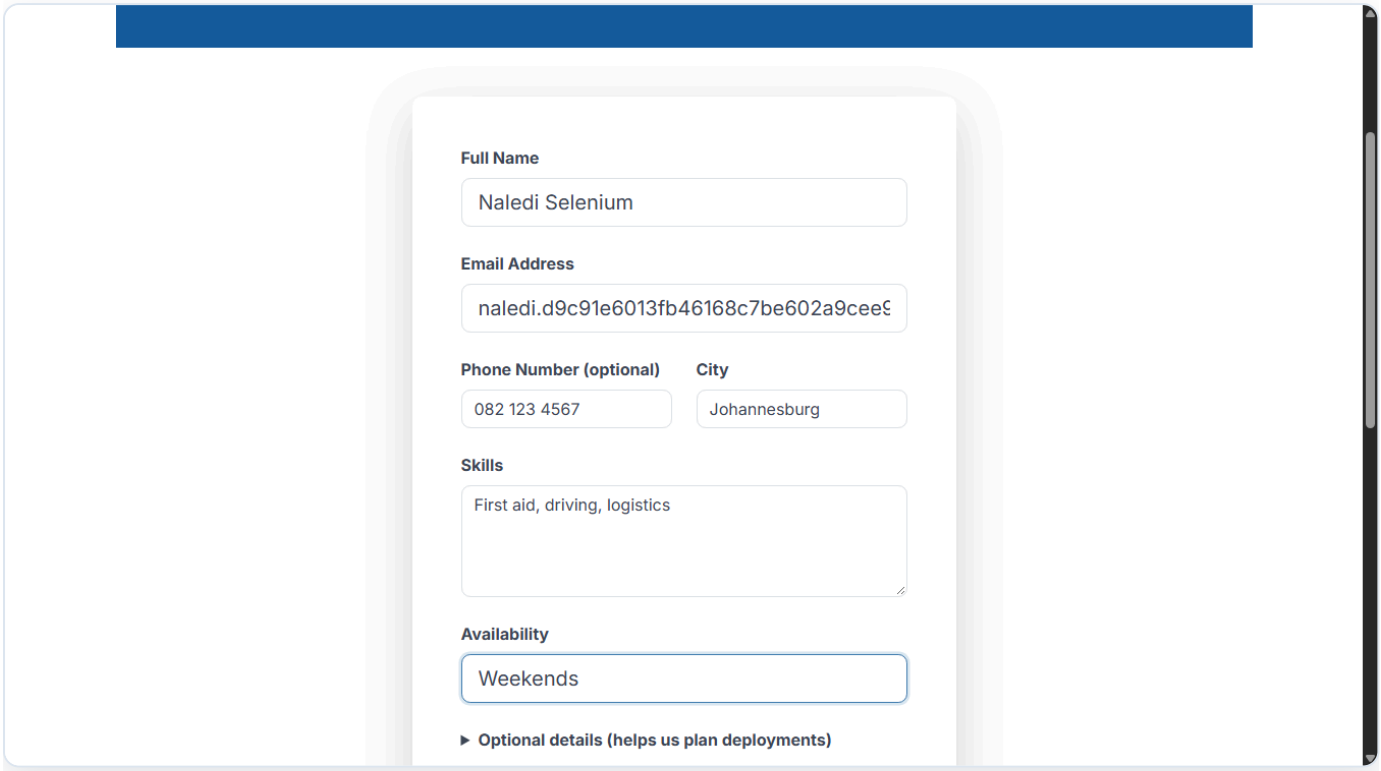


Figure 15 — The volunteer application form

5. The **Thank you for volunteering** page confirms your application (Figure 16). A coordinator reviews it and contacts you by email.

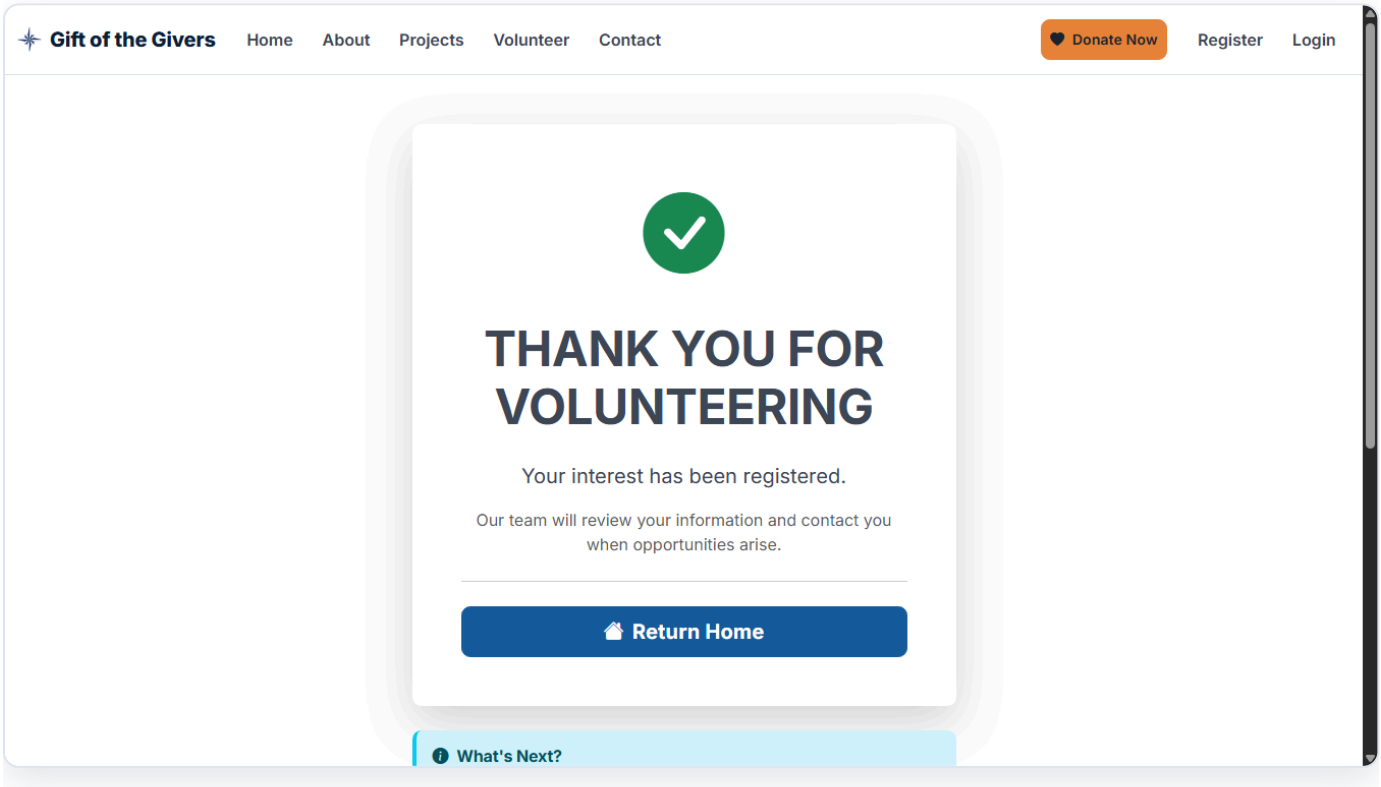


Figure 16 — Confirmation after applying

If required fields are missing, red messages appear under them and nothing is sent. Complete them and submit again.

For staff: new applications appear under **Employee Portal → Manage Volunteers** (and in *Pending Volunteers* on the dashboard). Open one, then click **Approve** or **Reject**. Approved volunteers can be **assigned** to active projects from the same page.

8. Accessibility and good practice

- The site works with a keyboard alone: **Tab** moves between fields and **Enter** presses a button. Screen readers announce form errors and the currency conversion.
- Text can be enlarged with **Ctrl +** (Windows) or ⌘ **+** (Mac) without breaking the layout. The site also adapts to phone screens (Figure 17).
- Colours are never the only signal: every status also has a word (*Paid*, *Awaiting payment*, *Failed*).
- Never share your password. Staff should sign out of shared computers and never email donor lists. Use **Export CSV** only on Foundation devices.

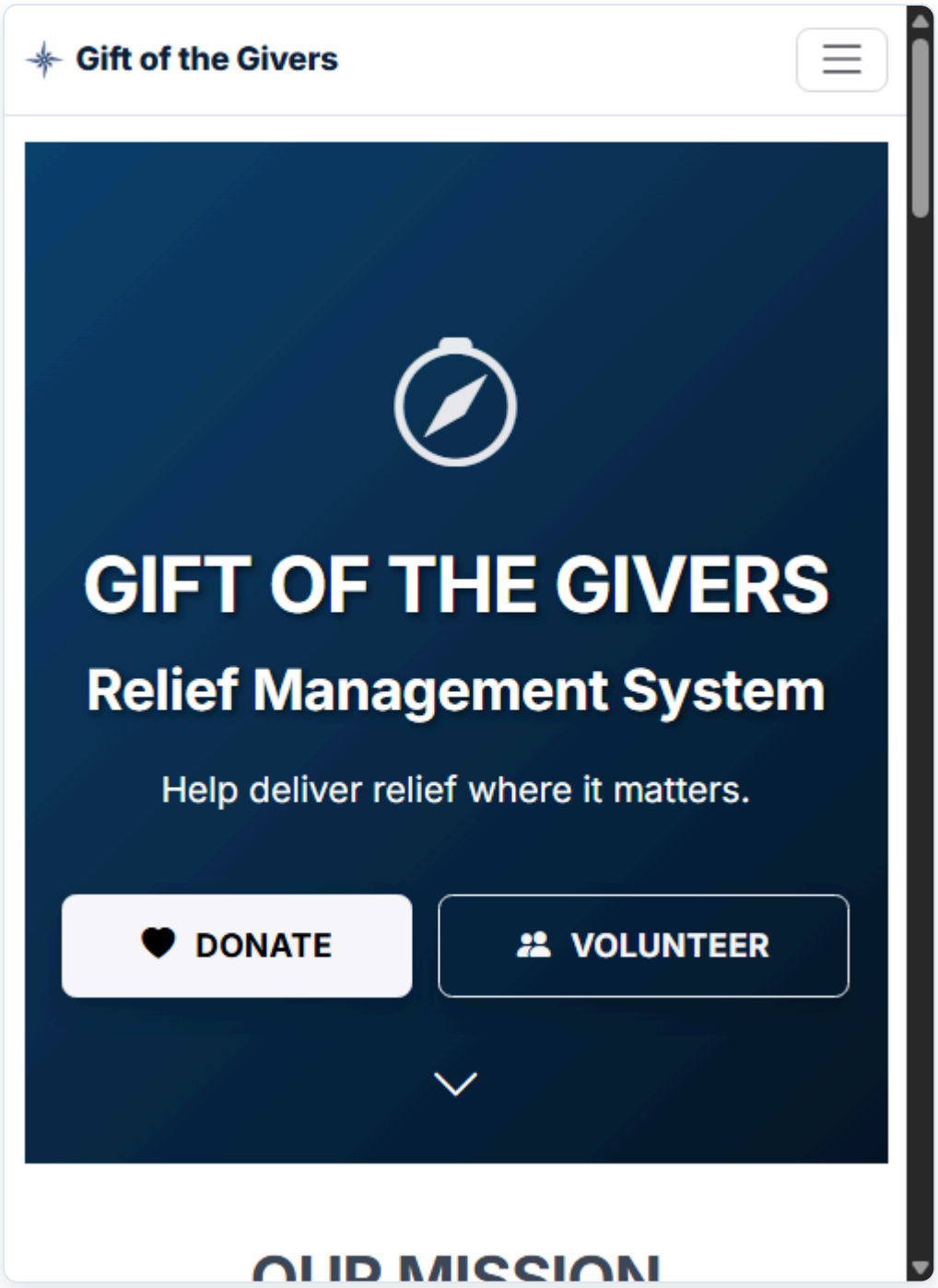


Figure 17 — The same home page on a phone. The menu is behind the ☰ button.

9. Frequently asked questions

Question	Answer
I donated but the status still says *Awaiting payment*.	PayFast may take a few minutes to confirm. Refresh the receipt page later. If it still says so after a day, contact the Foundation with your reference number.
Can I change or cancel a donation?	Not on the website. Contact the Foundation with your reference number.
Will a monthly pledge charge me automatically?	Not yet. It records your intention; you donate each month with the form.
I gave in dollars. Why does my bank show rand?	PayFast settles in rand. The receipt shows the rand amount charged.
Is my name shown publicly?	No. Donor names are never shown publicly. *Anonymous* also hides your name from staff reports and certificates.
I am an employee but cannot see Employee Portal.	Ask another employee to grant access (section 3), then sign out and in again.
A form is refused with error 429 (*too many requests*).	More than 20 forms were sent from your connection in one minute. Wait one minute and try again.

10. Getting help

- **Donors and volunteers:** use the **Contact** page. A staff member replies by email.
- **Staff:** questions arriving through the Contact page are listed under **Employee Portal** → **Inquiries**. Mark each as handled once answered.
- **Technical problems** (site down, errors): contact the system administrator named in the hand-over pack. Include the time, the page address and a screenshot.

Gift of the Givers · Relief Management System

DELIVERABLE 11

Azure Deployment and GitHub Sync Guide

Step-by-step instructions for the team member who sets up Azure, connects GitHub to Azure DevOps and turns on continuous deployment.

DOCUMENT	VERSION	DATE	STATUS
D-11	1.0	2026-11-02	Ready to follow — no step has been carried out in Azure yet

CONTENTS

1. What you will end up with
2. Before you start: access tokens
3. Create the Azure resources
4. Prepare Azure DevOps
5. Sync GitHub to Azure DevOps
6. Create the pipeline
7. What each deployment stage does
8. After the first deployment
9. Rollback and restore
10. Troubleshooting
11. Hand-over checklist
12. Costs and clean-up

1. What you will end up with

When you finish this guide:

- Every push to the **GitHub** repository is copied automatically to **Azure Repos** (Azure DevOps).
- Each copy starts the **Azure Pipeline** (`azure-pipelines.yml`). It builds the solution, runs the unit, integration and Selenium tests, publishes test and coverage reports, then on `main` it:
  1. applies database migrations to **Azure SQL**,
  2. deploys the web app to **App Service** (through a staging slot if your plan has one),
  3. deploys the tax-certificate **Azure Function**, and
  4. runs read-only Selenium smoke tests against the live site.
- The live site at `https://gift-relief-managemen-gdbsf9bbena9epgw.southafricanorth-01.azurewebsites.net` shows the application instead of the Azure "waiting for your content" page.

The path of one change: developer pushes to GitHub → GitHub Action *Mirror to Azure DevOps* → Azure Repos → Azure Pipeline *Build and test* → (approval) → *DeployDatabase* → *DeployWebApp* and *DeployFunctions* → *LiveSmokeTests*.

Time needed: about 2 hours the first time. **You need:** a browser, Owner or Contributor access to the team's Azure subscription, Project Administrator access to the Azure DevOps organisation `ST10438928` , and Admin access to the GitHub repository `jBlack-MC/GiftOfTheGivers-HumanitarianAid` . Installing nothing locally is fine; the Azure CLI is optional.

> **Write things down as you go.** Section 11 is a checklist of every name, URL and secret you create. Keep it in a password manager, never in the repository, a chat or a document.

2. Before you start: access tokens

Personal access tokens (PATs) are passwords. Rules for this project:

- **A PAT that has been pasted into a chat, email or document is compromised.** Revoke it (Azure DevOps → your avatar → *Personal access tokens* → *Revoke*) and create a new one. A token was shared in this way during Part 3; when tested on 2026-11-02 it was already rejected by Azure DevOps (HTTP 401), but revoke it anyway if it still appears in the list.
- Create each PAT with the **smallest scope** and an expiry date at the end of the module.
- Store PATs only as **GitHub repository secrets** or **secret pipeline variables**.

You will create two PATs in section 5:

PAT	Scope	Stored as
Repository mirror	*Code: Read & write*	GitHub secret `AZURE_DEVOPS_PAT`
Helper package publishing (Part 2)	*Packaging: Read & write*	GitHub secret `AZURE_ARTIFACTS_PAT`

3. Create the Azure resources

Use the Azure portal (). Use **South Africa North** for every resource, because the existing web app is already there. The names below are the defaults in `pipelines/templates/variables.yml`. If a name is taken, choose another and write it down; you will change the file in section 7.

3.1 Resource group

1. Search **Resource groups** → **Create**.
2. Subscription: the team subscription. Name: `rg-giftofthegivers`. Region: *South Africa North*.
3. **Review + create** → **Create**.

> The existing web app may already be in a different resource group. Open it (search **App Services** → `gift-relief-management…`) and look at *Resource group* on the Overview page. If it is different, either use that group everywhere below or move the app (*Overview* → *Move* → *Move to another resource group*).

3.2 Azure SQL server and database

1. Search **SQL databases** → **Create**.
2. Resource group `rg-giftofthegivers`. Database name `GiftOfTheGiversDb`.
3. Server → **Create new**: name `gift-relief-sql`, location *South Africa North*, authentication **Use SQL authentication**. Admin login, for example `gotgadmin`, and a strong password. **Write both down.**
4. *Want to use SQL elastic pool?* **No**. *Workload environment*: **Development**.
5. **Compute + storage** → **Configure database**: choose **Basic (5 DTU, 2 GB)**, or the *free serverless offer* if the subscription shows it. Both fit the module's B1 limits.

6. *Backup storage redundancy*: **Locally-redundant** (cheapest).
7. **Networking** tab: connectivity **Public endpoint**. Turn on **Allow Azure services and resources to access this server** (the web app, function and pipeline need it). Optionally also *Add current client IP address* so you can query from your PC.
8. **Review + create** → **Create**. Wait about 5 minutes.
9. Open the database → **Settings** → **Connection strings** → **ADO.NET (SQL authentication)**. Copy it and replace `{your_password}` with the real password. This is `sqlConnectionString`. It looks like:

```
Server=tcp:gift-relief-sql.database.windows.net,1433;Initial Catalog=GiftOfTheGiversDb;Persist Security Info=False;User ID=gotgadm
```

You do **not** need to create any tables. The pipeline creates and updates the schema.

3.3 Web app (App Service)

The web app already exists: `gift-relief-management…` on **Windows** (it currently shows the Azure placeholder page).

1. Open it. On **Overview** note the **Name**, the **Resource group**, the **Default domain** (the long `…azurewebsites.net` address) and the **App Service plan** pricing tier.
2. **Settings** → **Configuration** → **General settings** (or *Environment variables* in the newer portal):
   - *Stack*: **.NET**, version **.NET 10 (LTS)**.
   - *Platform*: **64 Bit**. *HTTPS Only*: **On**. *Always on*: On if the tier allows it (B1 and above).
   - **Save**.
3. **Health check** (under *Monitoring*): enable it with path `/health`. App Service then restarts instances that stop answering.
4. **Deployment slots** (only if the plan is **Standard S1 or higher**): **Add slot** → name `staging`, clone settings from production. Copy the slot's URL; it is `webAppStagingUrl`. On Free or Basic plans, skip this step and leave `useStagingSlot: 'false'`.

The pipeline sets all application settings itself (connection string, `Database__MigrateOnStartup=false`, and so on). You do not need to type them in the portal.

3.4 Function app

1. Search **Function App** → **Create** → hosting plan **Consumption** (Windows).
2. Resource group `rg-giftofthegivers`. Name `gift-relief-functions` (must be unique; write down what you use).
3. *Runtime stack* **.NET**, *Version* **10 (LTS), isolated worker model**, *Region* South Africa North, *Operating system* **Windows**.
4. **Storage**: let it create a new storage account. **Monitoring**: Application Insights **Yes** (gives you logs for the report).
5. **Review + create** → **Create**.

You will copy the function key **after the first deployment** (section 8).

3.5 (Optional) Azure CLI equivalent

If you prefer the command line (`az login` first), this script creates the same resources. Replace the password placeholder, and pick unique names if needed.

```
RG=rg-giftofthegivers; LOC=southafricanorth
az group create -n $RG -l $LOC
az sql server create -g $RG -n gift-relief-sql -l $LOC -u gotgadmin -p '<strong-password>'
az sql server firewall-rule create -g $RG -s gift-relief-sql -n AllowAzure --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
az sql db create -g $RG -s gift-relief-sql -n GiftOfTheGiversDb --service-objective Basic --backup-storage-redundancy Local
az storage account create -g $RG -n giftrelieffuncstore -l $LOC --sku Standard_LRS
az functionapp create -g $RG -n gift-relief-functions --consumption-plan-location $LOC \
  --runtime dotnet-isolated --runtime-version 10 --functions-version 4 --os-type Windows \
  --storage-account giftrelieffuncstore
```

4. Prepare Azure DevOps

Open and the team project (the one that holds the `giftgivers-helpers` feed from Part 2).

4.1 Azure Repos repository

1. **Repos → Files**. If the project has no repository named `GiftOfTheGivers-HumanitarianAid`, use the repository drop-down → **New repository**: Git, that name, **no** README.
2. Click **Clone** and copy the HTTPS URL, e.g. `https://ST10438928@dev.azure.com/ST10438928/<Project>/_git/GiftOfTheGivers-HumanitarianAid`. Remove the `ST10438928@` part. This is `AZURE_DEVOPS_REPO_URL`.

4.2 Service connection to Azure

1. **Project settings → Service connections → New service connection → Azure Resource Manager**.
2. *Identity type*: **App registration (automatic)**. *Credential*: **Workload identity federation**.
3. *Scope level*: **Subscription**; pick the subscription and resource group `rg-giftofthegivers`.
4. *Service connection name*: `giftgivers-azure-connection`. Tick **Grant access permission to all pipelines** (or authorise it on the first run).
5. **Save**.

If your student account cannot create app registrations, ask the subscription owner to create the connection, or use *Identity type: Managed identity* if offered.

4.3 Environment with an approval

1. **Pipelines → Environments → New environment** → name `giftgivers-production`, resource **None** → **Create**.
2. ⋮ → **Approvals and checks → Approvals** → add the deployment lead (and a second member if you want four-eyes). Every production deployment now waits for that approval. This is your release gate.

4.4 Variable group with the secrets

1. **Pipelines → Library → + Variable group**. Name: `giftgivers-azure` (exact spelling).
2. Add these variables. Click the **lock** icon for every row marked secret:

Variable	Value	Secret
`sqlAdminUser`	the SQL admin login (e.g. `gotgadmin`)	yes
`sqlAdminPassword`	the SQL admin password	yes
`sqlConnectionString`	the full ADO.NET connection string from 3.2	yes
`functionKey`	`pending` for now; replaced in section 8	yes

3. **Pipeline permissions** → **+** → allow the pipeline (after it exists in section 6), or answer *Permit* on the first run.
4. **Save**.

5. Sync GitHub to Azure DevOps

The repository already contains `.github/workflows/mirror-to-azure-devops.yml`. On **every push to any branch** it runs `git push --mirror` to Azure Repos. GitHub stays the place where the team works, and Azure Repos is an exact copy that Azure Pipelines builds from.

5.1 Create the mirror PAT

1. Azure DevOps → avatar (top right) → **Personal access tokens** → **New Token**.
2. Name `github-mirror`. Organisation `ST10438928`. Expiry: end of the module.
3. Scopes → **Custom defined** → **Code: Read & write** only. **Create**, then copy the token immediately (it is shown once).

5.2 Add the GitHub secrets

1. GitHub → repository → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. Add:

Secret	Value
`AZURE_DEVOPS_REPO_URL`	the HTTPS clone URL from 4.1 (without `user@`)
`AZURE_DEVOPS_PAT`	the token from 5.1
`AZURE_ARTIFACTS_PAT`	a token with **Packaging: Read & write** (only if not already set in Part 2)

5.3 Test the sync

1. Make any small commit on GitHub (for example, edit the README on a feature branch) and push it.
2. GitHub → **Actions** → *Mirror to Azure DevOps* should be green within a minute.
3. Azure DevOps → **Repos** → **Commits**: the same commit appears with the same SHA.

> **Never commit directly in Azure Repos.** The mirror overwrites it on the next GitHub push and deletes anything that exists only there. All work goes through GitHub pull requests.

Alternative (if the PAT route is blocked): create the pipeline straight from GitHub. In 6.1 choose **GitHub** instead of *Azure Repos Git* and install the Azure Pipelines GitHub app. The pipeline then triggers from GitHub directly and the mirror is only a backup copy.

6. Create the pipeline

6.1 First run, build and test only

1. **Pipelines → New pipeline → Azure Repos Git** → `GiftOfTheGivers-HumanitarianAid` → **Existing Azure Pipelines YAML file** → branch `main`, path `/azure-pipelines.yml` → **Continue** → **Run**.
2. Rename it `GiftOfTheGivers-CI-CD` (⋮ → **Rename/move**).
3. If the run asks to **authorise** the `giftgivers-helpers` feed or other resources, click **Permit**.
4. With `deployEnabled: 'false'` (the default) only the **Build and test** stage runs. Check that:
   - the **Tests** tab shows three runs: *Unit tests*, *Integration tests*, *Selenium UI tests*, all passing;
   - the **Code Coverage** tab shows the merged report, and the *Coverage gate* step prints at least 70%;
   - the **Artifacts** list has `drop`, `migrations` and `ui-screenshots`.

If restore fails with *401 Unauthorized* for `giftgivers-helpers`: **Artifacts → giftgivers-helpers → ⚙ Feed settings → Permissions → Add users/groups** → add `<Project> Build Service (ST10438928)` as **Feed and Upstream Reader (Collaborator)**.

6.2 Turn on deployment

1. On GitHub, open `pipelines/templates/variables.yml` and set:
   - `deployEnabled: 'true'`
   - `webAppName`, `webAppUrl`, `resourceGroup`, `functionAppName`, `sqlServerName`, `sqlDatabaseName` to the names you wrote down;
   - `useStagingSlot: 'true'` and `webAppStagingUrl` **only** if you created the staging slot in 3.3;
   - `webAppType: webApp` and `functionAppType: functionApp` for Windows (the defaults).
2. Commit through a pull request into `main`. When it merges, the mirror copies it and the pipeline starts.
3. The run pauses at **DeployDatabase** waiting for approval (4.3). Open it → **Review → Approve**.
4. Watch the stages: *DeployDatabase* → *DeployWebApp* + *DeployFunctions* → *LiveSmokeTests*.

The first *DeployFunctions* run fails at **Test the deployed function**, because the function key is still `pending`. That is expected. Do section 8, then re-run that stage.

7. What each deployment stage does

Stage	Steps	Evidence for the report
Build and test	restore (locked) → build → unit → integration → Selenium (Chrome, desktop + mobile) → publish TRX → publish coverage → coverage gate ≥ 70% → vulnerability scan → EF model-drift check → `migrate.sql`, `rollback.sql`, `schema-version.txt` → zip web app and function	Tests tab, Code Coverage tab, artifacts
DeployDatabase	approval → show target schema version → run idempotent `migrate.sql` on Azure SQL → list `__EFMigrationsHistory`	the two SQL task logs (applied migrations are listed)
DeployWebApp	apply app settings + connection string → deploy zip (to `staging` if enabled) → smoke test → swap → smoke test production; **on failure: swap back**	deployment log, smoke-test HTTP codes
DeployFunctions	deploy zip → apply settings (connection string from the secret group) → call the function three ways: no key (401), bad reference (400), unknown reference (404)	the three HTTP results in the log
LiveSmokeTests	Selenium tests tagged *Smoke* against the live URL	Tests tab (*Live smoke tests*), `live-screenshots` artifact

8. After the first deployment

8.1 Function key

1. Portal → Function App `gift-relief-functions` → **Functions** → **GenerateTaxCertificate** → **Function Keys** → copy `default`.
2. Azure DevOps → **Library** → **giftgivers-azure** → set `functionKey` to that value (keep it secret) → **Save**.
3. Open the last run → **DeployFunctions** → **Rerun failed jobs**. It should now pass.
4. Manual test in a browser (replace the key): `https://gift-relief-functions.azurewebsites.net/api/tax-certificate/SEED000000000001?code=<key>` downloads the demo certificate PDF while demo data is on.

8.2 Check the live site

1. Open the web app URL. You should see the intro animation and then the home page.
2. Visit `/health`. It should say `Healthy` (this proves the database connection works).
3. Log in with the demo employee `employee@giftofthegivers.org` / `Employee#123` (present because `seedDemoData: 'true'`).

8.3 First real staff account

Before real users arrive:

1. Set `seedDemoData: 'false'` in `variables.yml` (removes demo seeding on new databases).
2. Register the real staff member on the live site.
3. App Service → **Environment variables** → add `StaffAccess__BootstrapEmployeeEmails__0` = that email → **Apply** (the app restarts and grants the Employee role).
4. Delete the demo accounts from **Staff Access** in the app.

9. Rollback and restore

What broke	How to roll back	Time
Web app, with a staging slot	Automatic: if a smoke test fails after the swap, the pipeline swaps back. Manually: portal → App Service → **Deployment slots** → **Swap** (staging ↔ production). The previous build is always in `staging` after a swap.	~1 min
Web app, no slot	Pipelines → **Runs** → open the last good run → **DeployWebApp** → **Rerun stage**. It redeploys that run's own `webapp.zip`.	~3 min
Function	Re-run **DeployFunctions** from the last good run.	~2 min
Database schema (latest migration is bad)	**Run pipeline** → tick **Roll the database back by one migration** → Run. The DB stage then runs `rollback.sql` (the `Down()` of the newest migration) instead of `migrate.sql`, and web/function deployment is skipped. Then revert the commit on GitHub.	~3 min
Database data (rows deleted or corrupted)	Portal → SQL database → **Restore** → *Point-in-time* → pick a time before the incident → restore as a new database (e.g. `GiftOfTheGiversDb-restored`) → check it → update `sqlConnectionString` in the variable group → re-run DeployWebApp and DeployFunctions. Basic tier keeps 7 days of backups.	15–30 min

The rollback and re-apply of the newest migration was rehearsed on SQL Server on 2026-11-02 (see the Part 3 report, section 6.3), and the CI workflow repeats it on every push.

10. Troubleshooting

Symptom	Cause and fix
Mirror workflow: `AZURE_DEVOPS_PAT is not configured`	Add the GitHub secrets (5.2). The names are case-sensitive.
Mirror workflow: `Authentication failed` / 401	PAT expired, revoked, wrong organisation or missing *Code: Read & write*. Create a new one.
Pipeline: *variable group giftgivers-azure could not be found*	The name is spelled differently, or the pipeline is not authorised: Library → group → **Pipeline permissions**.
Pipeline: *The pipeline is not valid … service connection*	Service connection name must equal `azureServiceConnection` in `variables.yml`, and it must be authorised for the pipeline.
Restore 401 on `giftgivers-helpers`	Give the Build Service identity *Feed Reader* (6.1).
SQL task: *Cannot open server … requested by the login* or firewall error	Turn on *Allow Azure services* on the SQL server (3.2, step 7). The task also adds a temporary rule for the agent IP. The service connection needs Contributor on the resource group.
Site shows **HTTP Error 500.30**	The app failed to start. App Service → **Log stream**. Usually a wrong connection string (check the *Connection strings* tab) or the database not migrated (re-run DeployDatabase).
Site still shows the Azure placeholder page	The DeployWebApp stage did not run (check `deployEnabled`, branch `main`, approval) or it deployed to `staging` without swapping.
`/health` returns `Unhealthy`	The database is unreachable: firewall, password, or the Basic database is paused (serverless).
Function smoke test 401 on every call	`functionKey` is still `pending` or wrong (section 8.1).
Function returns 500	Function App → **Monitor** / Application Insights. Usually `ConnectionStrings__DefaultConnection` is missing: re-run DeployFunctions.
Live Selenium smoke tests fail but the site works	The site was still warming up. Re-run the stage.

11. Hand-over checklist

Tick each item and keep the values in the team password manager.

✓	Item	Value / where
☐	Old shared PAT revoked	Azure DevOps → Personal access tokens
☐	Resource group	`rg-giftofthegivers`
☐	SQL server / database / admin login	`gift-relief-sql` / `GiftOfTheGiversDb` / password manager
☐	Web app name, URL, plan tier	portal Overview
☐	Staging slot created (S1+ only)	yes / no
☐	Function app name	`gift-relief-functions`
☐	Azure Repos clone URL	GitHub secret `AZURE_DEVOPS_REPO_URL`

✓	Item	Value / where
☐	Mirror PAT (Code R/W), expiry date	GitHub secret `AZURE_DEVOPS_PAT`
☐	Service connection	`giftgivers-azure-connection`
☐	Environment + approver	`giftgivers-production`
☐	Variable group with 4 secrets	`giftgivers-azure`
☐	Pipeline created and renamed	`GiftOfTheGivers-CI-CD`
☐	`deployEnabled: 'true'` merged to `main`	commit SHA
☐	Function key stored	`functionKey`
☐	Screenshots collected for the PoE	see the Part 3 report, section 10

12. Costs and clean-up

- Azure SQL Basic costs about US$5 a month. A Consumption function is free at this volume. The App Service cost depends on the existing plan (Free F1 = free; B1 ≈ US$13 a month; S1, needed for slots, ≈ US$70 a month, so use it only for the demonstration).
- After marking, delete the resource group to stop all charges: portal → `rg-giftofthegivers` → **Delete resource group**. Revoke the PATs as well.

Gift of the Givers · Relief Management System

DELIVERABLE 12

Team Contribution Report

Roles, responsibilities and evidence of each member's contribution, Parts 1–3.

DOCUMENT	VERSION	DATE	STATUS
D-12	0.9	2026-11-02	DRAFT — names, student numbers and roles to be completed and agreed by every member before submission

Members and roles

Each member confirms their own row. The git identities are taken from the repository history (section *Evidence*).

Member (name, student number)	Git identity	Role(s)	Main responsibilities and deliverables
[Full name, ST number]	`jBlack-MC`	*[e.g. Lead developer]*	*[e.g. web app features (donation flow, tax-certificate PDF, ERD alignment), most pull requests]*
Clarity Masuku, ST10438928	`MPCwGamer`, `ST10438928@rcconnect.edu.za`	*[e.g. DevOps / deployment lead]*	*[e.g. initial Azure Pipelines set-up (commit 2277bdf), Azure DevOps organisation and Artifacts feed]*
[Full name, ST number]	`Katlego-lab001`	*[e.g. Tester / documentation]*	*[e.g. co-authored 15 commits; testing, user manual]*
[Full name, ST number]	*[identity]*	*[role]*	*[responsibilities]*
[Full name, ST number]	*[identity]*	*[role]*	*[responsibilities]*

Delete unused rows. The brief allows at most 5 members.

Evidence (from `git log --all`, 2026-08-16 → 2026-11-02)

Measure	Value
Commits (all branches)	102, of which 36 are merge commits
Pull requests	15 merged into `main` (#1–#18)
Commits authored by `jBlack-MC`	86 (plus 3 from a second account, `MPCwGamer`)
Commits authored by Clarity Masuku	1 (2277bdf, *Set up CI with Azure Pipelines*)
`Co-Authored-By` trailers	Katlego-lab001 × 15, Clarity Masuku (MPCwGamer) × 13, jBlack-MC × 16
Automated agent commits	`copilot-swe-agent[bot]` × 11 (PRs #8, #10, #17); see the AI declaration in deliverable 09
Other	1 commit as `Student <student@example.com>` (5724267): the author must be identified

Reproduce these numbers with:

```
git shortlog -sne --all
git log --all --format='%(trailers:key=Co-Authored-By,valueonly)' | sort | uniq -c
gh pr list --state all
```

Contribution by Part 3 task

Tick who did each task, so the marker can link work to people.

Part 3 task	Member(s)
A.1 Unit tests (`DonationServiceTests`, existing unit suites)	*[ ]*
A.2 Integration tests (`EmployeeWorkflowTests`, `DonorPortalTests`, `TaxCertificateFunctionTests`)	*[ ]*
A.3 Load and stress tests (`GiftOfTheGivers.LoadTests`, caching mitigation)	*[ ]*
A.4 Selenium UI tests (`GiftOfTheGivers.UiTests`)	*[ ]*
B Pipeline with tests and coverage (`azure-pipelines.yml`, `pipelines/templates`)	*[ ]*
C.1–C.3 Continuous deployment (web app, function, SQL) and Azure set-up	*[ ]*
D User manual (deliverable 10)	*[ ]*
Azure deployment guide (deliverable 11) and this report	*[ ]*

Observations for the next iteration

- Most commit messages are short (*good*, *test*, *updates*). Use descriptive messages such as *Add Selenium test for anonymous USD donation* so each member's work can be read from the history.
- Work done in pairs should be committed by the person who typed it, with a `Co-Authored-By:` line for the partner.

Signed (each member): _____ _____ _____ _____ _____